09-visualize-layers

9 Layers

In chap 9 “Layers” you will learn about the layered grammar of graphics in R.

In chap 10 “Exploratory data analysis”, you’ll combine visualization with your curiosity and skepticism to ask and answer interesting questions about data.

Finally, in chap 11 “Communication” you will learn how to take your exploratory graphics, elevate them, and turn them into expository graphics, graphics that help the newcomer to your analysis understand what’s going on as quickly and easily as possible.

BEST PLACE to learn ggplot2 visuals? ggplot2: elegant graphics for data analysis

What if you’re trying to do something unique/difficult with ggplot2? https://exts.ggplot2.tidyverse.org/gallery/

9.1 Introduction

9.1.1 prerequisites

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.6
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.1     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.2.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```r
library(patchwork)
sessionInfo()
```

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.5.1

Matrix products: default
BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framewo
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/New_York
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] patchwork_1.3.2 lubridate_1.9.4 forcats_1.0.1   stringr_1.6.0
 [5] dplyr_1.1.4     purrr_1.2.0     readr_2.1.6     tidyr_1.3.1
 [9] tibble_3.3.0    ggplot2_4.0.1   tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] gtable_0.3.6       jsonlite_2.0.0     compiler_4.5.2     tidyselect_1.2.1
 [5] scales_1.4.0       yaml_2.3.11        fastmap_1.2.0      R6_2.6.1
 [9] generics_0.1.4     knitr_1.50         pillar_1.11.1      RColorBrewer_1.1-
3
[13] tzdb_0.5.0         rlang_1.2.0        stringi_1.8.7      xfun_0.54
[17] S7_0.2.1           timechange_0.3.0   cli_3.6.6          withr_3.0.2
[21] magrittr_2.0.4     digest_0.6.39      grid_4.5.2         rstudioapi_0.17.1
[25] hms_1.1.4          lifecycle_1.0.5    vctrs_0.6.5        evaluate_1.0.5
[29] glue_1.8.0         farver_2.1.2       rmarkdown_2.30     tools_4.5.2
[33] pkgconfig_2.0.3    htmltools_0.5.9
```

```r
# https://stackoverflow.com/questions/21967254/how-to-write-a-reader-friendly-sessioninfo-to
if (!file.exists("./session_info.txt")) {
  writeLines(capture.output(sessionInfo()), "sessionInfo.txt")
}
```

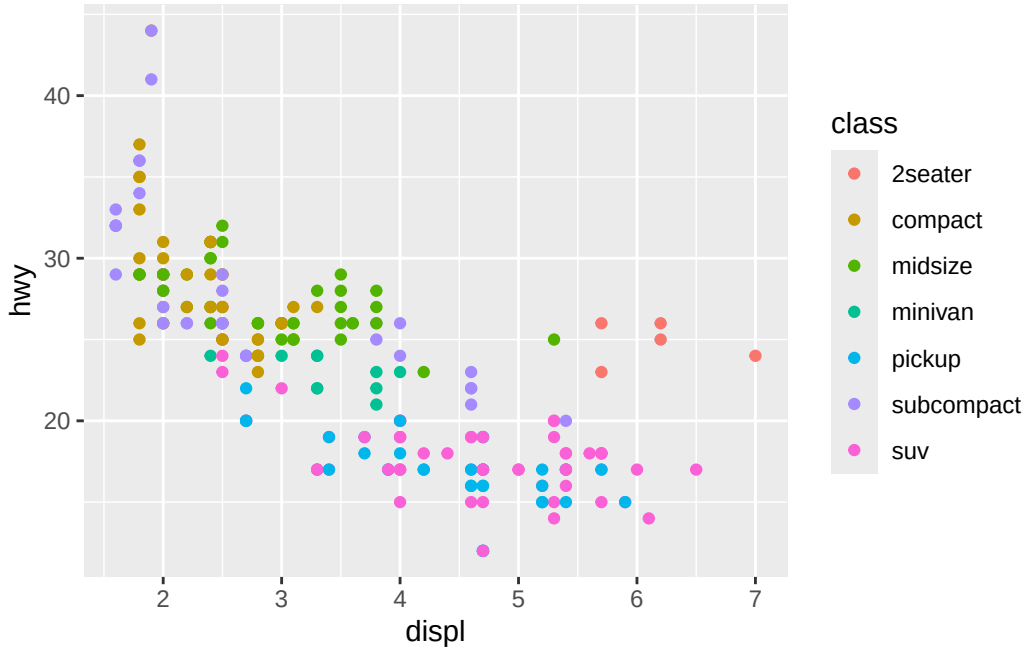
9.2 Aesthetic mappings

```
mpg
```

```
# A tibble: 234 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 audi         a4           1.8  1999     4 auto~ f        18    29 p     comp~
 2 audi         a4           1.8  1999     4 manu~ f        21    29 p     comp~
 3 audi         a4           2    2008     4 manu~ f        20    31 p     comp~
 4 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
 5 audi         a4           2.8  1999     6 auto~ f        16    26 p     comp~
 6 audi         a4           2.8  1999     6 manu~ f        18    26 p     comp~
 7 audi         a4           3.1  2008     6 auto~ f        18    27 p     comp~
 8 audi         a4 quattro   1.8  1999     4 manu~ 4        18    26 p     comp~
 9 audi         a4 quattro   1.8  1999     4 auto~ 4        16    25 p     comp~
10 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
# i 224 more rows
```

visualize relationship between `displ` (engine size) and `hwy` (highway mpg) for various `class`es of car

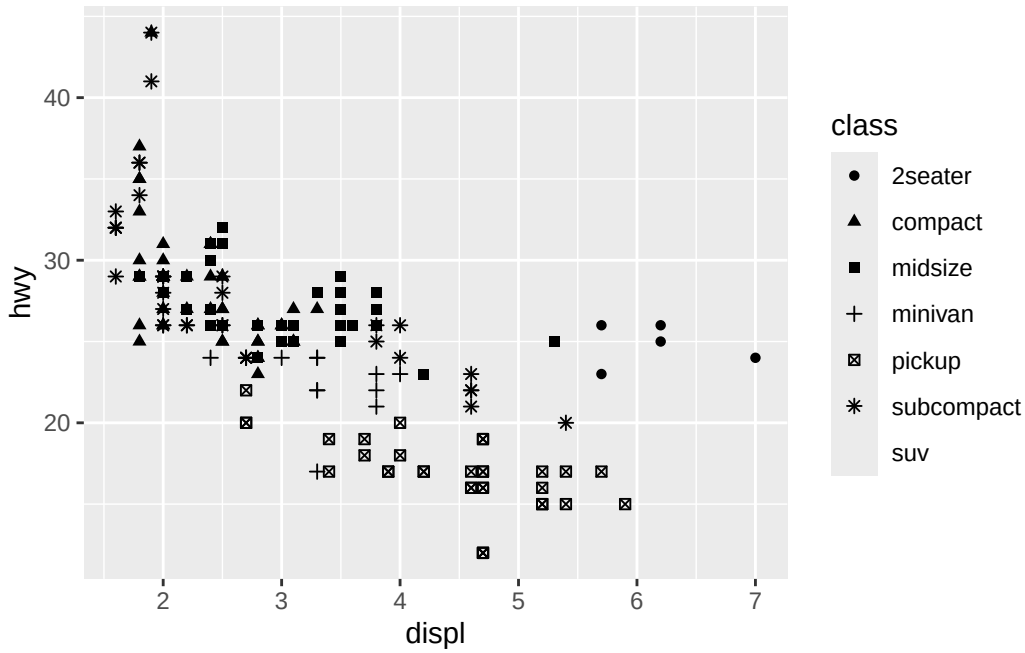
```
ggplot(mpg, aes(x = displ, y = hwy, color = class)) +
  geom_point()
```




```
ggplot(mpg, aes(x = displ, y = hwy, shape = class)) +
  geom_point()
```

```
Warning: The shape palette can deal with a maximum of 6 discrete values because more
than 6 becomes difficult to discriminate
i you have requested 7 values. Consider specifying shapes manually if you need
  that many of them.
```

```
Warning: Removed 62 rows containing missing values or values outside the scale range
(`geom_point()`).
```




```
#> Warning: The shape palette can deal with a maximum of 6 discrete values because more
#> than 6 becomes difficult to discriminate
#>   you have requested 7 values. Consider specifying shapes manually if you
#>   need that many of them.
#> Warning: Removed 62 rows containing missing values or values outside the scale range
#> (`geom_point()`).
```

the warnings for the plot above indicate:

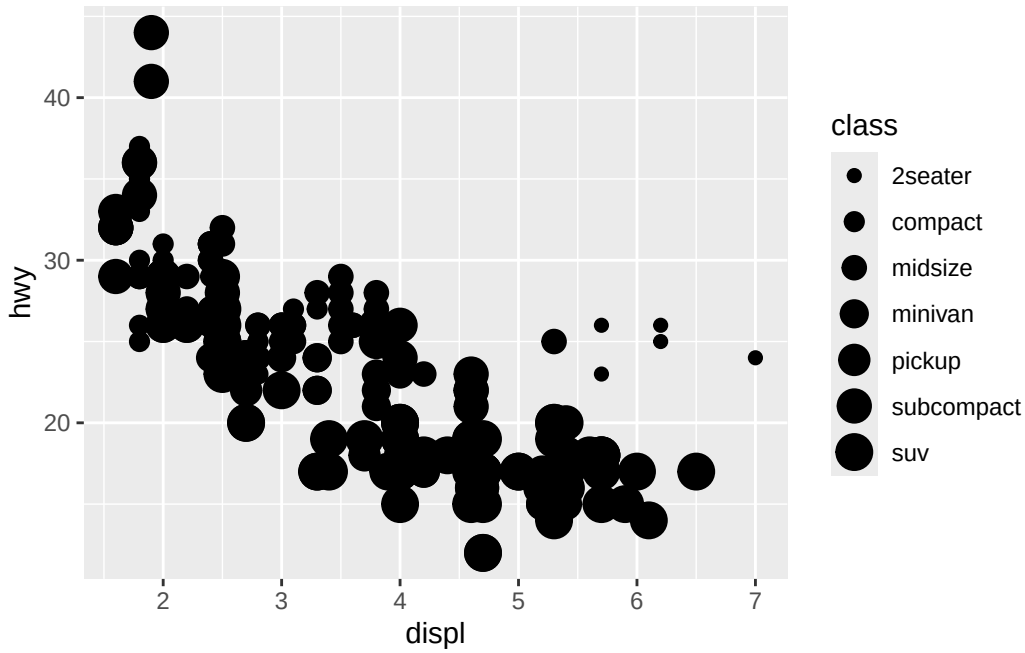
1. 62 rows were removed because of missing vals or vals outside geom_point() scale range
2. default shape pallete deals with a max of 6 shapes, specify manually if we want more

- both result in data removed and not plotted as result

let's try letting class determine the `size` of points and opacity (`alpha`)

```
ggplot(mpg, aes(x = displ, y = hwy, size = class)) +
  geom_point()
```

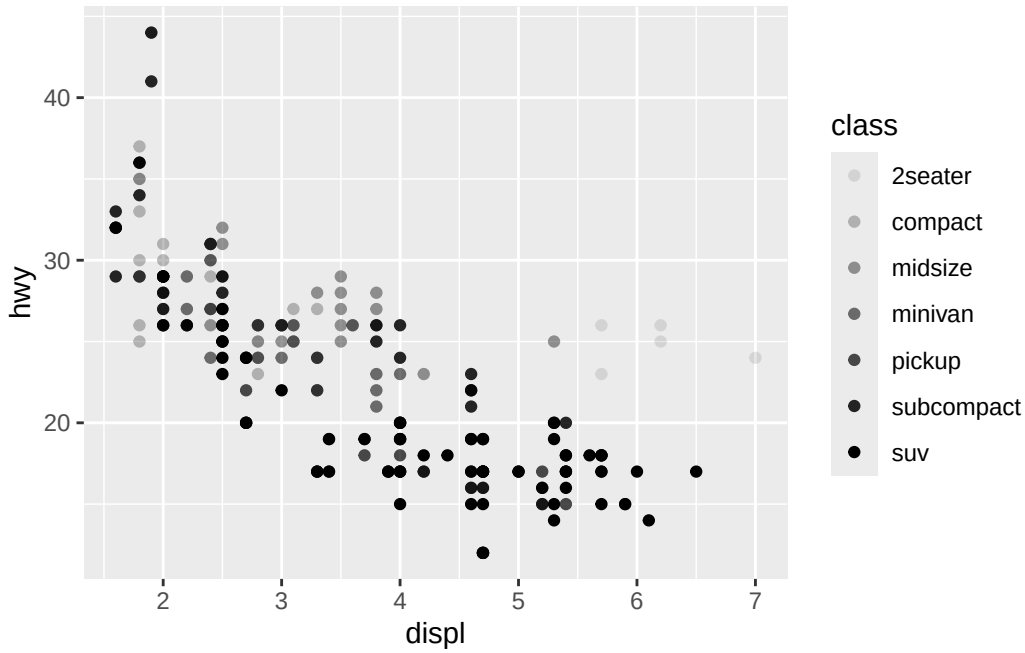
```
Warning: Using size for a discrete variable is not advised.
```




```
#> Warning: Using size for a discrete variable is not advised.
```

```
ggplot(mpg, aes(x = displ, y = hwy, alpha = class)) +
  geom_point()
```

```
Warning: Using alpha for a discrete variable is not advised.
```




```
#> Warning: Using alpha for a discrete variable is not advised.
```

key idea: mapping an unordered discrete/categorical var/class to an ordered aesthetic is generally BAD because this implies a ranking that does not exist (why is pickup smaller than a subcompact car or SUV, in the class legend for `size`?)

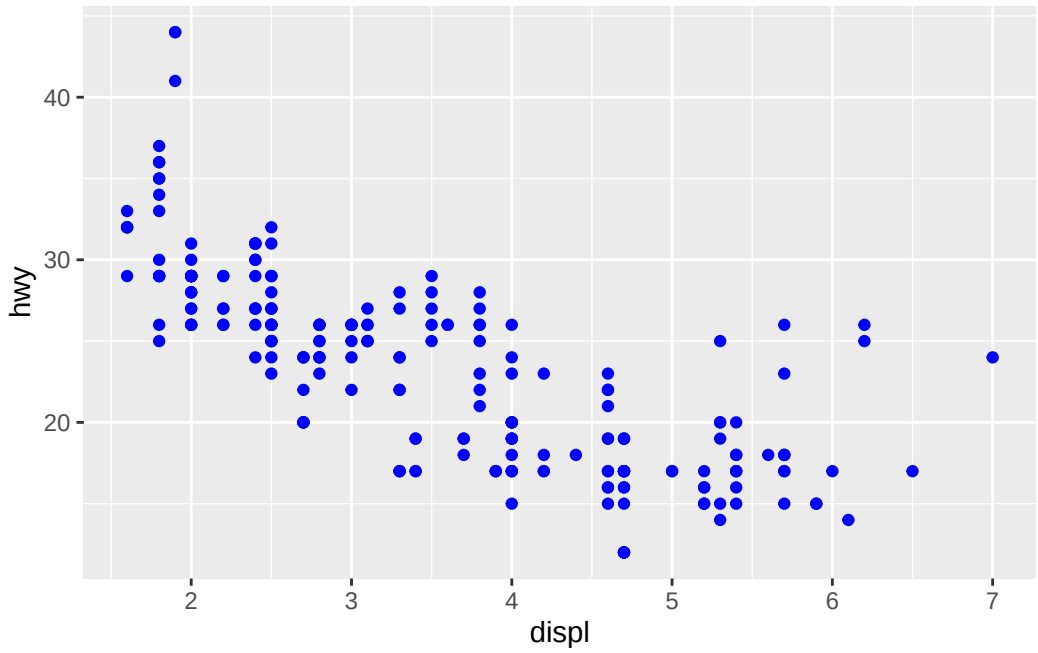
once aesthetics are mapped, ggplot2 handles everything:

- auto select reasonable scale to use with aes
- construct legend explaining mapping between levels and values
- for x and y aes, dont make legend but create axis line with tick marks and label
  - axis line provides SAME info as legend: explain mapping between location and value

can use geoms to adjust visual properties unrelated to data (outside the `aes`):

- `color` as character string or hexadecimal color code
- `size` of point in mm, e.g. `size = 1`
- `shape` of point as number, see figure 9.1 below

```
# make points in plot blue
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = "blue")
```




```
"blue"
```

```
[1] "blue"
```

```
"#f52a27"
```

```
[1] "#f52a27"
```

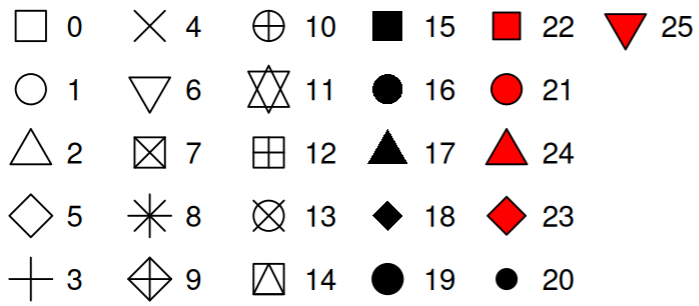


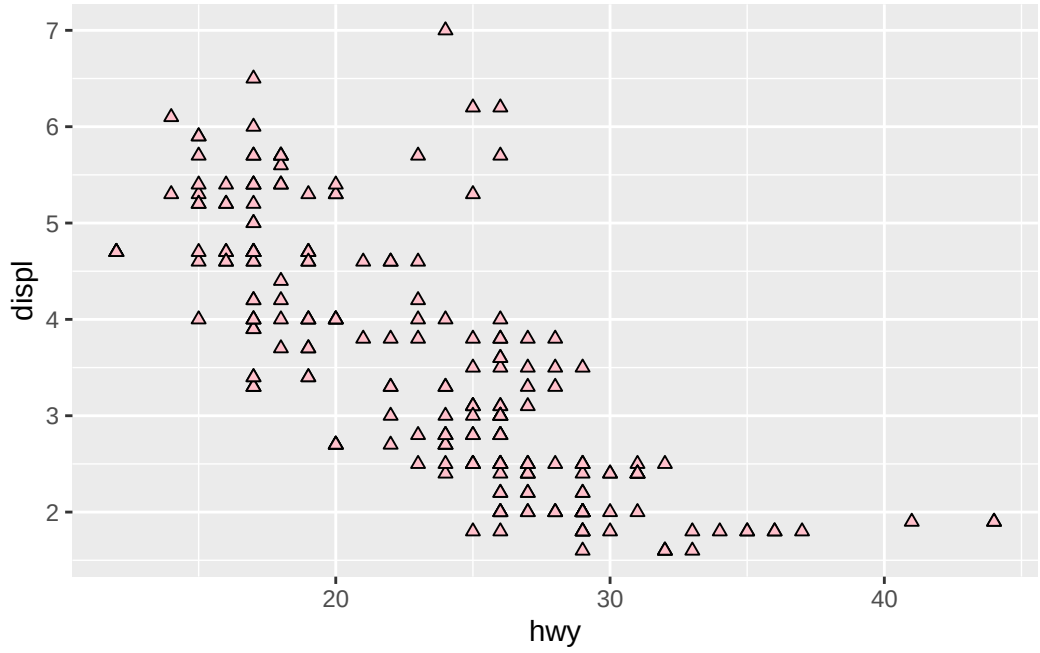
Figure 1: Figure 9.1 R's 26 built-in shapes. note, some are distinguished by color

9.2.1 Exercises

1. Create a scatterplot of `hwy` vs. `displ` where the points are pink filled in triangles.

- answer:

```
ggplot(mpg, aes(x = hwy, y = displ)) +
  geom_point(shape = 24, fill = "pink")
```

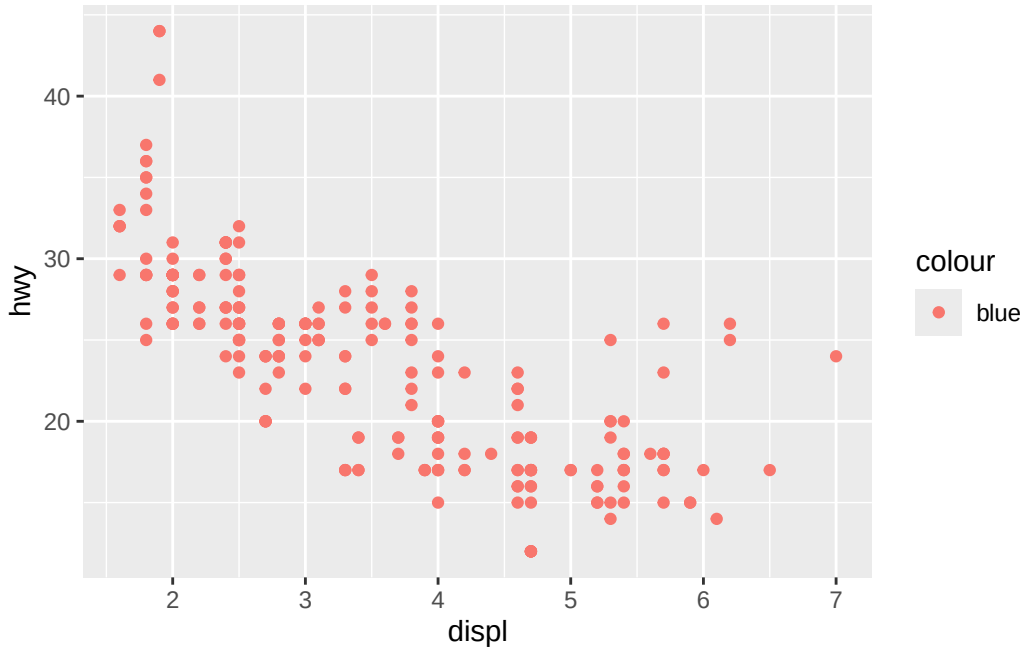


```
# geom_point(color = "pink", shape = 24, fill = "pink") # completely pink
```

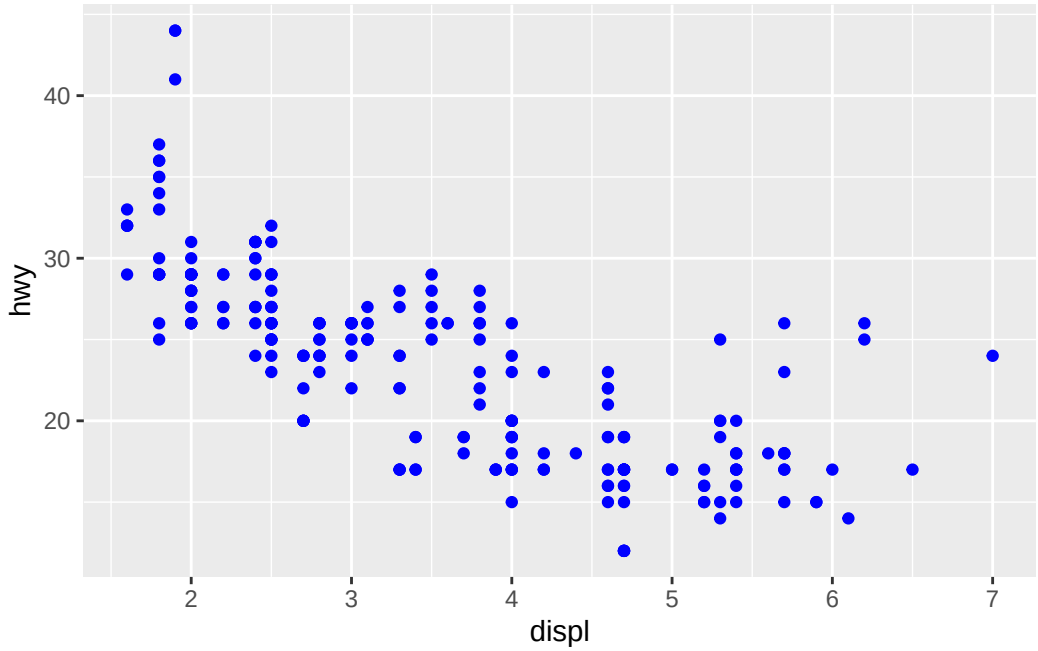
2. Why did the following code not result in a plot with blue points?

- answer: because `color` was passed to the `aes()` of `geom_point()` instead of directly to `geom_point()`. based on the experiments below, im guessing that color passed to the topmost level inside/outside aes doesnt make sense unless we are passing a variable in, since there is not anything to color yet (the `geom_point` doesnt exist yet). see answer chatgpt for an improved response :)

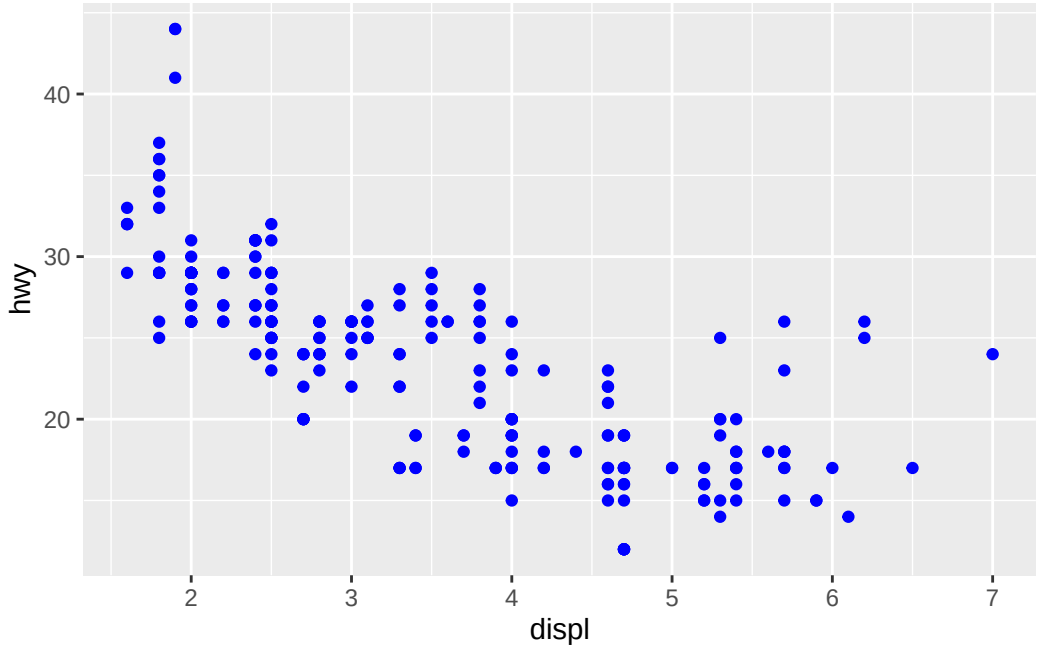
```
# this doesnt work. the color is overriden by default?
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy, color = "blue"))
```




```
# this works
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy), color = "blue")
```

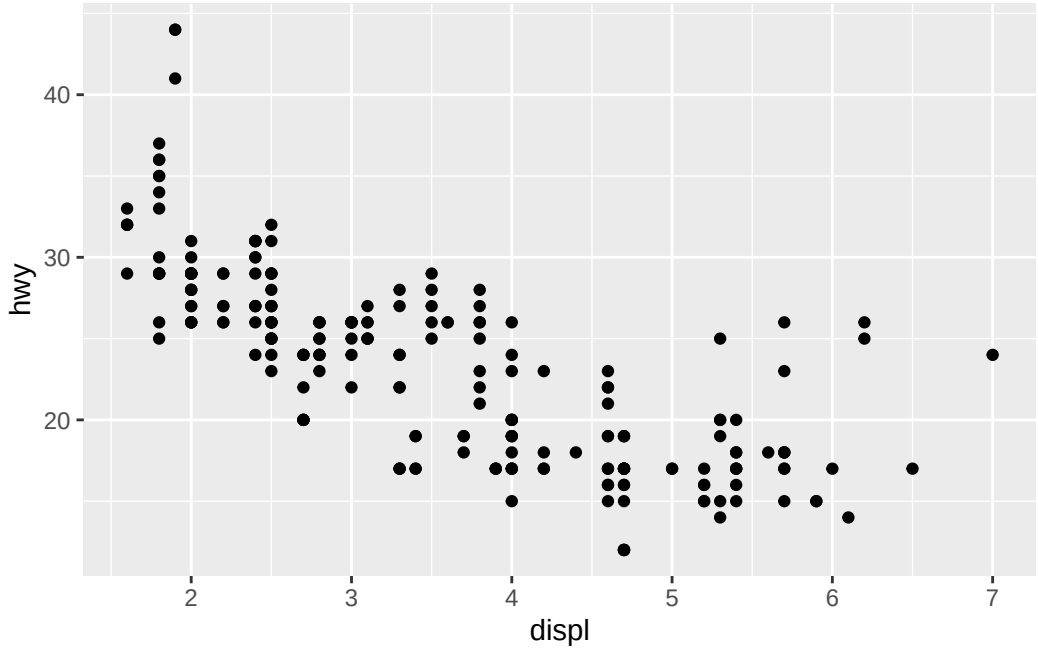



```
# this works, and is standard
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = "blue")
```

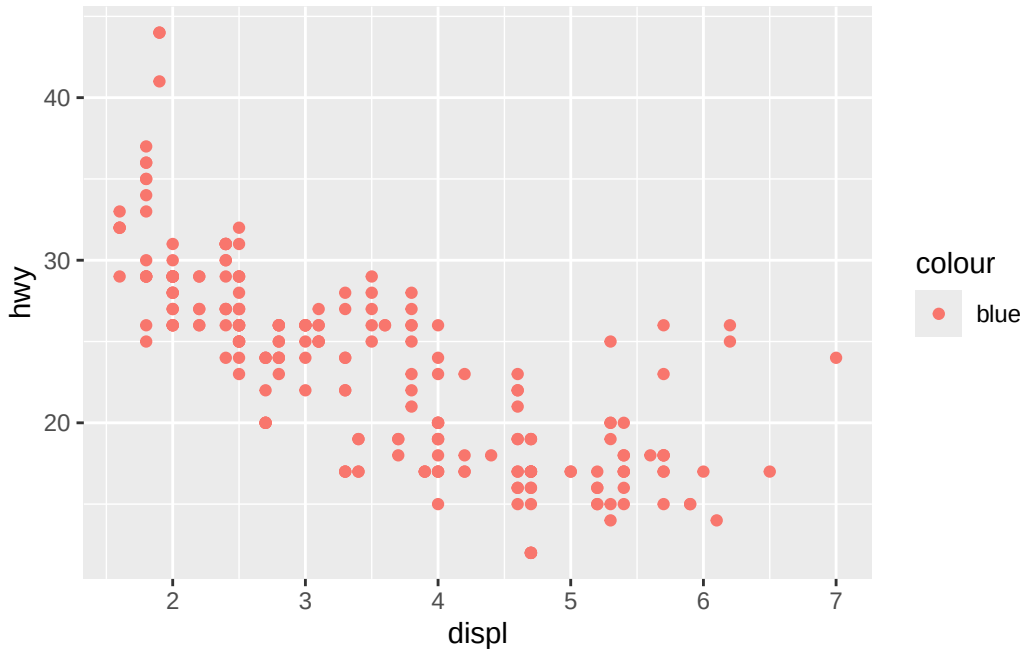


```
# what if we set the color at the topmost level outside aes()?
ggplot(mpg, aes(x = displ, y = hwy), color = "blue") +
  geom_point()
```

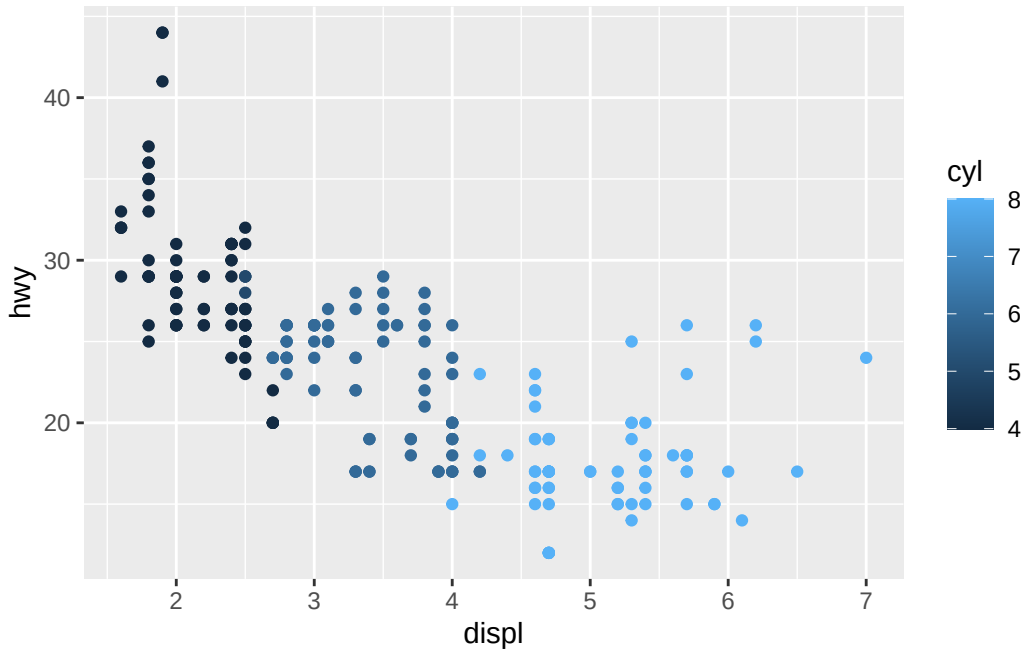
```
Warning in fortify(data, ...): Arguments in `...` must be used.
x Problematic argument:
* color = "blue"
i Did you misspell an argument name?
```



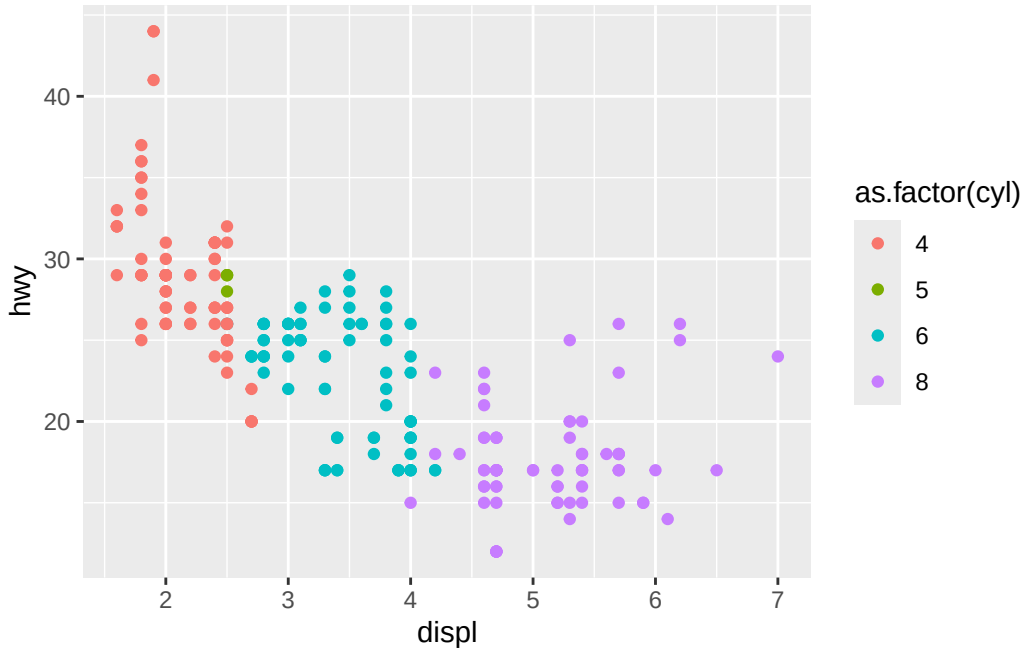

```
# what if we set the color at the topmost level inside aes()? answer: huh, it controls the le
ggplot(mpg, aes(x = displ, y = hwy, color = "blue")) +
  geom_point()
```




```
# ggplot(mpg, aes(x = displ, y = hwy, color = c("blue", "red"))) +
#   geom_point()

ggplot(mpg, aes(x = displ, y = hwy, color = cyl)) +
  geom_point()
```




```
ggplot(mpg, aes(x = displ, y = hwy, color = as.factor(cyl))) +
  geom_point()
```



- answer chatgpt: Yes. In ggplot2, anything you put inside aes() is treated as a mapped aesthetic, not a fixed value. The given code creates a color mapping where every point belongs to a category named "blue". ggplot then assigns that category a color from its default discrete color scale (which is not necessarily blue). To set the color directly, move it outside aes(). A useful rule of thumb:
  - Inside: aes() → map data to visual properties. O
  - Outside: aes() → set a constant visual property.

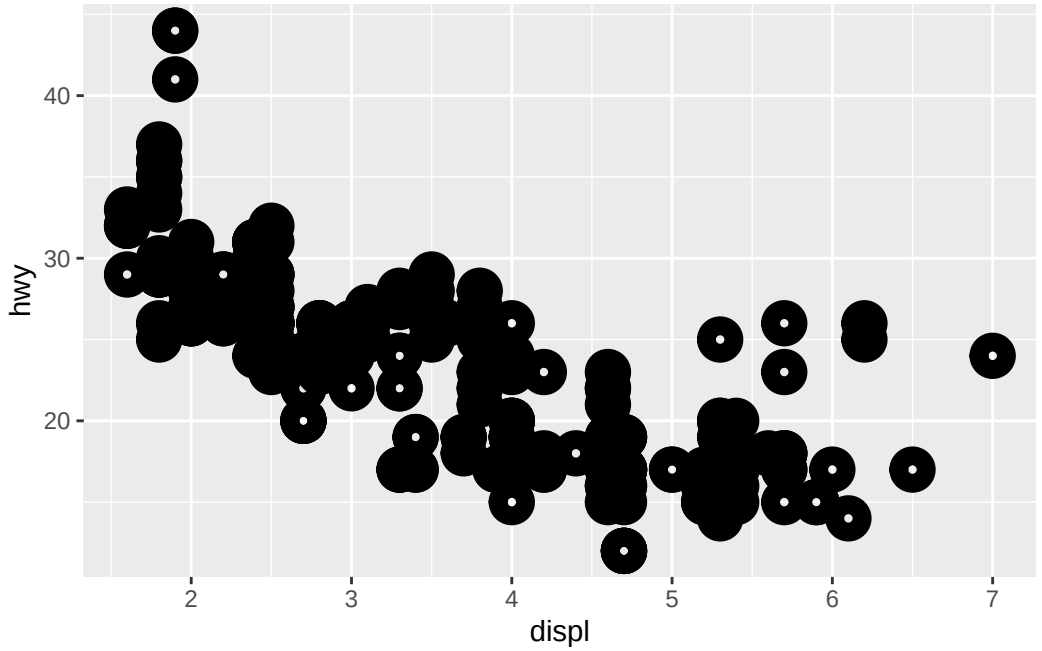
  For example:

```
# Mapping
aes(color = drv)     # color depends on data

# Setting
color = "red"        # every point is red
```

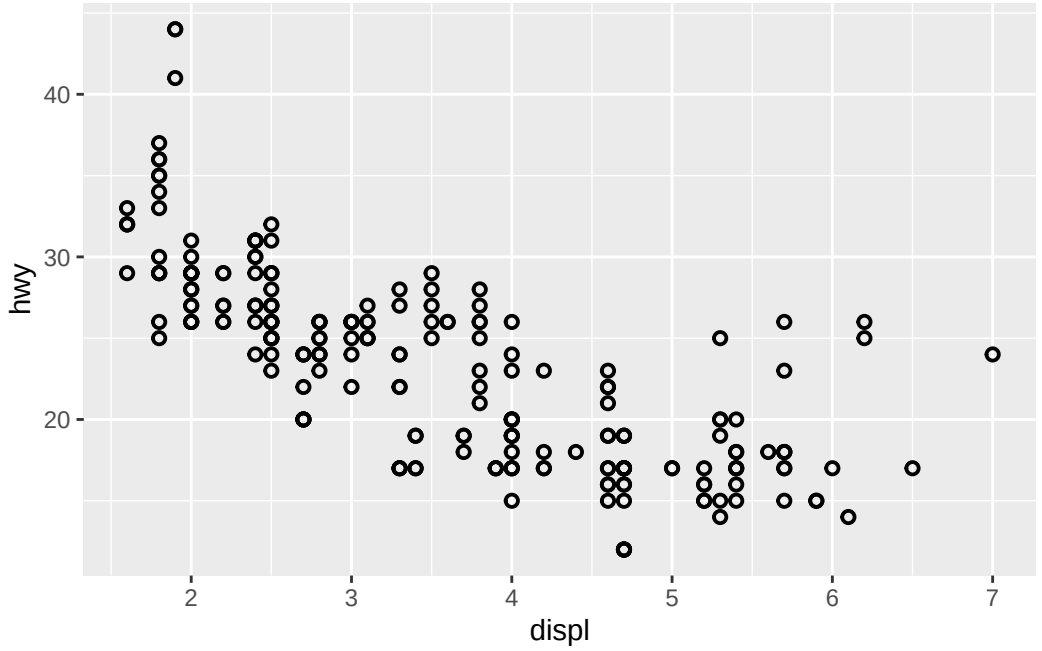
3. What does the `stroke` aesthetic do? What shapes does it work with? (Hint: use `?geom_point`)

- answer: for ggplot2_3.5.2, `stroke` is not documented very well by `?geom_point` nor by the associated vignette `vignette("ggplot2-specs")`. Based on the below, it appears to control width of the marker's border

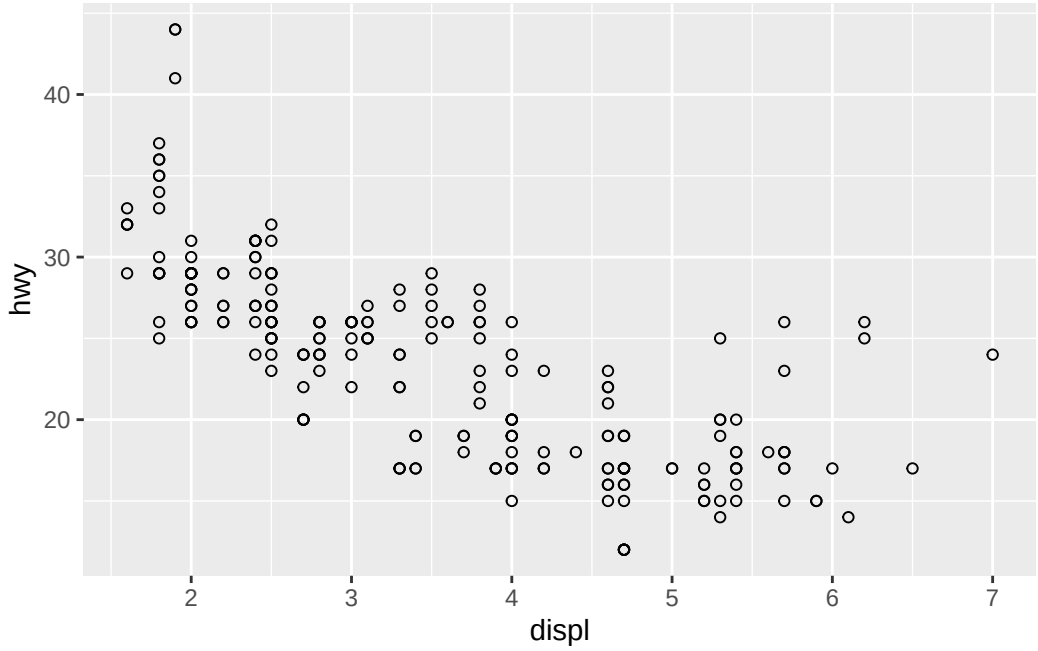
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(stroke = 5, shape = 1)
```




```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(stroke = 1, shape = 1)
```



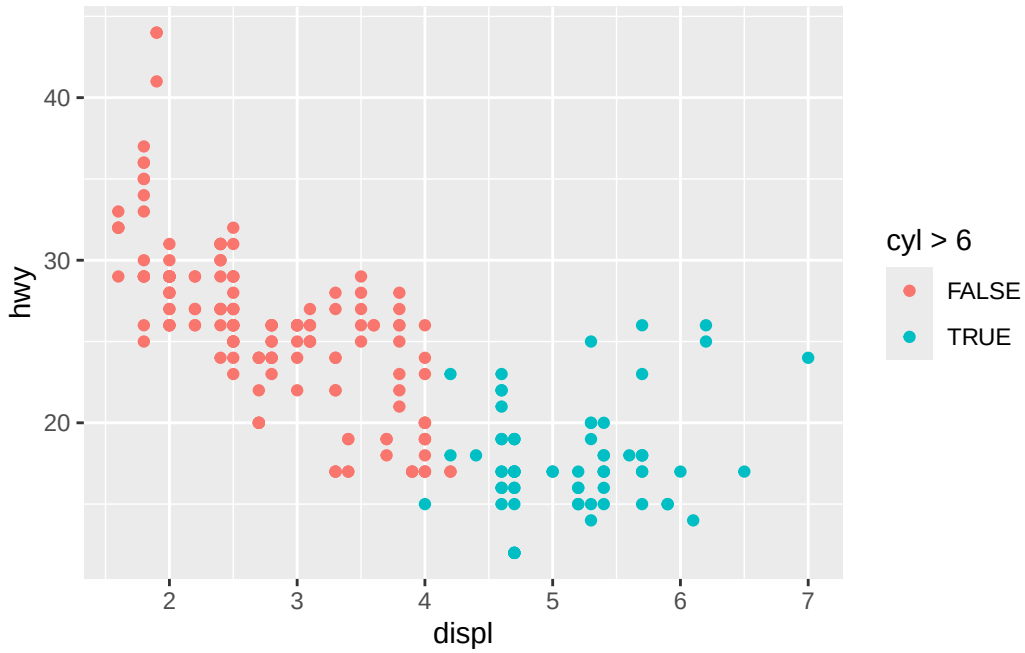
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(shape = 1)
```



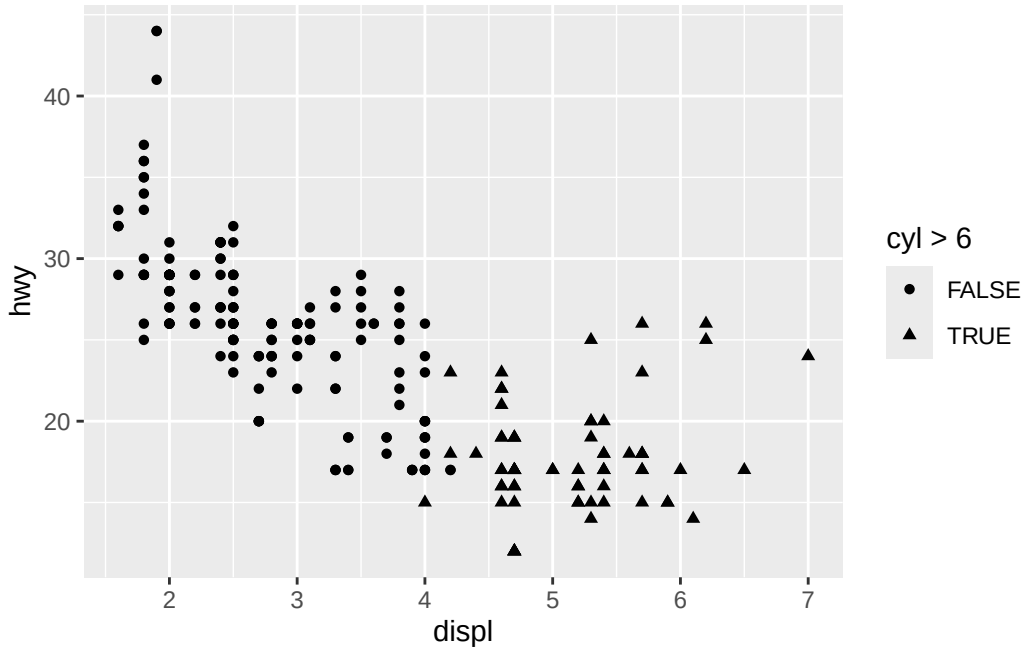
4. What happens if you map an aesthetic to something other than a variable name, like `aes(color = displ < 5)`? Note, you'll also need to specify x and y.

- answer: for a boolean expression, the legend gets split into `FALSE` and `TRUE`. we learned above a static value (like `color = "red"`) will cause the value to be applied globally

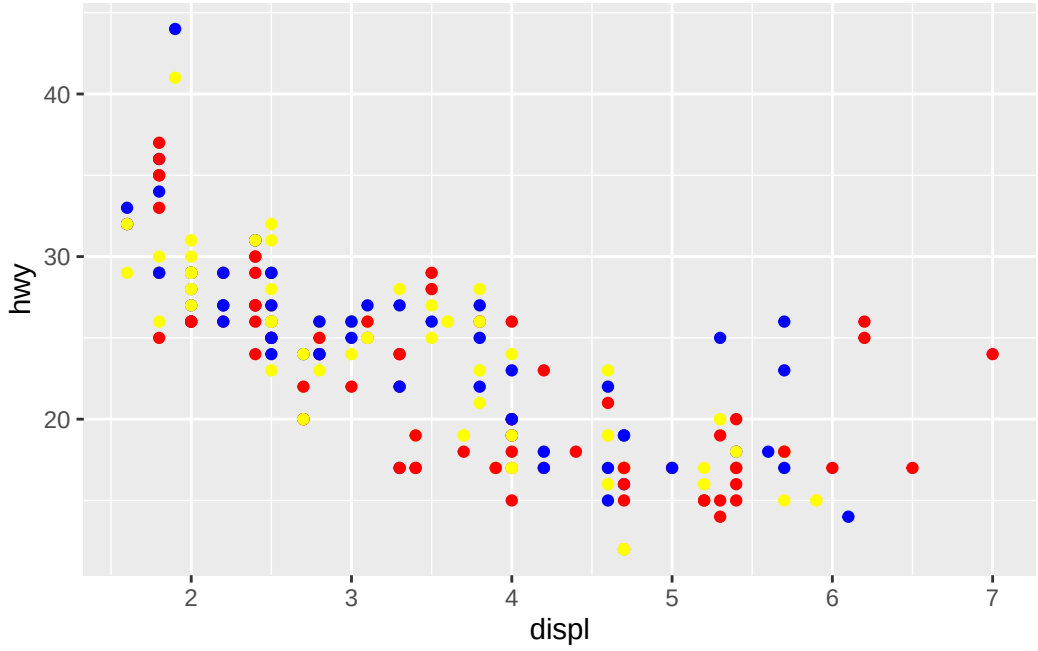
```
# note: unique(mpg$cyl) gives 4, 5, 6, 8

# expression to color
ggplot(mpg, aes(x = displ, y = hwy, color = cyl > 6)) +
  geom_point()
```




```
# expression to shape
ggplot(mpg, aes(x = displ, y = hwy, shape = cyl > 6)) +
  geom_point()
```



```r
# hard coded colors to color
selected_colors <- sample(
  x = c("red", "blue", "yellow"),
  size = length(mpg$hwy),
  replace = TRUE
)

ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = selected_colors)
```

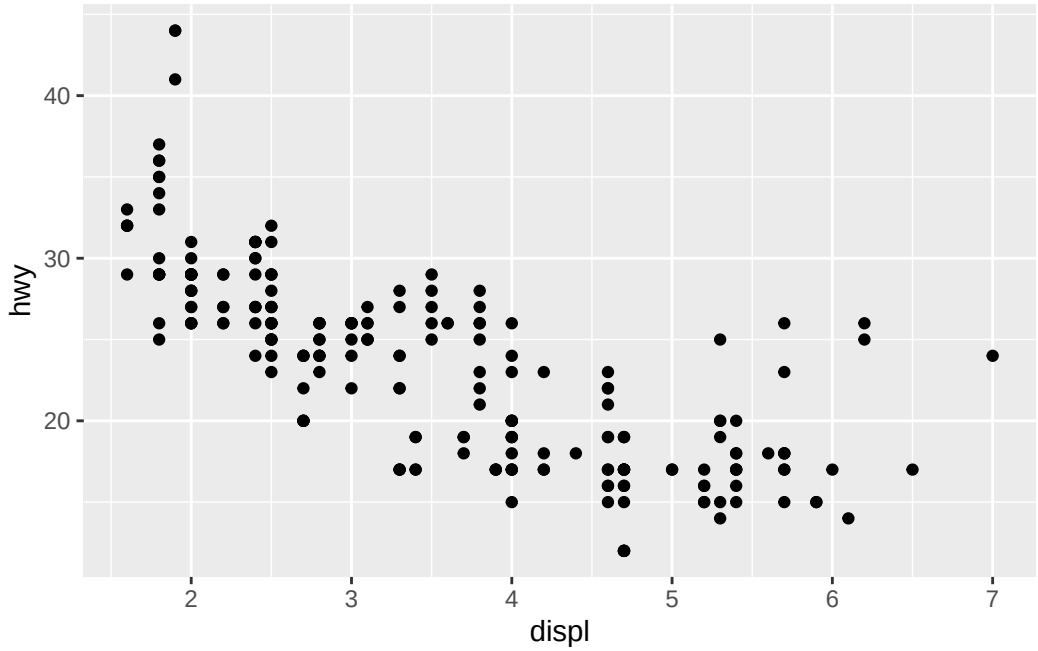



```r
rm(selected_colors)
```

9.3 Geometric objects (geoms)

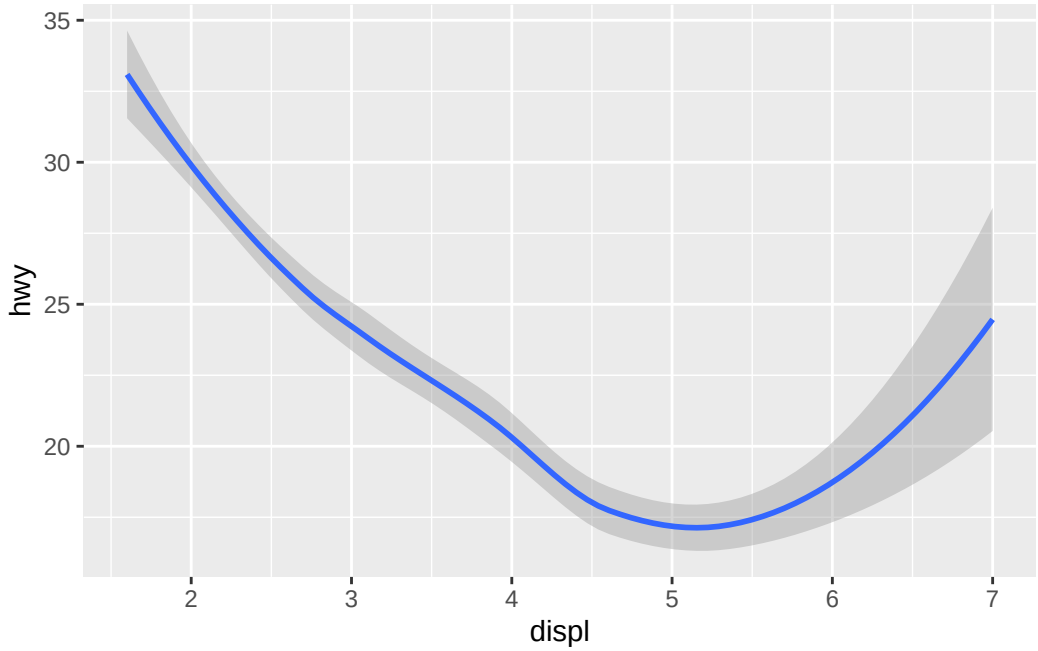
Two different geoms on the same data and ggplot base

```r
# Left
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point()
```




```
# Right
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

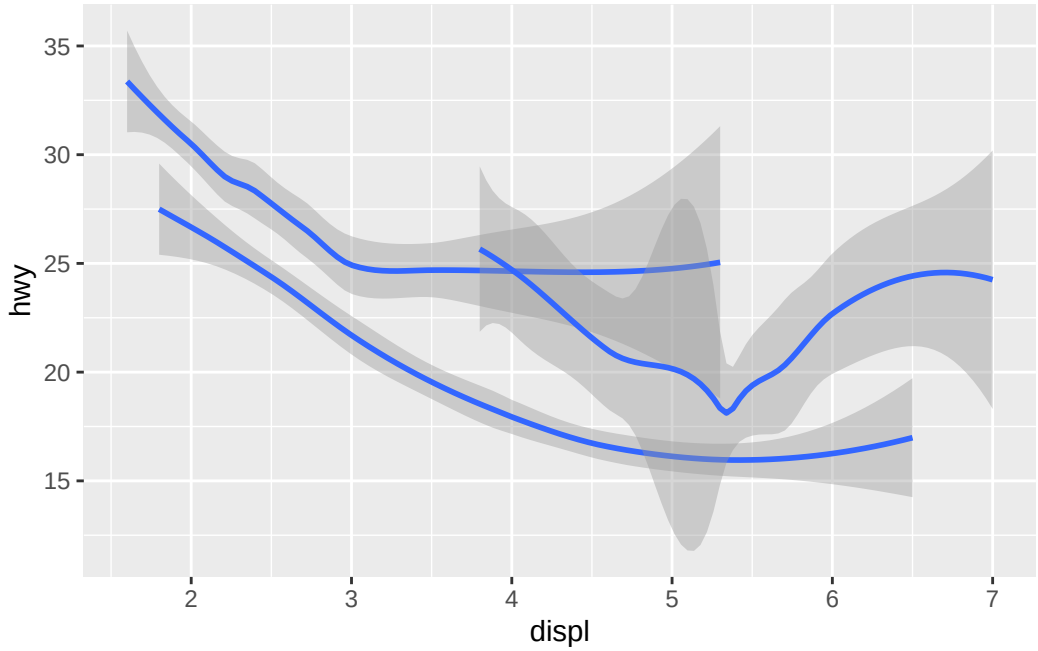



```
#> `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

- every geom func takes an `aes()` mapping (or inherits from top level `ggplot()`)
- but not every `aes()` works with each geom
  - if you specify an `aes()` that doesnt work with a geom, it will be silently ignored
- `geom_smooth()` will draw different line with different linetype for each unique variable mapped to linetype

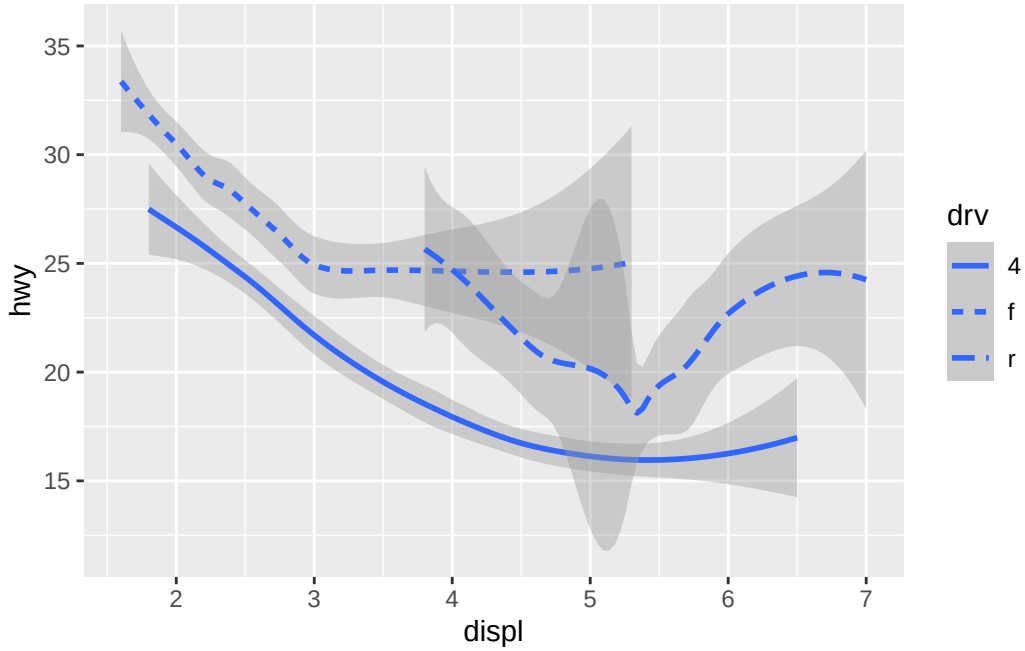
```
# Left - shape is ignored by geom_smooth
ggplot(mpg, aes(x = displ, y = hwy, shape = drv)) +
  geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
# Right - line type is accepted
ggplot(mpg, aes(x = displ, y = hwy, linetype = drv)) +
  geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

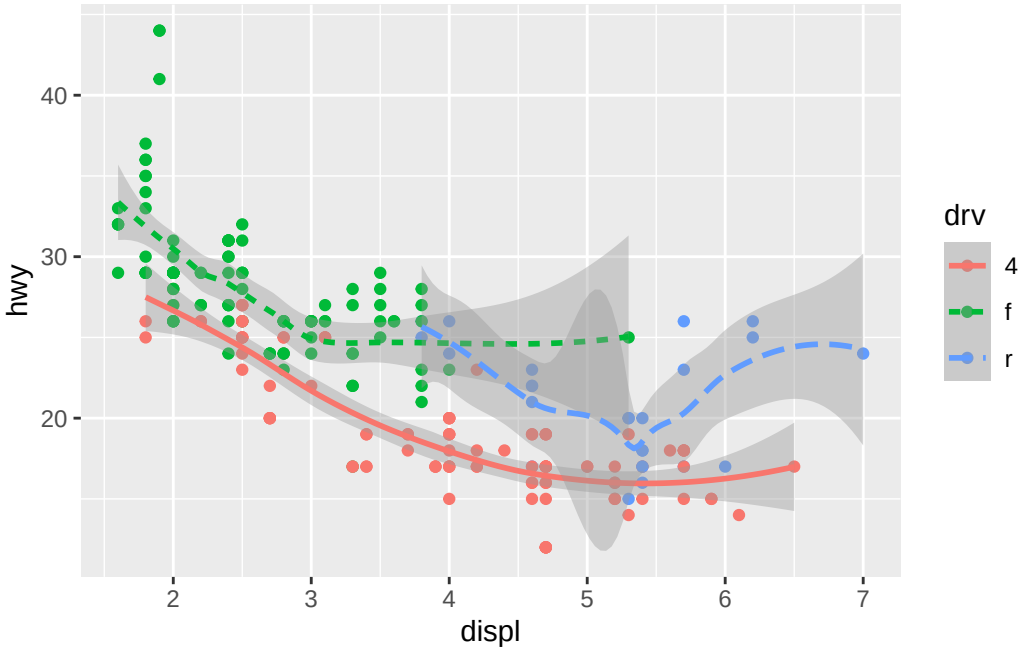


we can make the plot even clearer by:

- overlaying lines on respective data
- coloring everything by `drv`

```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  geom_smooth(aes(linetype = drv))
```

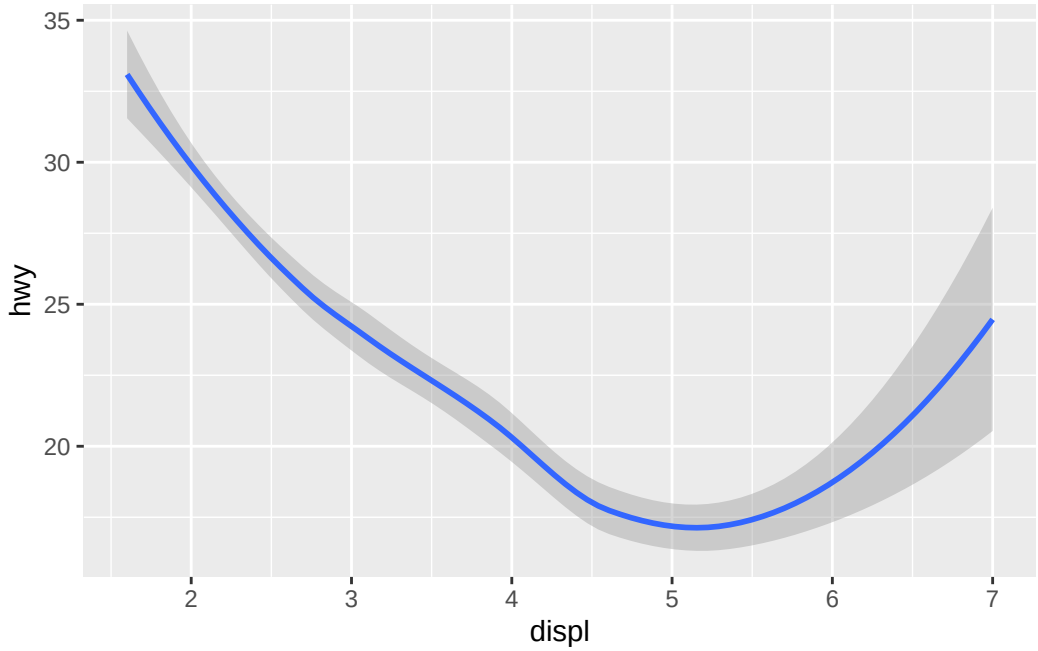
```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



- many `geom`s use single geometric object to display multiple rows from df
- can use `aes(group = ...)` to draw multiple objects
  - each unique value of grouping var will get a separate object (like the different lines for `drv` above)
  - ggplot2 will auto group data for `geom`s whenever `aes` is mapped to discrete var
    - this auto produces a legend, whereas `group =` does not

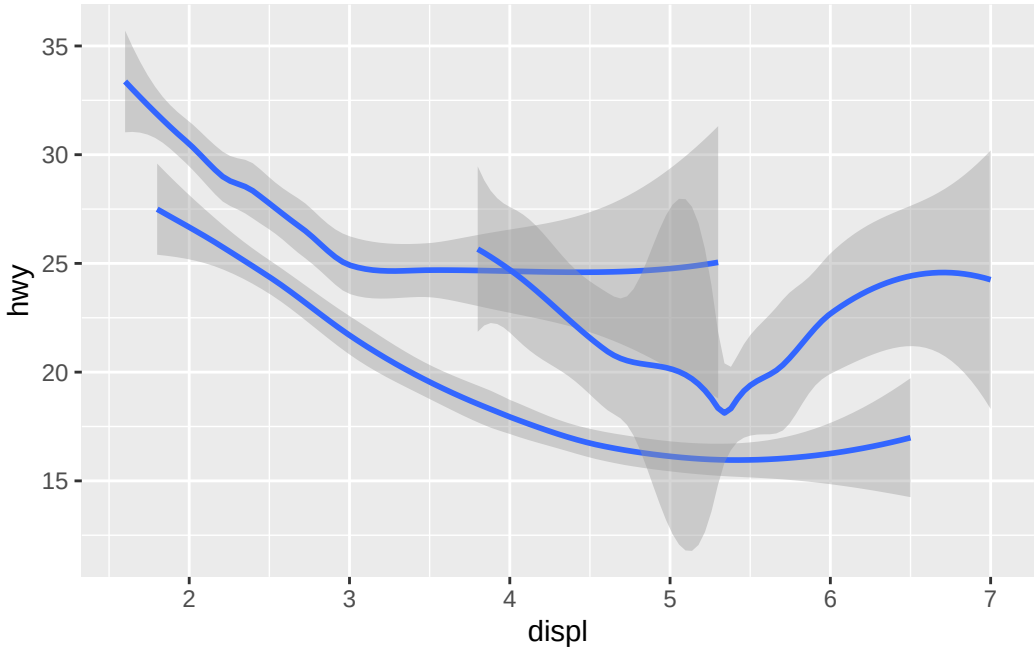
```
# Left
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



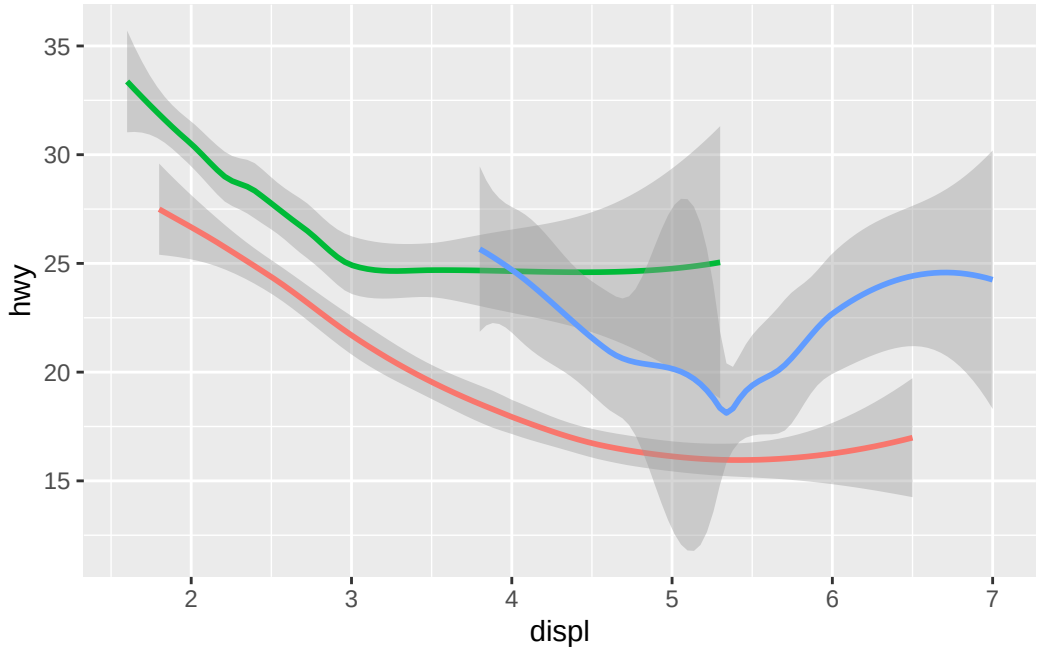

```
# Middle- generates 3 geom_smooth lines, no legend
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(group = drv))
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
# Right - generates 3 geom_smooth lines and legend (which can be suppressed), and colors the
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(color = drv), show.legend = FALSE)
```

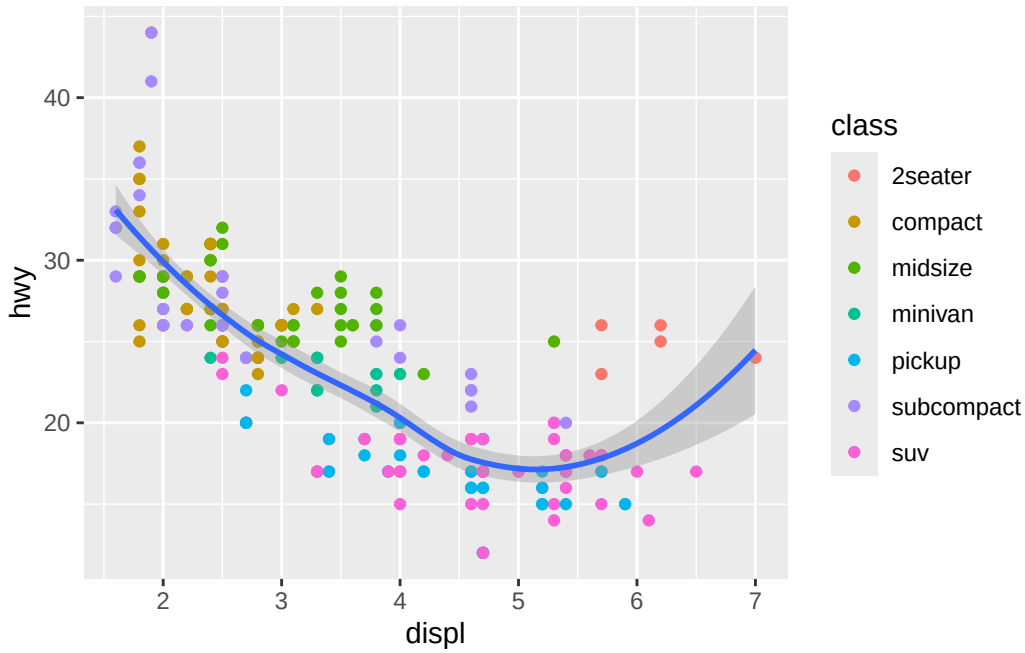
```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



- if mappings placed in geom and not top layer, ggplot2 will use them to extend or overwrite global mappings *for that layer only*

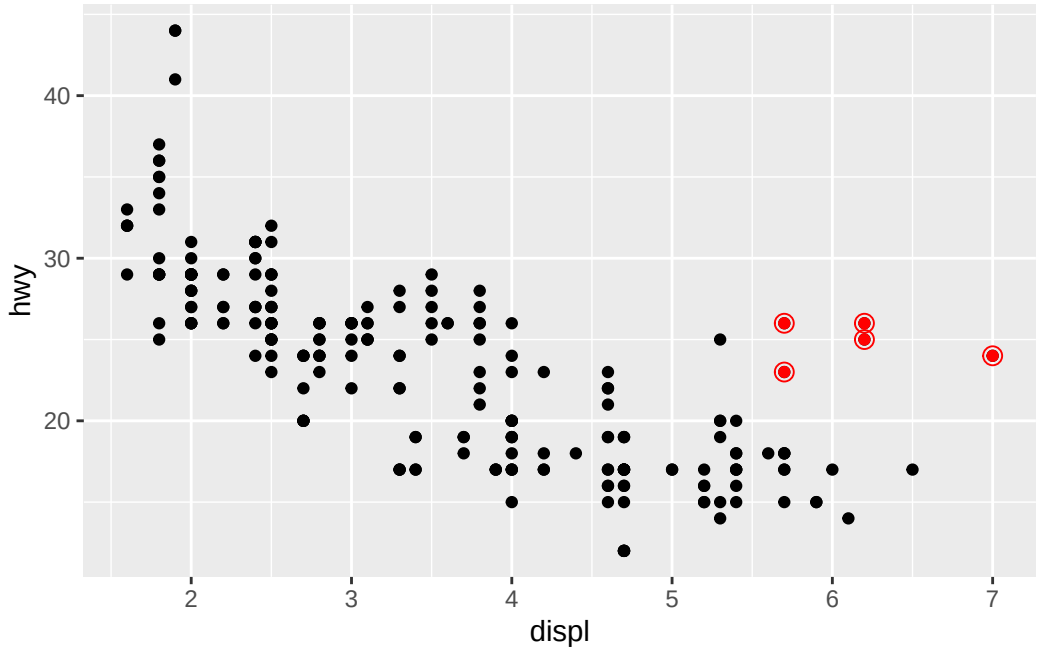
```
# leave color of line alone, but color the points by `class`
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



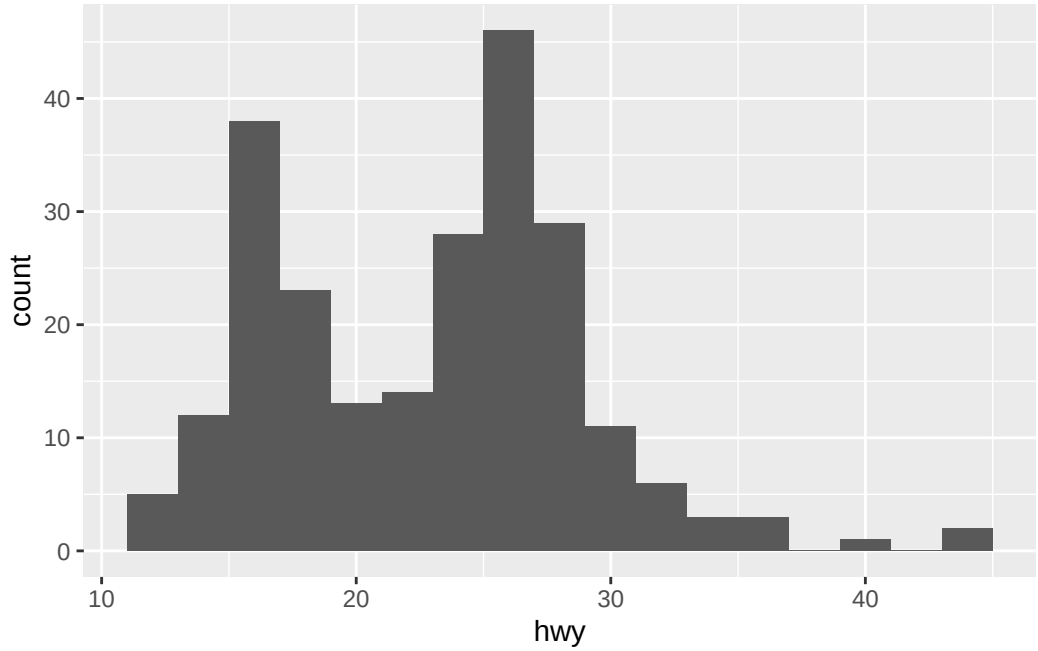
- same idea can be used to specify different `data` for each layer
- example below: red points and open circles to highlight 2-seater cars

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_point(
    data = mpg |> filter(class == "2seater"),
    color = "red"
  ) +
  geom_point(
    data = mpg |> filter(class == "2seater"),
    shape = "circle open", size = 3, color = "red"
  )
```

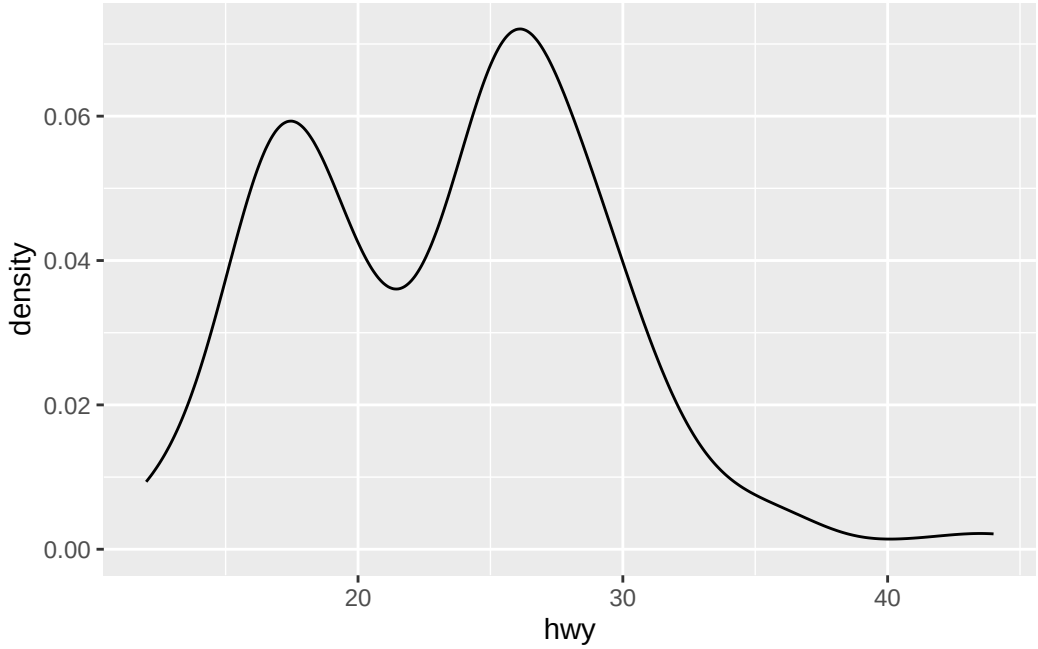


controlling look of plot by `geom` can reveal different features of data. - histogram/density reveals distribution of hwy mileage is bimodal and right skewed - boxplot reveals two potential outliers

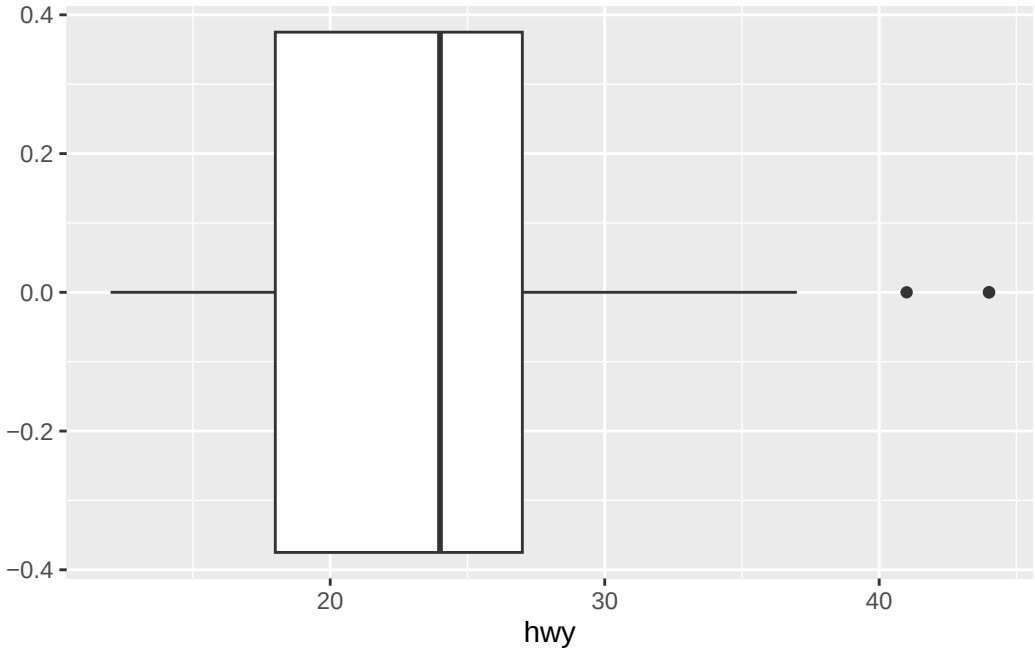
```
# Left
ggplot(mpg, aes(x = hwy)) +
  geom_histogram(binwidth = 2)
```




```
# Middle
ggplot(mpg, aes(x = hwy)) +
  geom_density()
```



```
# Right
ggplot(mpg, aes(x = hwy)) +
  geom_boxplot()
```

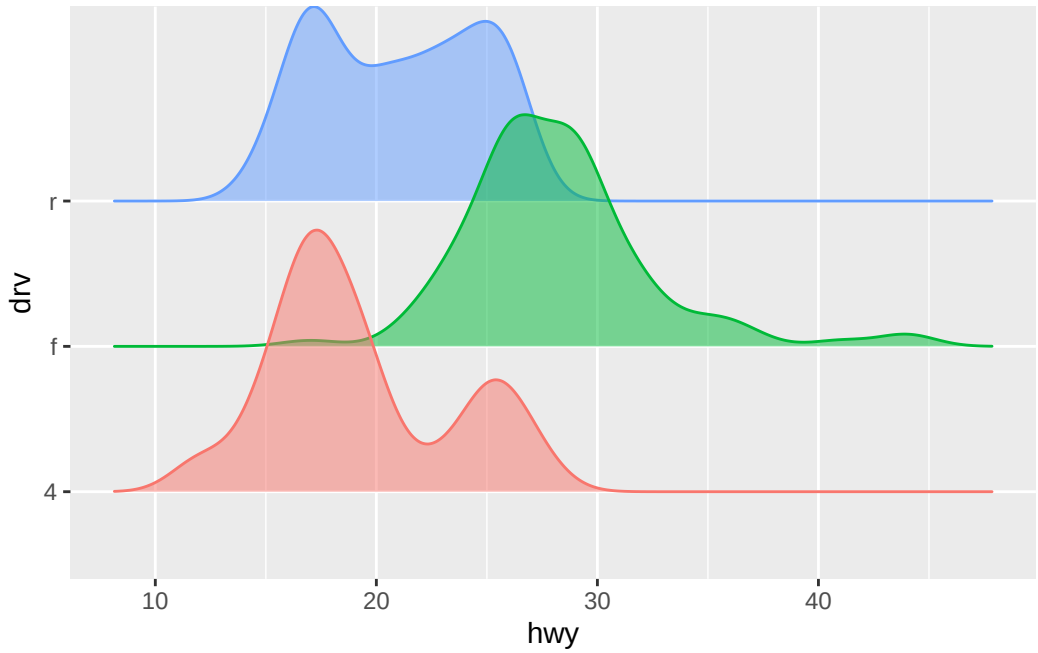


ggplot2 provides 40 geoms, but there are many many more plots one can make. for some examples, see the extension packages in https://exts.ggplot2.tidyverse.org/gallery/

for example, ridgeline plots for visualizing density of a numerical variable after grouping on a categorical var

```
ggplot(mpg, aes(x = hwy, y = drv, fill = drv, color = drv)) +
  ggridges::geom_density_ridges(alpha = 0.5, show.legend = FALSE)
```

```
Picking joint bandwidth of 1.28
```




```
#> Picking joint bandwidth of 1.28
```

read this to get a comprhensive overview of ggplot2: https://ggplot2.tidyverse.org/reference/

9.3.1 exercises

1. What geom would you use to draw a line chart? A boxplot? A histogram? An area chart?

- answer: see below

```
mpg # |> View()
```

```
# A tibble: 234 x 11
   manufacturer model      displ  year   cyl trans drv     cty   hwy fl    class
   <chr>        <chr>      <dbl> <int> <int> <chr> <chr> <int> <int> <chr> <chr>
 1 audi         a4           1.8  1999     4 auto~ f        18    29 p     comp~
 2 audi         a4           1.8  1999     4 manu~ f        21    29 p     comp~
 3 audi         a4           2    2008     4 manu~ f        20    31 p     comp~
 4 audi         a4           2    2008     4 auto~ f        21    30 p     comp~
 5 audi         a4           2.8  1999     6 auto~ f        16    26 p     comp~
```

```
 6 audi         a4           2.8  1999     6 manu~ f        18    26 p     comp~
 7 audi         a4           3.1  2008     6 auto~ f        18    27 p     comp~
 8 audi         a4 quattro   1.8  1999     4 manu~ 4        18    26 p     comp~
 9 audi         a4 quattro   1.8  1999     4 auto~ 4        16    25 p     comp~
10 audi         a4 quattro   2    2008     4 manu~ 4        20    28 p     comp~
# i 224 more rows
```

```
# line chart
ggplot(mpg, aes(x = year, y = hwy, color = manufacturer)) +
  # geom_line() # connects observations in the order which they appear on the x axis. NOT lin
  geom_smooth() # fits a line (with `color = manufacturer` specified, each unique value of ma
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: at 2008

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: radius 0.002025

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: all data on boundary of neighborhood. make span bigger

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 0.002025

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: zero-width neighborhood. make span bigger

Warning: Failed to fit group 7.
Caused by error in `predLoess()`:
! NA/NaN/Inf in foreign function call (arg 5)

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: span too small.  fewer data values than degrees of freedom.

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : span too small.  fewer
data values than degrees of freedom.

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: span too small.  fewer data values than degrees of freedom.

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: radius 0.002025

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: all data on boundary of neighborhood. make span bigger
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 0.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 1

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: zero-width neighborhood. make span bigger

Warning: Failed to fit group 9.
Caused by error in `predLoess()`:
! NA/NaN/Inf in foreign function call (arg 5)

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: span too small.  fewer data values than degrees of freedom.

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : span too small.  fewer
data values than degrees of freedom.
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: span too small.  fewer data values than degrees of freedom.

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: radius 0.002025

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: all data on boundary of neighborhood. make span bigger

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 0.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 1

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: zero-width neighborhood. make span bigger

Warning: Failed to fit group 12.
Caused by error in `predLoess()`:
! NA/NaN/Inf in foreign function call (arg 5)
```

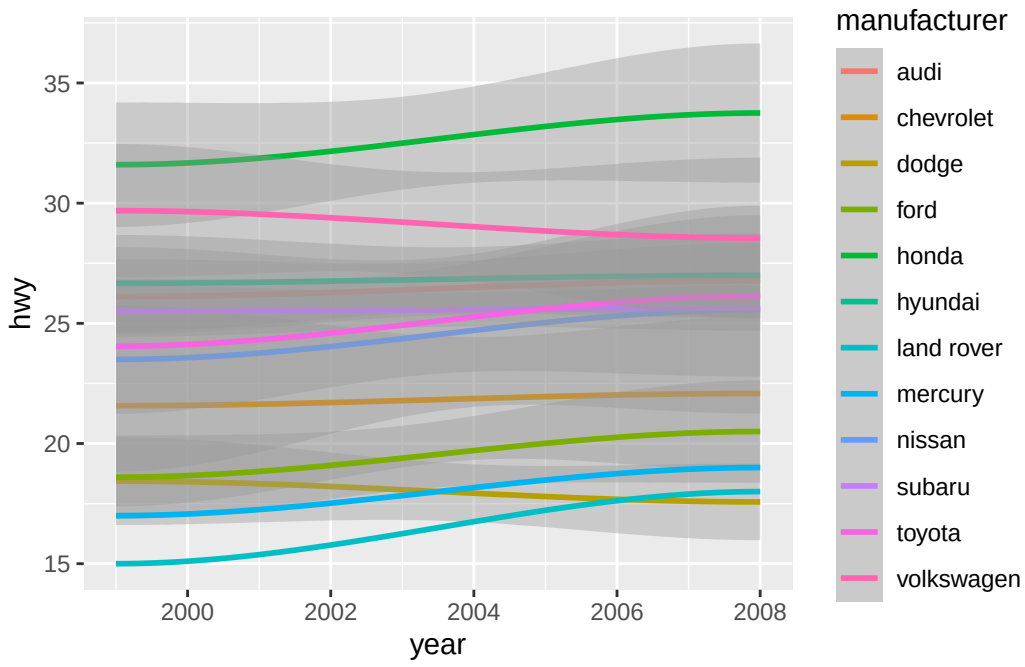
```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0
```

```
Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: pseudoinverse used at 1999

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: neighborhood radius 9.045

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: reciprocal condition number 0

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: There are other near singularities as well. 81.812

Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : pseudoinverse used at
1999
```

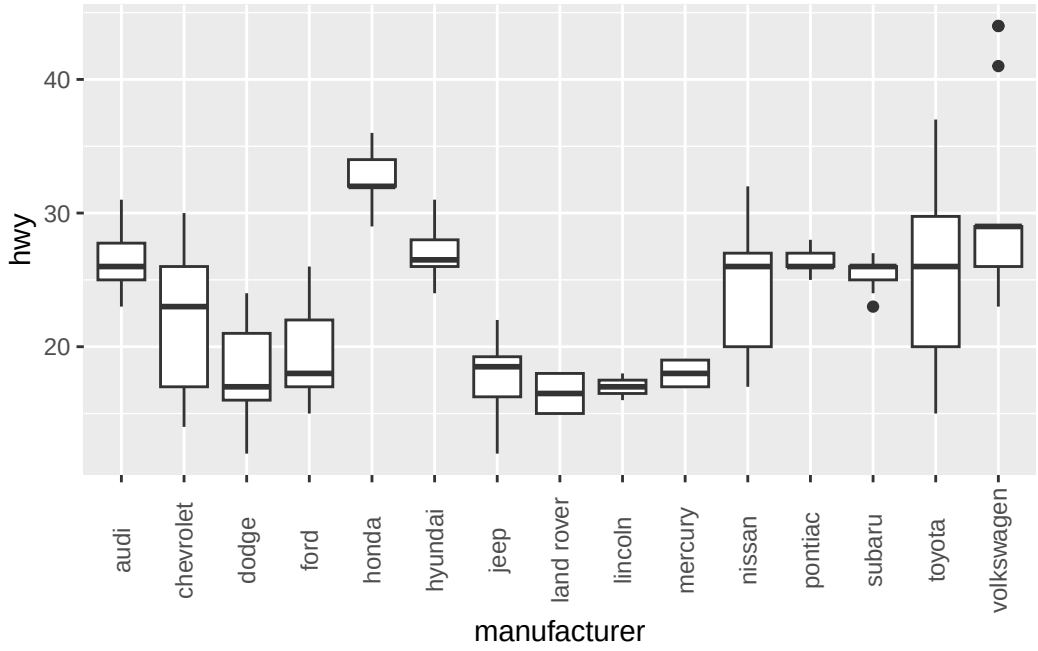
```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : neighborhood radius
9.045
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : reciprocal condition
number 0
```

```
Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x
else if (is.data.frame(newdata))
as.matrix(model.frame(delete.response(terms(object)), : There are other near
singularities as well. 81.812
```




```
# boxplot
ggplot(mpg, aes(x = manufacturer, y = hwy)) +
  geom_boxplot() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, )) # hjust=1
```

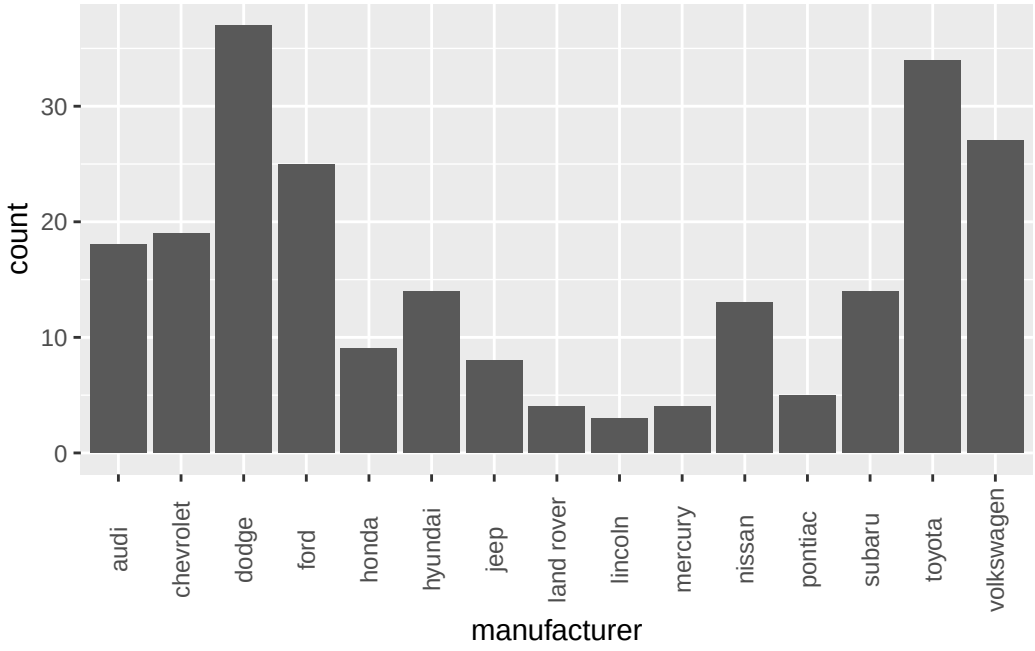



```
# histogram
# just to check the histogram
mpg |> summarize(n_manufac = n(), .by = manufacturer) # |> View()
```

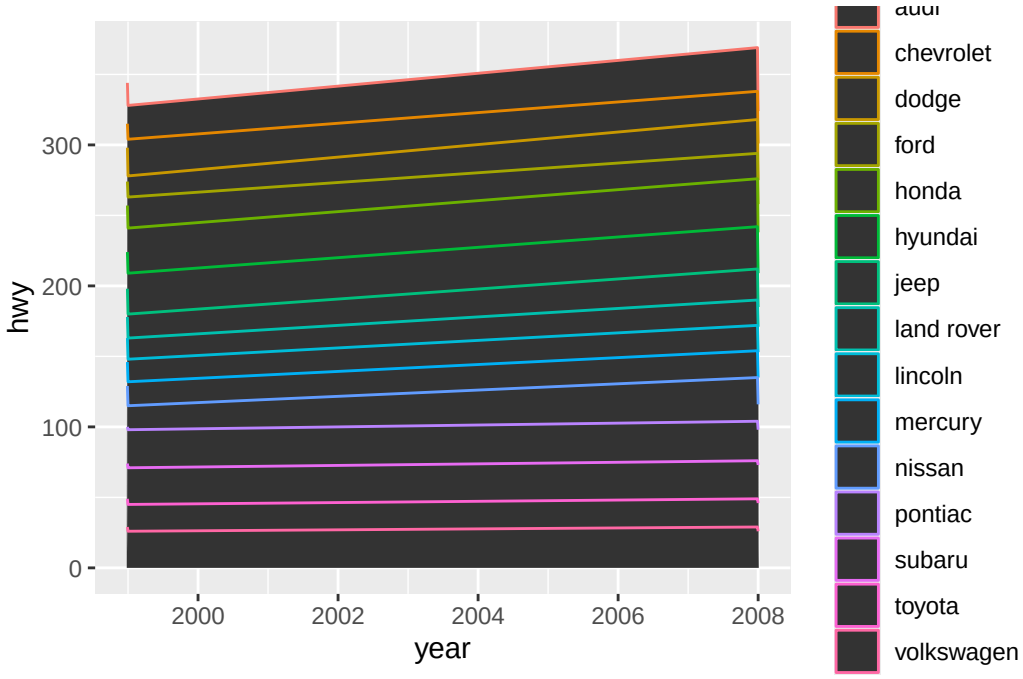
```
# A tibble: 15 x 2
   manufacturer n_manufac
   <chr>            <int>
 1 audi                18
 2 chevrolet           19
 3 dodge               37
 4 ford                25
 5 honda                9
 6 hyundai             14
 7 jeep                 8
 8 land rover           4
 9 lincoln              3
10 mercury              4
11 nissan              13
12 pontiac              5
13 subaru              14
14 toyota              34
15 volkswagen          27
```

```r
ggplot(mpg, aes(x = manufacturer)) +
  geom_histogram(stat = "count") + # fails without stat = "count" for categorical variables
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, )) # hjust=1
```

```
Warning in geom_histogram(stat = "count"): Ignoring unknown parameters:
`binwidth` and `bins`
```




```r
# area chart
ggplot(mpg, aes(x = year, y = hwy, color = manufacturer)) +
  geom_area()
```

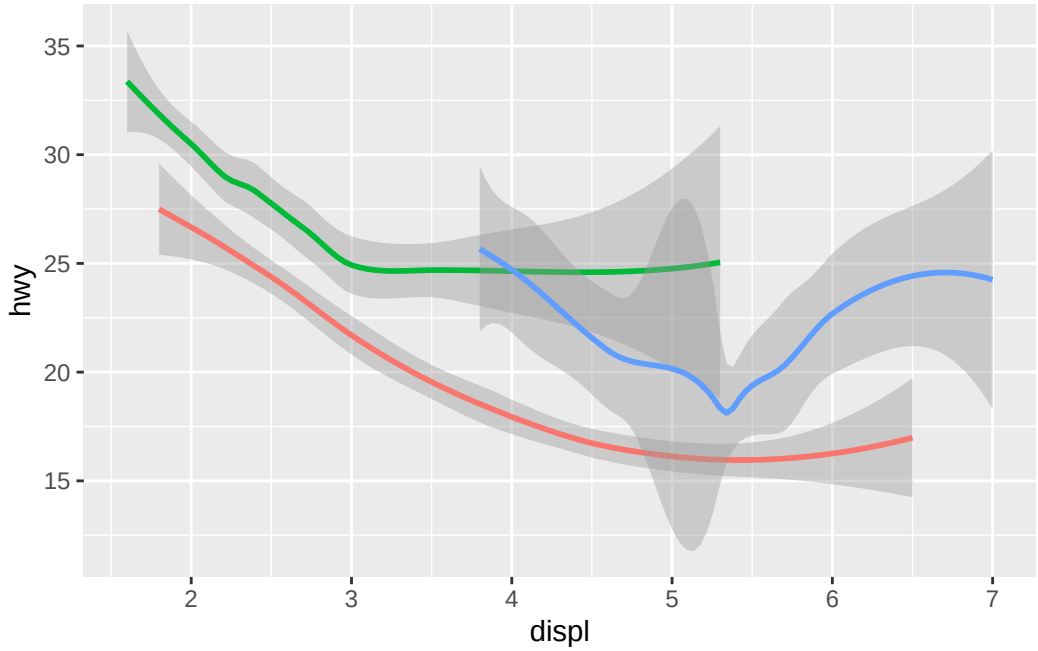


2. Earlier in this chapter we used `show.legend` without explaining it. What does `show.legend = FALSE` do here? What happens if you remove it? Why do you think we used it earlier?

Answer: `show.legend` removes the legend from the plot (see example below). they likely removed it earlier because they wanted it to be consistent with the middle and left plots (see the code copied below). other situations where suppressing legend is useful: too many things in the legend, wrapping/patchwork multiple plots together and only need one...

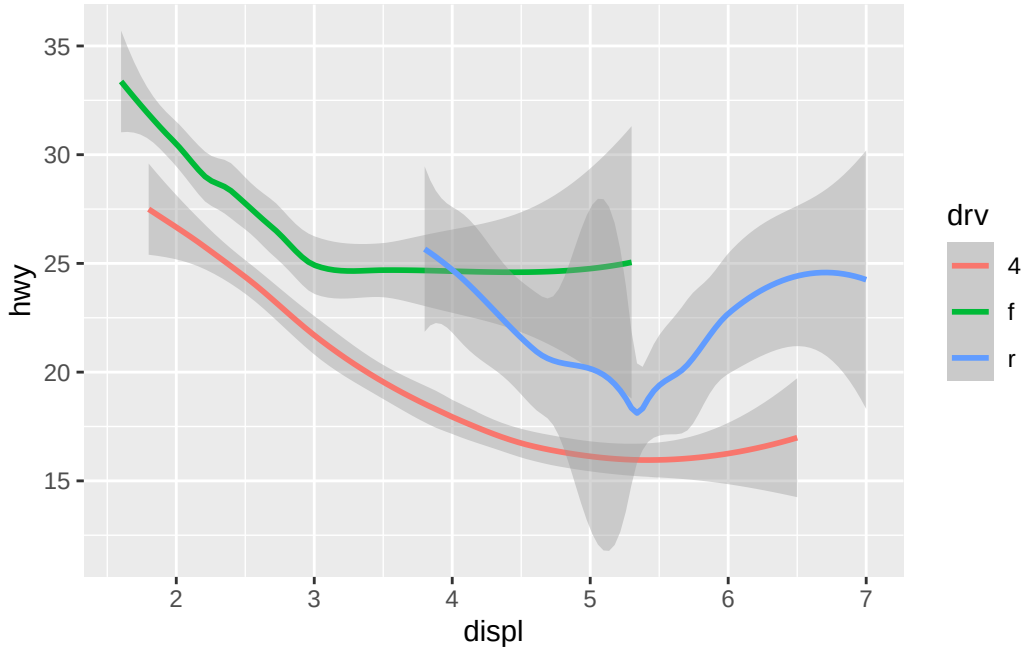
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(color = drv), show.legend = FALSE)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(color = drv))
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
# copied code exemplifying show.legend
# # Right - generates 3 geom_smooth lines and legend (which can be suppressed), and colors th
# ggplot(mpg, aes(x = displ, y = hwy)) +
#   geom_smooth(aes(color = drv), show.legend = FALSE)
```

3. What does the `se` argument to `geom_smooth()` do?

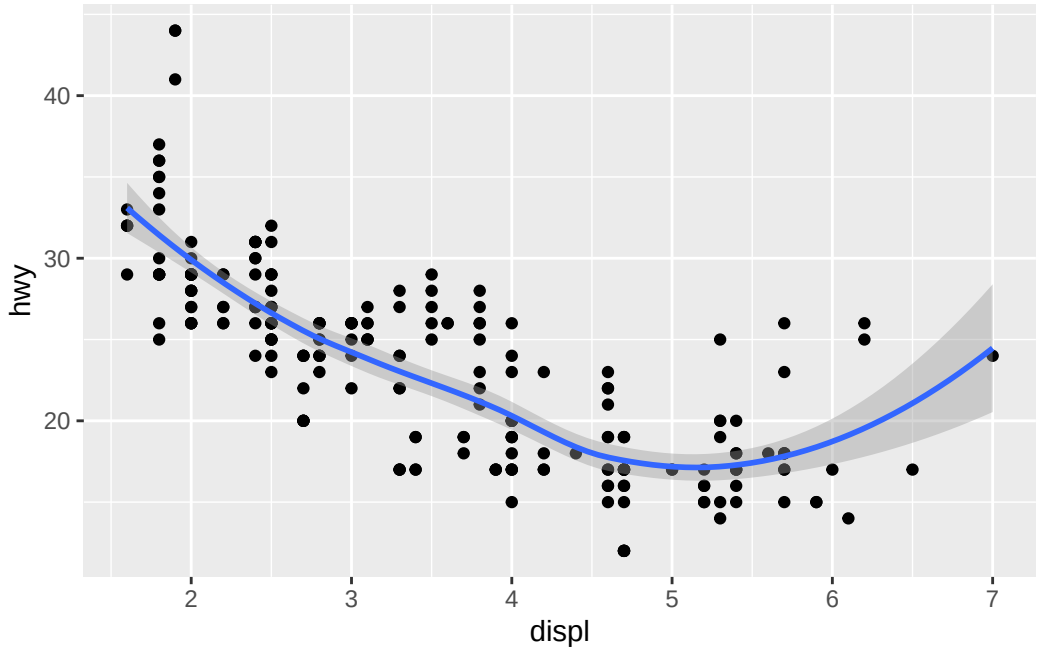
Answer: quoting from `?geom_smooth`: `se Display confidence interval around smooth? (TRUE by default, see level to control.)` So `se` calculates some type of confidence interval that the learned statistical model from `geom_smooth`, represented by the curve, has in its predictions?

```
?geom_smooth
```

4. Recreate the R code necessary to generate the following graphs. Note that wherever a categorical variable is used in the plot, it's `drv`.

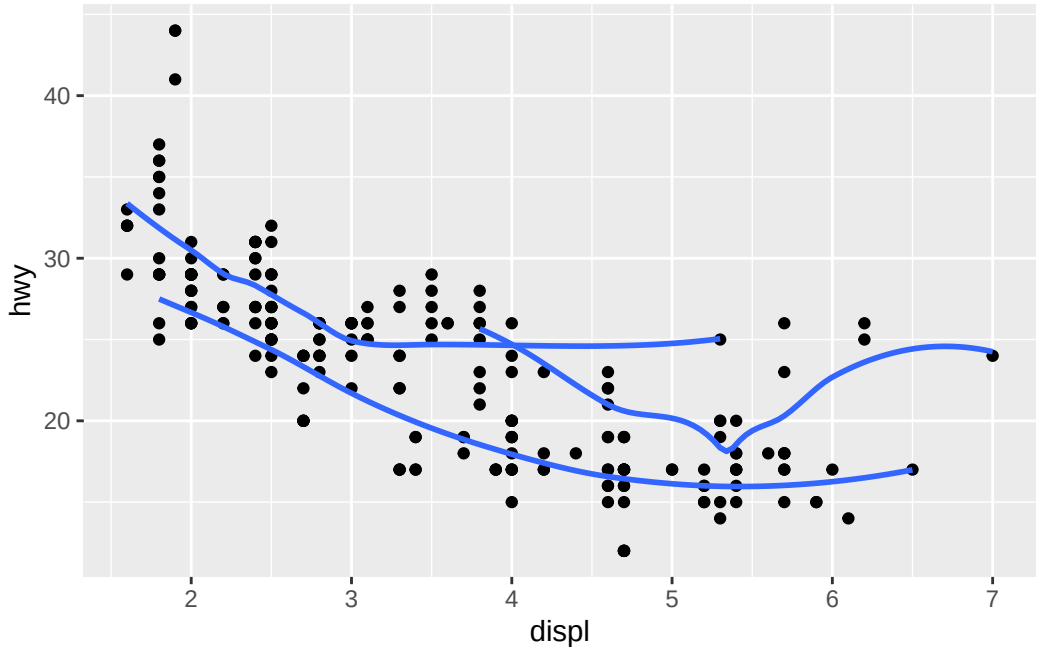
```
# plot 1: (1,1)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth()
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



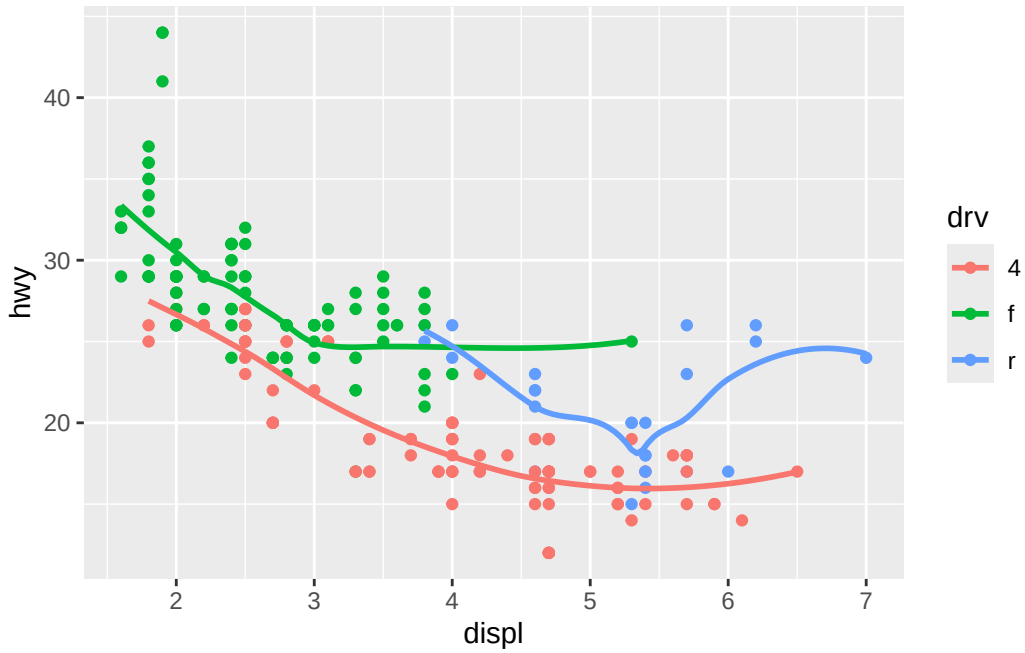

```
# plot 2: (1,2)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth(mapping = aes(group = drv), se = FALSE)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



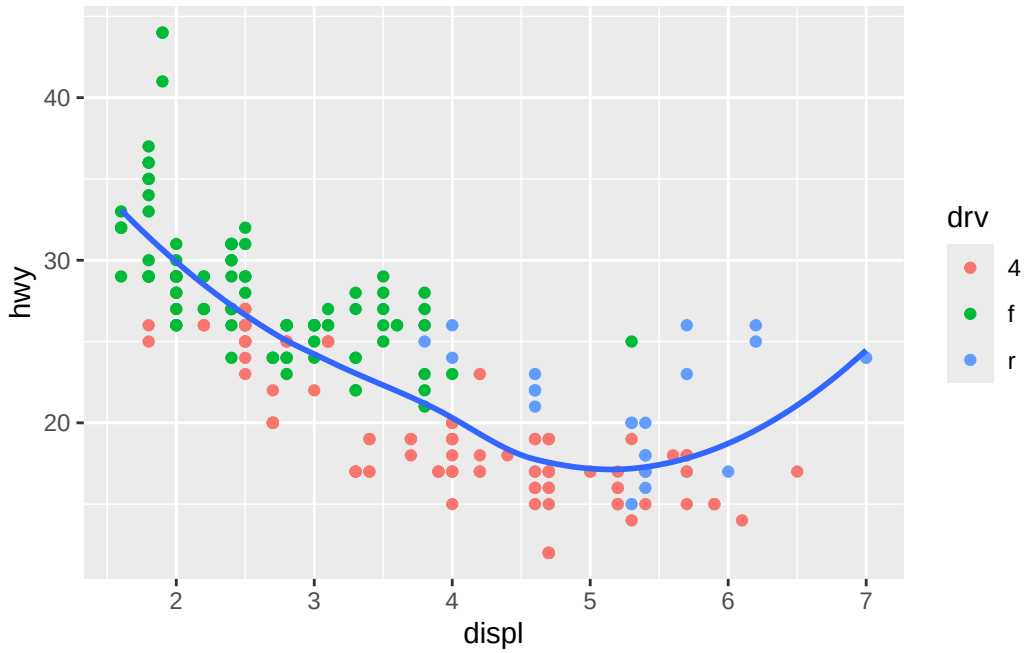

```
# plot 3: (2,1)
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  geom_smooth(se = FALSE)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



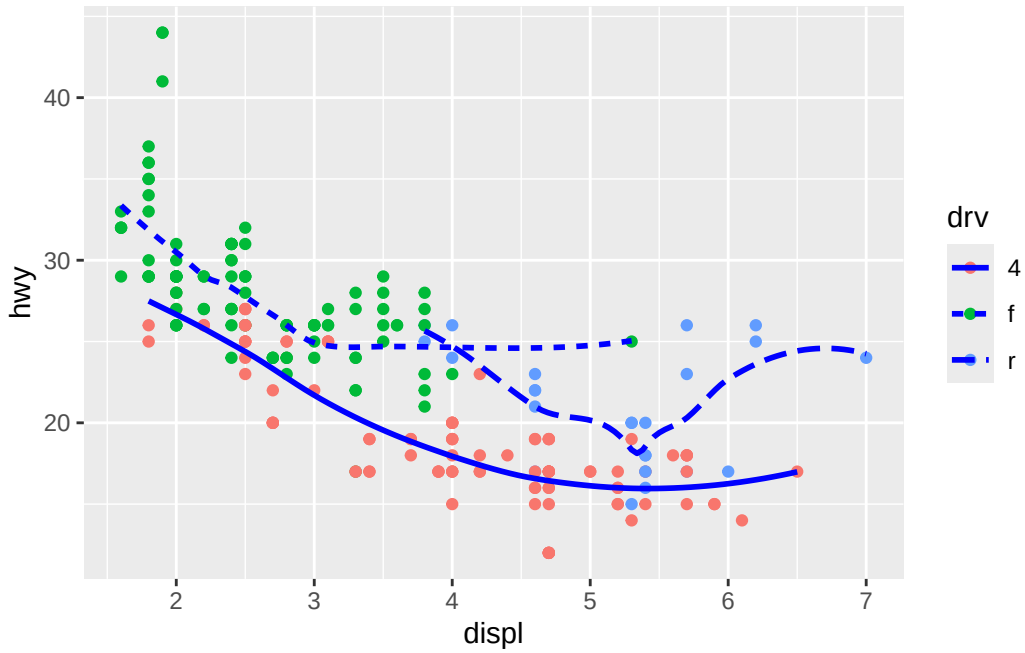

```
# plot 4: (2,2)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth(se = FALSE)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

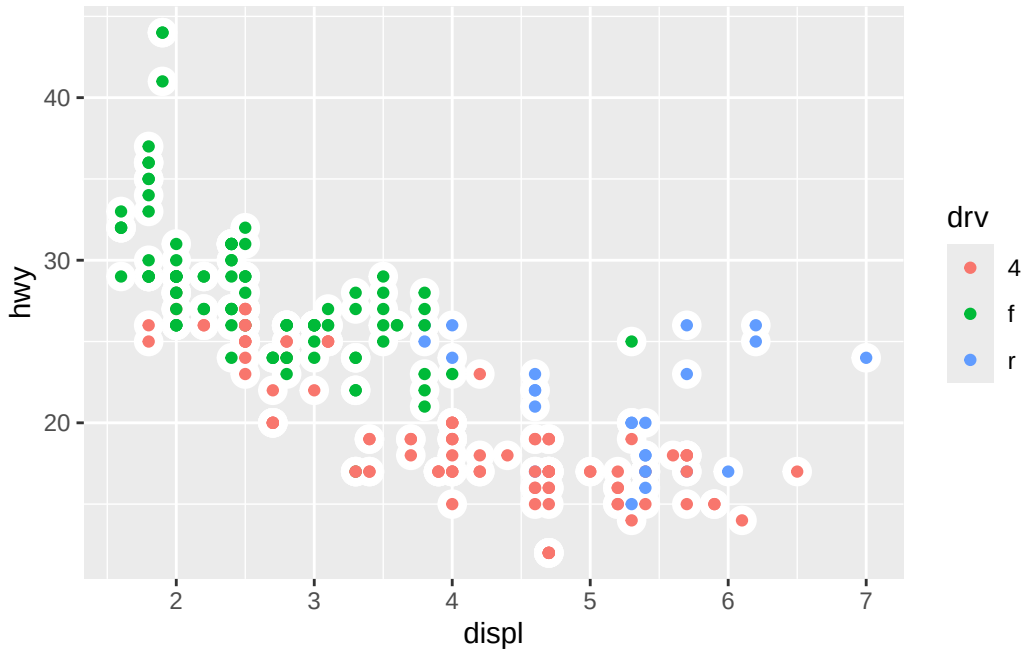



```
# plot 5: (3,1)
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  geom_smooth(aes(linetype = drv), se = FALSE, color = "blue")
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



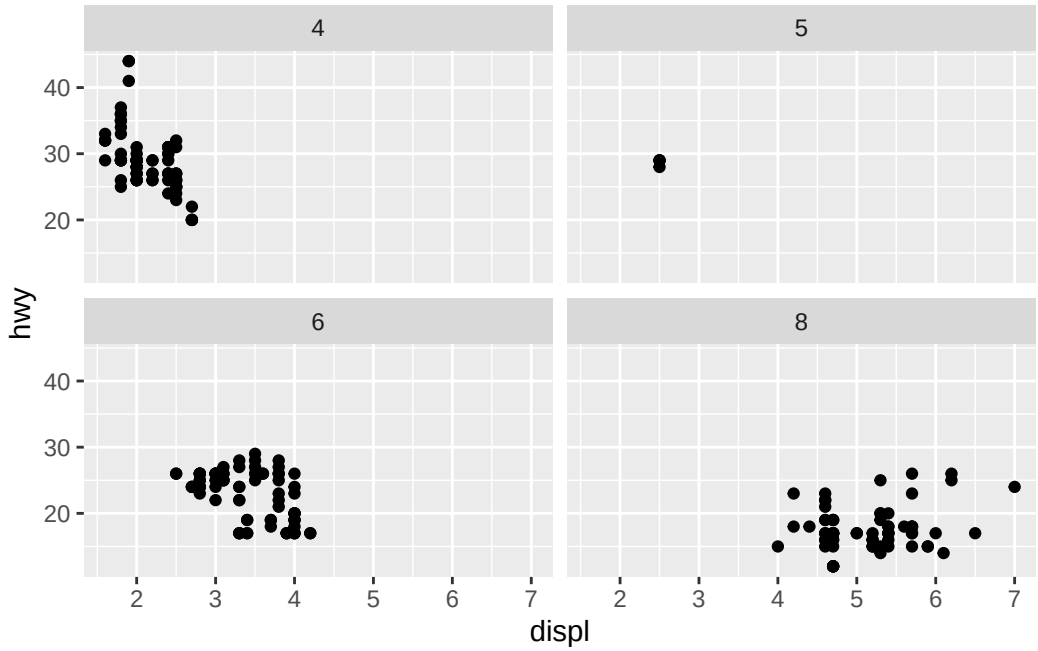

```
# plot 6: (3,2)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(size = 2), color = "white", show.legend = FALSE) +
  geom_point(aes(color = drv))
```



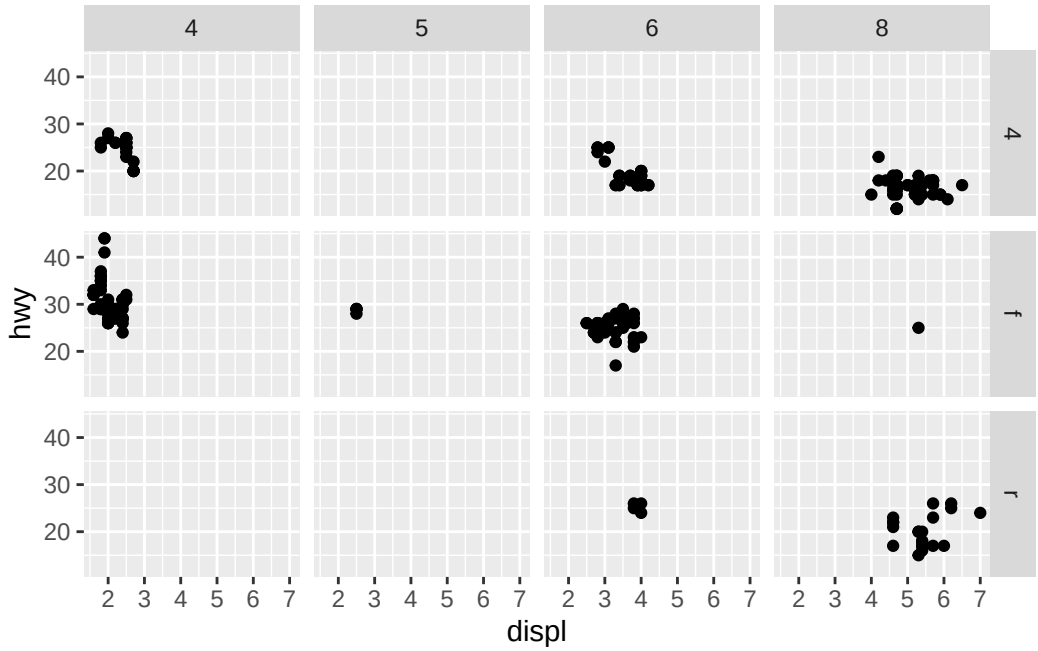
9.4 Facets

- `facet_wrap(~x)` for subsetting on a single variable (one sided formula)
- `facet_grid(~x)` for combination of two variables (double sided formula with format `rows ~ cols`)
  - default, each facet shares same scale range for x axes (and y axes)
    - `"free_x"` allows different x axis scales across cols,
    - `"free_y"` allows different scales across rows,
    - `"free"` allows both

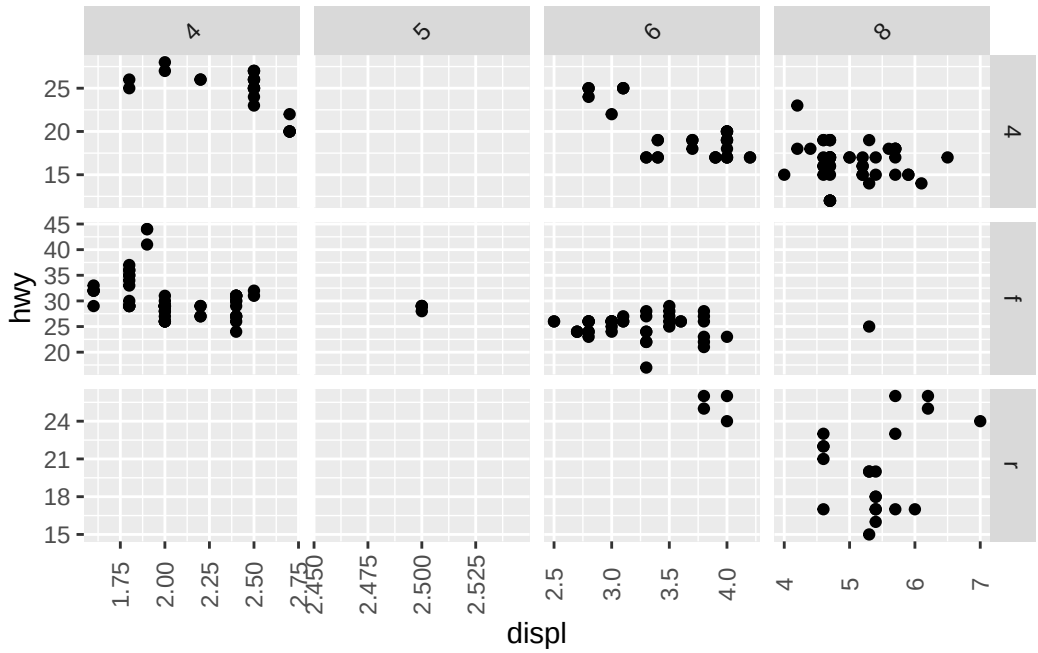
```
# facet on cylinder
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_wrap(~cyl)
```




```
# facet on drv (row) and cyl (col)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_grid(drv ~ cyl)
```




```
# facet on drv (row) and cyl (col), with free x and y axes
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_grid(drv ~ cyl, scales = "free") +
  theme(
    strip.text.x = element_text(angle = 45), # labels in the strip facetting variable
    axis.text.x = element_text(angle = 90, vjust = 0.5) # x axis tick
  )
```

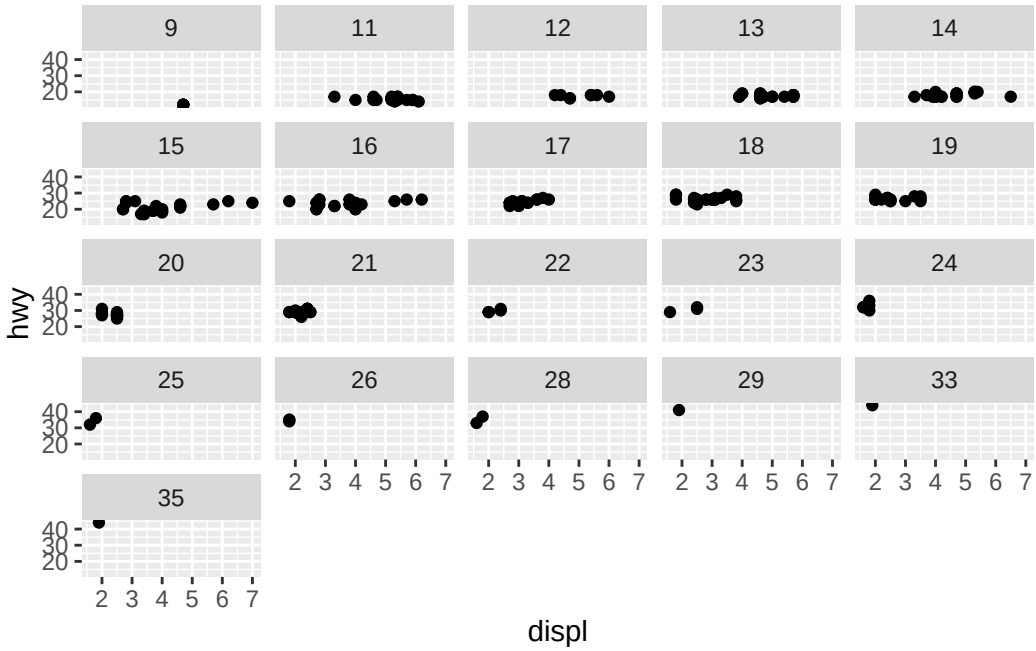


9.4.1 Exercises

1. What happens if you facet on a continuous variable?

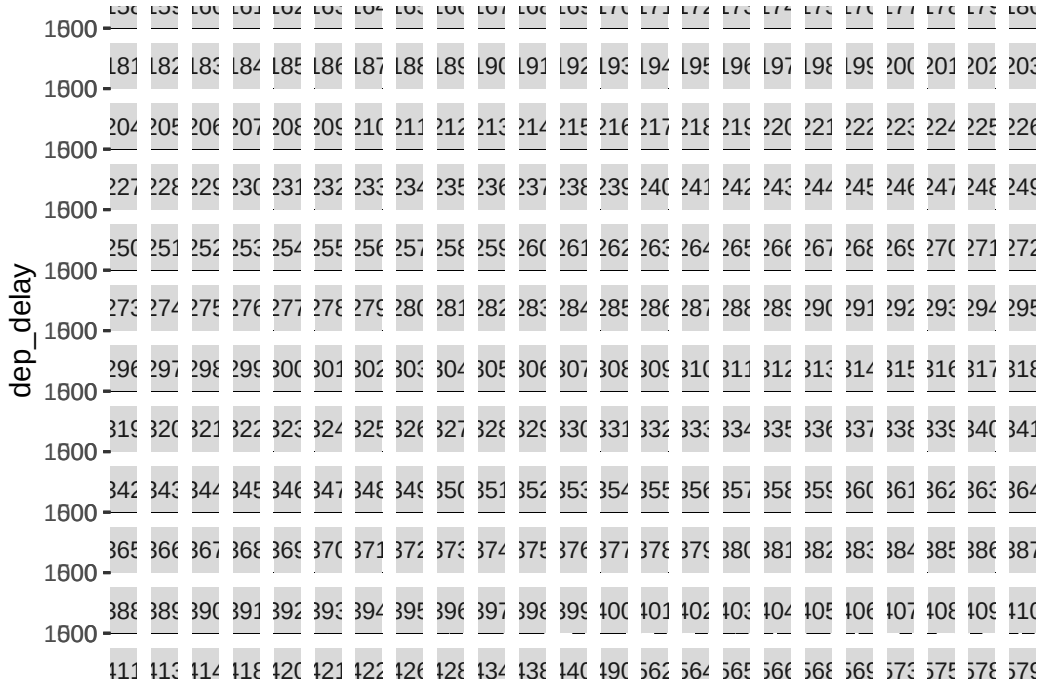
- answer: when you facet on a continuous variable, ggplot just takes every instance of that exact continuous value and gives it its own facet. the `rnorm` plot below demonstrates this best

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_wrap(~cty) # maybe too easy
```




```
ggplot(nycflights13::flights, aes(x = carrier, y = dep_delay)) +
  geom_point() +
  facet_wrap(~air_time) # still too easy since integer. let's get a float value
```

```
Warning: Removed 8255 rows containing missing values or values outside the scale range
(`geom_point()`).
```

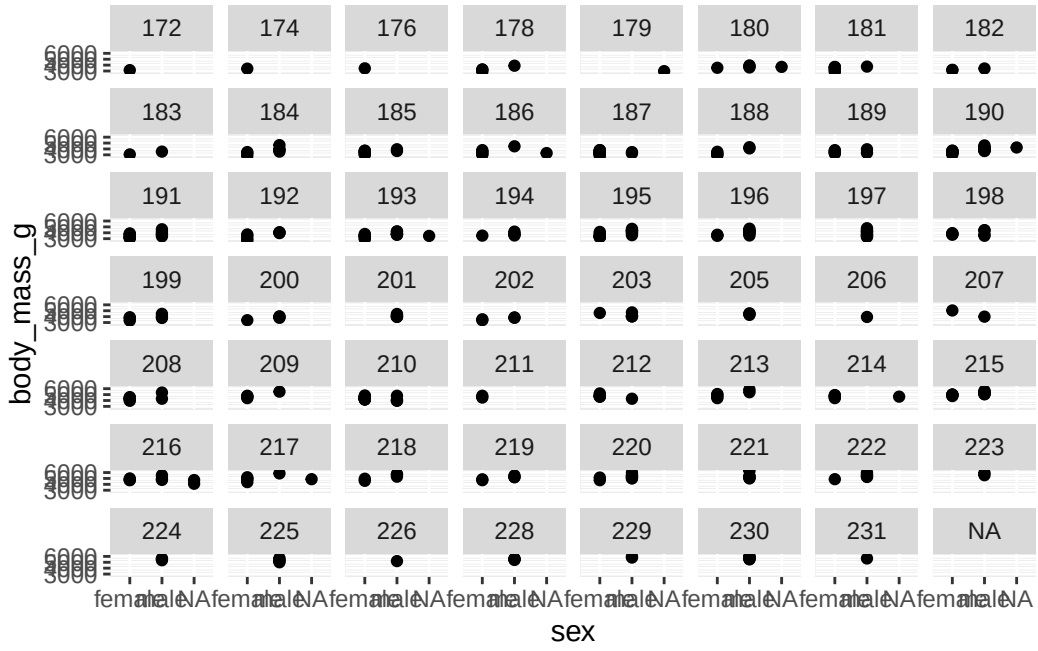



```
palmerpenguins::penguins |> head()
```

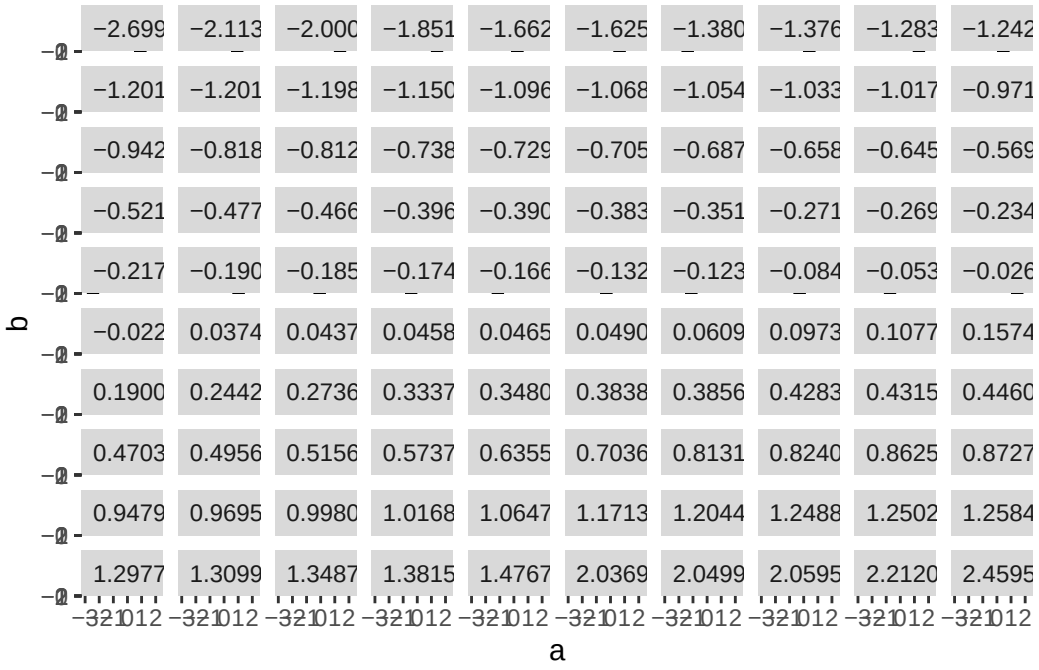
```
# A tibble: 6 x 8
  species island    bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
  <fct>   <fct>              <dbl>         <dbl>             <int>       <int>
1 Adelie  Torgersen           39.1          18.7               181        3750
2 Adelie  Torgersen           39.5          17.4               186        3800
3 Adelie  Torgersen           40.3          18                 195        3250
4 Adelie  Torgersen           NA            NA                  NA          NA
5 Adelie  Torgersen           36.7          19.3               193        3450
6 Adelie  Torgersen           39.3          20.6               190        3650
# i 2 more variables: sex <fct>, year <int>
```

```
ggplot(palmerpenguins::penguins, aes(x = sex, y = body_mass_g)) +
  geom_point() +
  facet_wrap(~flipper_length_mm) # still too easy, not enough values?
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



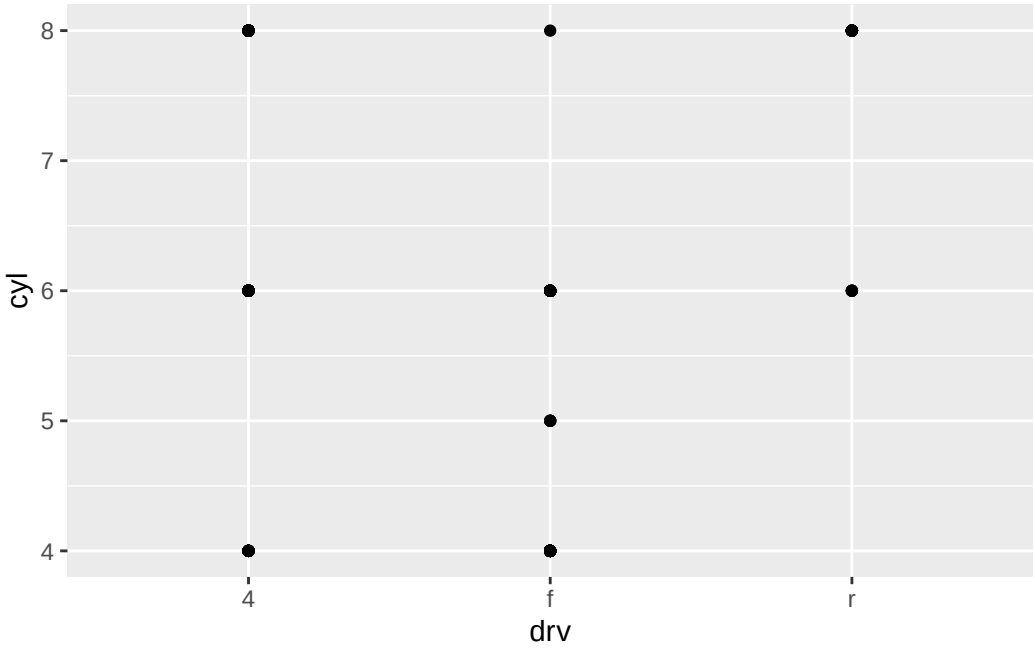

```r
# let's sample from the normal distribution 30000 times, creating 3 variables of 10000 observ
nrows <- 100
ncols <- 3
n <- nrows * ncols
set.seed(42)
rnorm_sample <- matrix(
  data = rnorm(n = n, mean = 0, sd = 1),
  nrow = nrows,
  ncol = ncols,
  dimnames = list(1:nrows, letters[1:ncols])
)
# rnorm_sample |> View()
cur_plot <- ggplot(rnorm_sample, aes(x = a, y = b)) +
  geom_point() +
  facet_wrap(~c) +
  # strip.text, adjusts the facet wrap strip titles
  # element_text, is ggplot function for adjusting attributes of text in ggplot (note, it doe
  # hjust to change horizontal position, hjust = 0 makes left aligned
  theme(strip.text = element_text(hjust = 0))

cur_plot
```



2. What do the empty cells in the plot above with `facet_grid(drv ~ cyl)` mean? Run the following code. How do they relate to the resulting plot?

- answer: the empty cells in `facet_grid(drv ~ cyl)` correspond to combinations of `drv` and `cyl` that do not exist, specifically `(drv,cyl)` values `(4,5)`, `("r", 4)`, `("r", 5)`. the plot in the code below have points at combinations of `drv` and `cyl` that do exist

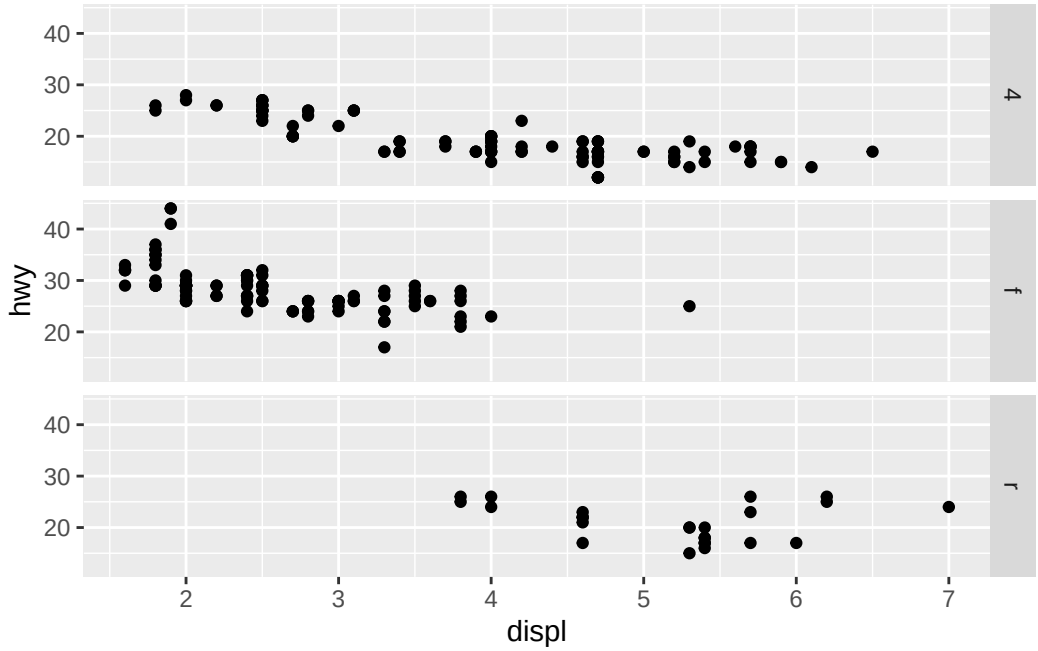
```
ggplot(mpg) +
  geom_point(aes(x = drv, y = cyl))
```



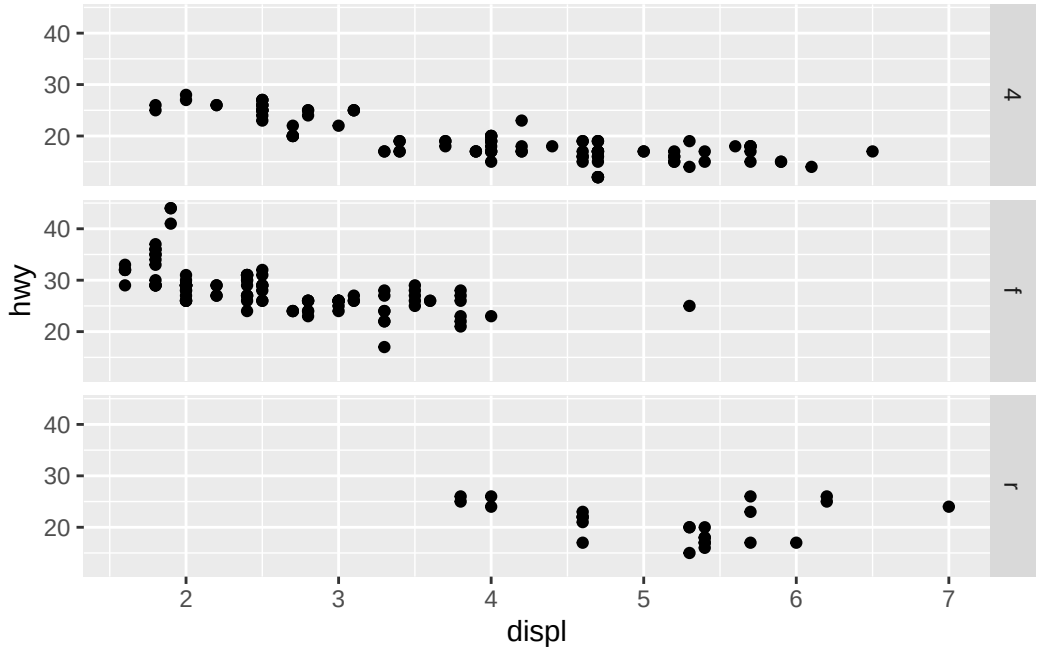
3. What plots does the following code make? What does `.` do?

- answer: the `.` means "do not place a variable here". The result is `facet_grid` can be made to look like `facet_wrap` with a few extra args to `facet_wrap`

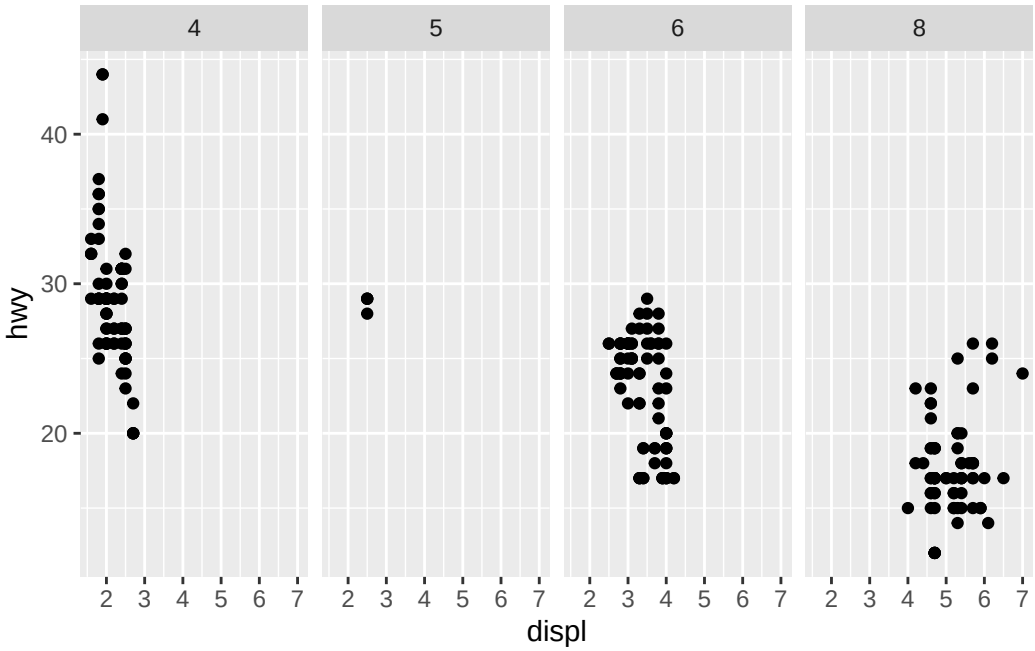
```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_grid(drv ~ .)
```




```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_wrap(~drv, strip.position = "right", ncol = 1)
```



```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_grid(. ~ cyl)
```

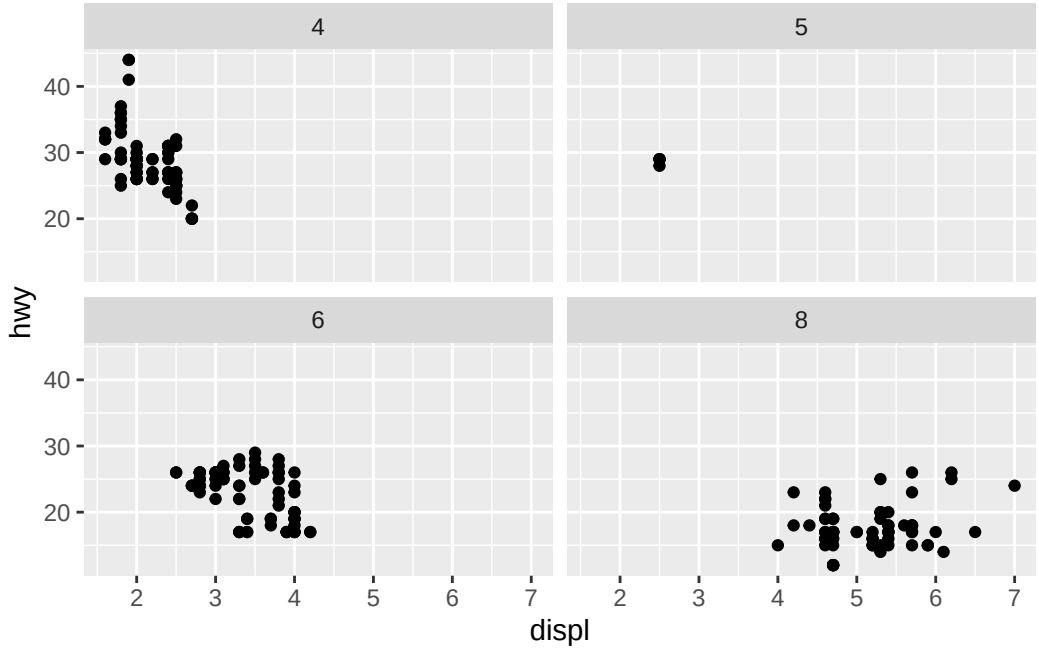


4. Take the first faceted plot in this section (see below). What are the advantages to using faceting instead of the color aesthetic? What are the disadvantages? How might the balance change if you had a larger dataset?

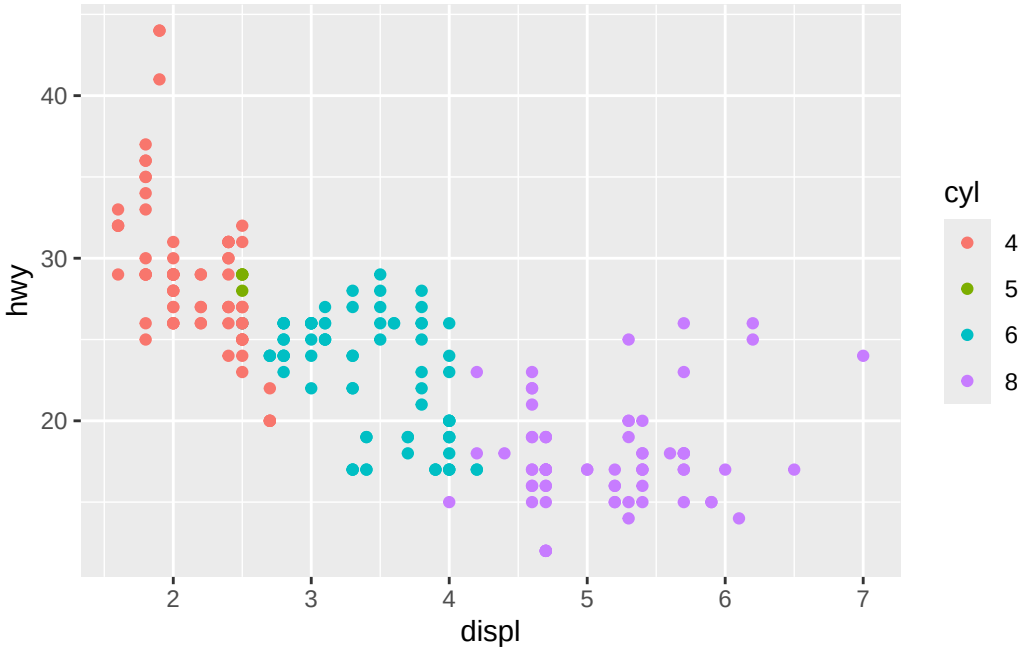
- answer: faceting advantages over color include de-cluttering plots, allows focus on specific distributions when using a faceted variable (and relationships between the specific distributions if there are any, for example we can see that as cylinders increase, `hwy` mpg decreases but `displ` increases). disadvantages compared to coloring includes: faceting may be difficult for variables with many categories - the number of facets may be large and hard to plot all separately (for example 9.4.1.1 facet of `rnorm_sample`); whereas, diverse colors can all fit on to one plot (which may be cluttered and the number of colors will be prohibitive without a color scale package). The balance may change for larger datasets in that faceting a dataset with many observations on a variable with small to medium number of categories may help us see specific distributions more clearly than colors since lots of observations can overlap each other and make the plot cluttered.

```
# first faceted plot in this section
ggplot(mpg) +
```

```
  geom_point(aes(x = displ, y = hwy)) +
  facet_wrap(~cyl, nrow = 2)
```



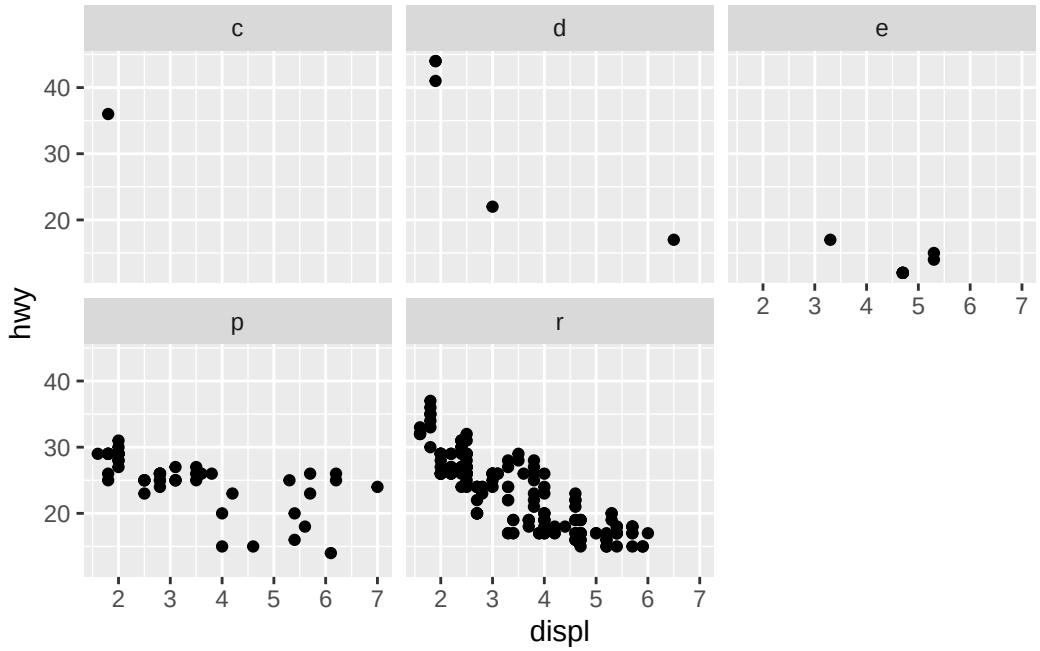

```
# color
ggplot(mpg |> mutate(cyl = factor(cyl, levels = sort(unique(cyl))))) +
  geom_point(aes(x = displ, y = hwy, colour = cyl))
```



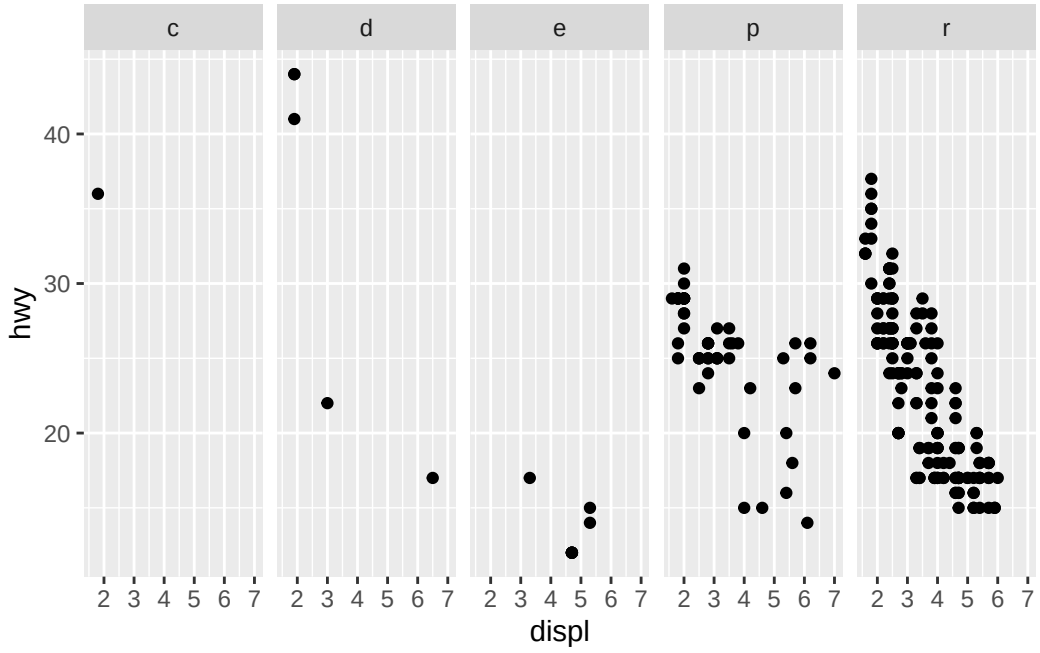
5. Read `?facet_wrap`. What does `nrow` do? What does `ncol` do? What other options control the layout of the individual panels? Why doesn't `facet_grid()` have `nrow` and `ncol` arguments?

- answer: `nrow` controls number of rows when faceting on a variable. `ncol` does the same for number of columns. the two can be combined, and `facet_wrap()` will fill as many plots as it can in `nrow * ncol` (makes most sense if `nrow * ncol` equals number of categories of the faceting variable). `facet_grid()` doesn't have these since the two variables being used determine the number of rows and cols (the LHS of formula number of rows, RHS of formula is number of columns)

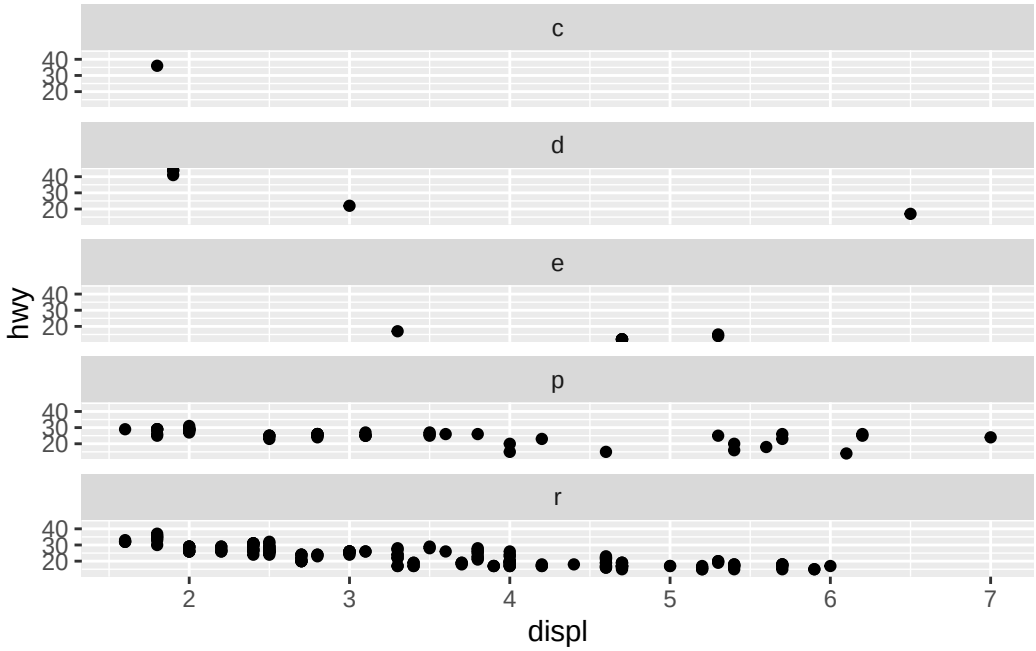
```
ggplot(mpg, aes(displ, hwy)) +
  geom_point() +
  facet_wrap(~fl)
```



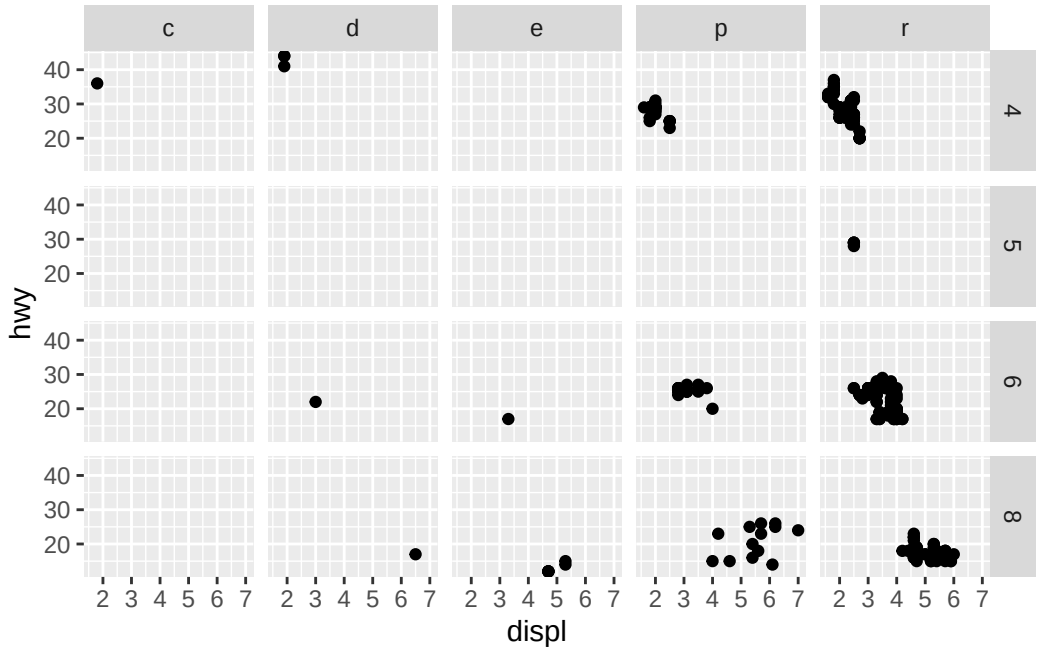

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point() +
  facet_wrap(~fl, nrow = 1, ncol = 100)
```



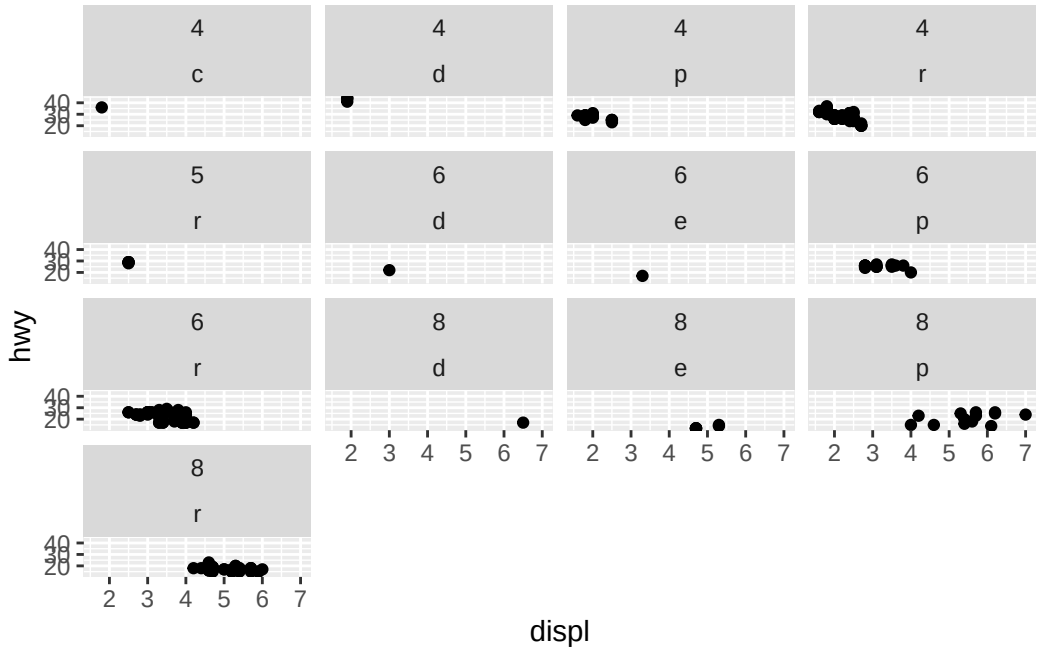
```
ggplot(mpg, aes(displ, hwy)) +
  geom_point() +
  facet_wrap(~fl, nrow = 100, ncol = 1)
```



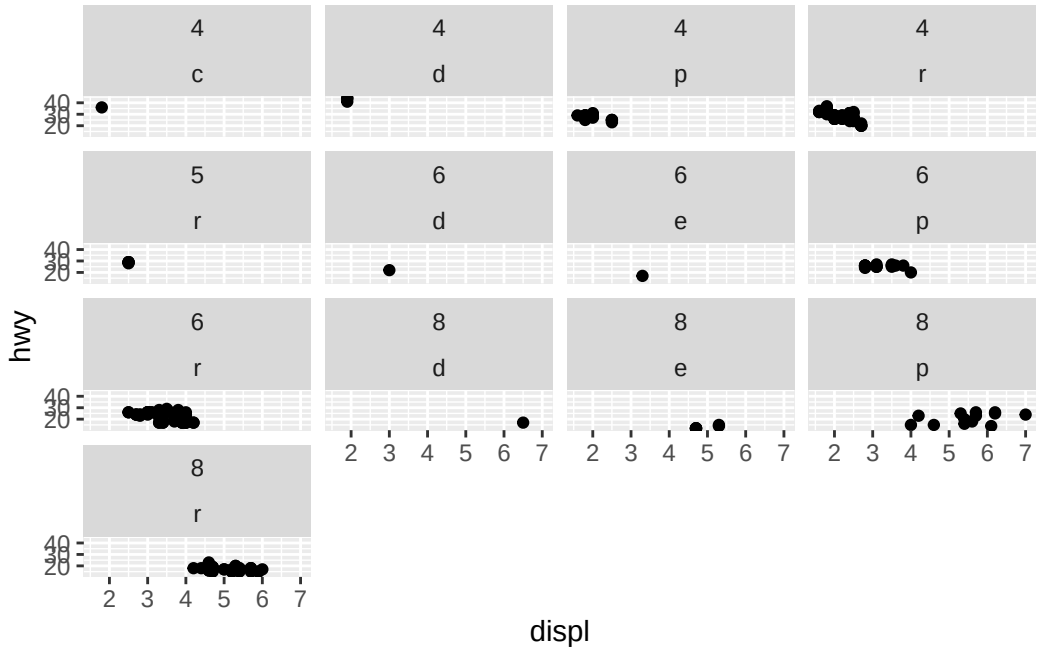

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point() +
  facet_grid(cyl ~ fl)
```




```
# ACCIDENTAL DISCOVERY: if you pass a two sided formula to facet wrap it combines the two va
ggplot(mpg, aes(displ, hwy)) +
  geom_point() +
  facet_wrap(cyl ~ fl)
```




```
# same as above
ggplot(mpg, aes(displ, hwy)) +
  geom_point() +
  facet_wrap(~ cyl + fl)
```



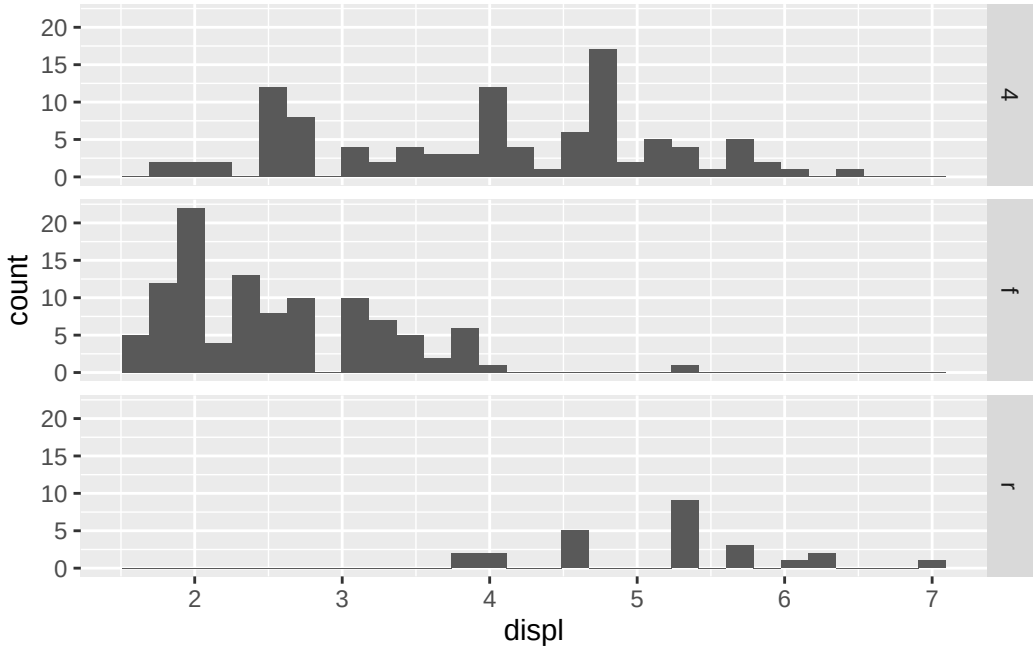

```
# these standard R formulas dont work
# ggplot(mpg, aes(displ, hwy)) +
#   geom_point() +
#   facet_wrap(~ fl - cyl)
#
# ggplot(mpg, aes(displ, hwy)) +
#   geom_point() +
#   facet_wrap(~ (fl | cyl))
```

6. Which of the following plots makes it easier to compare engine size (`displ`) across cars with different drive trains? What does this say about when to place a faceting variable across rows or columns?

- answer: For comparing `displ` across cars with different `drv` trains, `facet_grid(drv ~ .)` row wise comparion is easier than `facet_grid(. ~ drv)` col wise comparison. The shared x axis allows us to say things like "when displ = 4, drv = 4 has much higher number of cars compared to f or r wheel drv." Or broader "when displ <= 4, there are a lot more cars with drv = 4 or drv = f compared to drv = r."

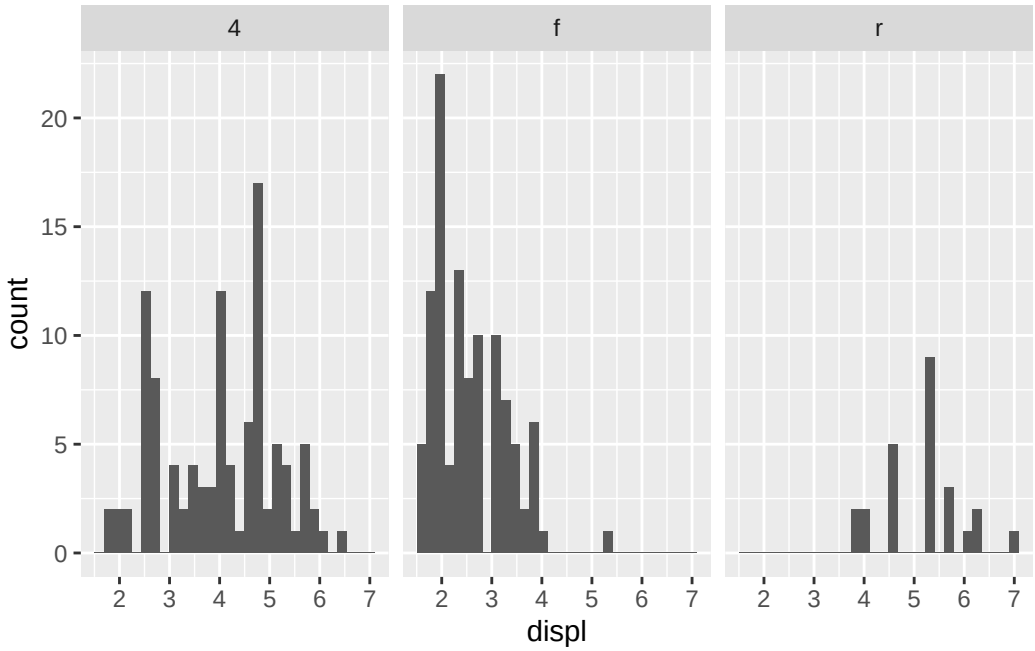
```
ggplot(mpg, aes(x = displ)) +
  geom_histogram() +
  facet_grid(drv ~ .)
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```




```
ggplot(mpg, aes(x = displ)) +
  geom_histogram() +
  facet_grid(. ~ drv)
```

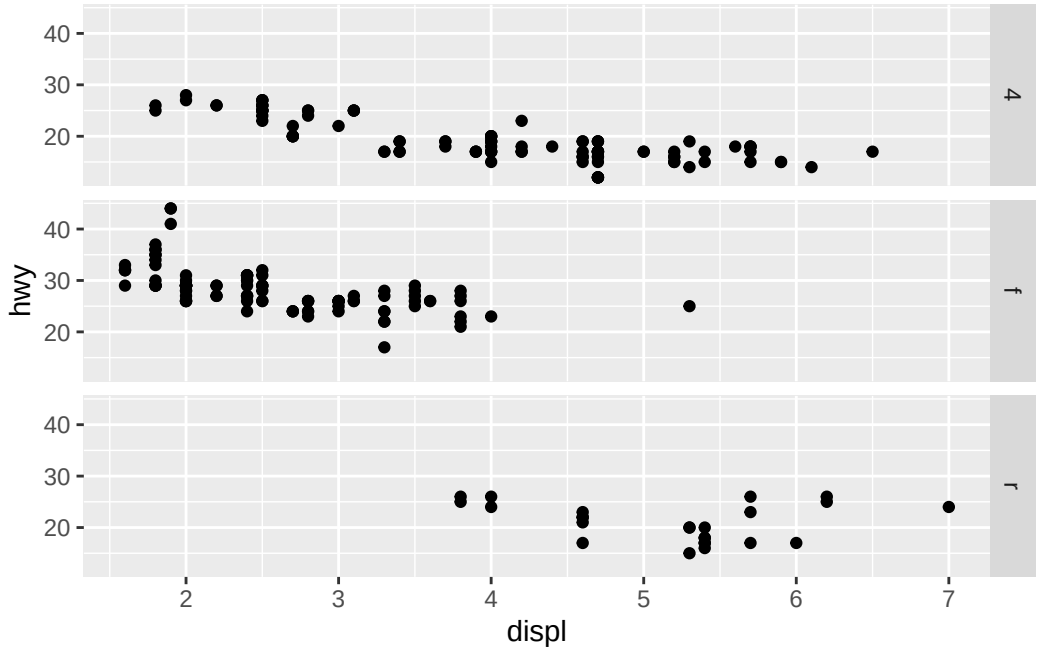
```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



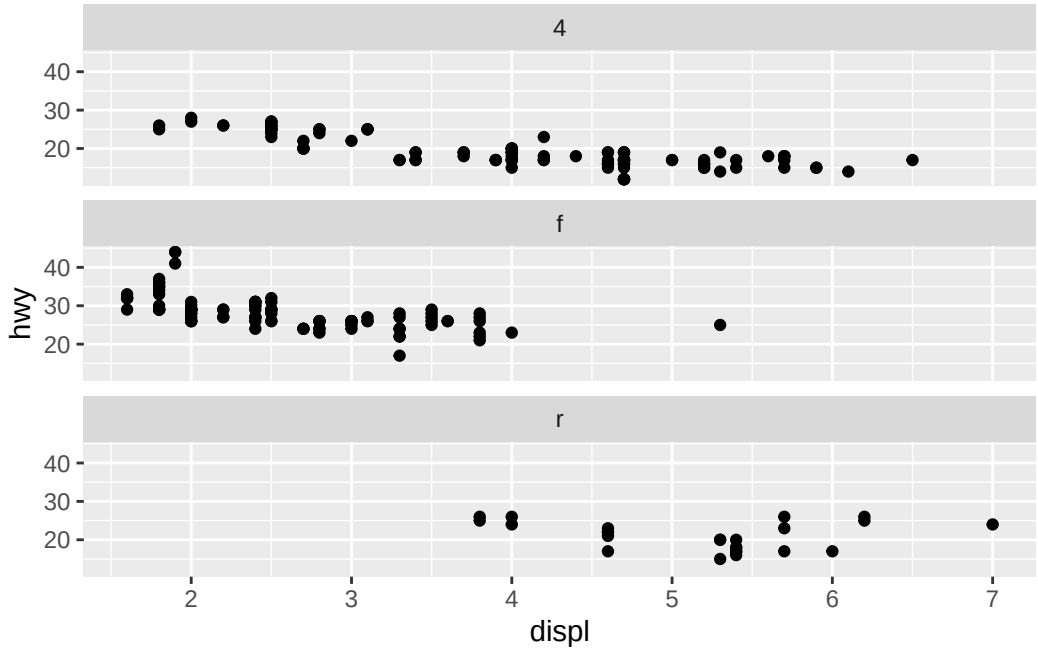
7. Recreate the following plot using `facet_wrap()` instead of `facet_grid()`. How do the positions of the facet labels change?

- answer: the positions of the facet labels with `facet_wrap()` default to being on top. but, this can be changed with this `strip.position` arg, see below, such that `facet_wrap()` and `facet_grid()` give the same output for a 1 sided formula

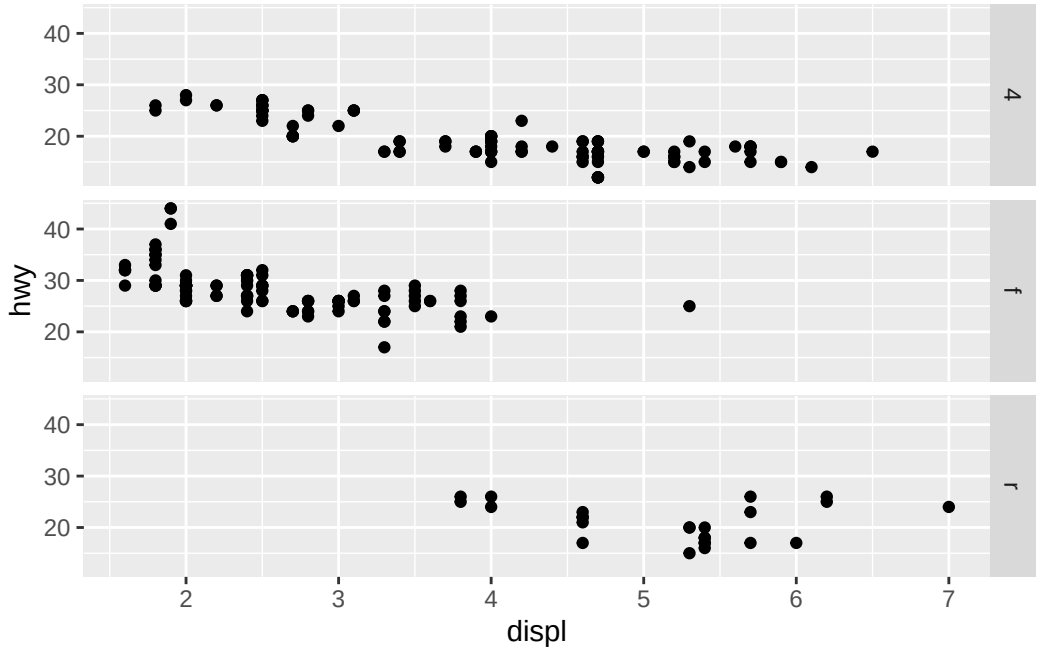
```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_grid(drv ~ .)
```




```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_wrap(~drv, ncol = 1)
```

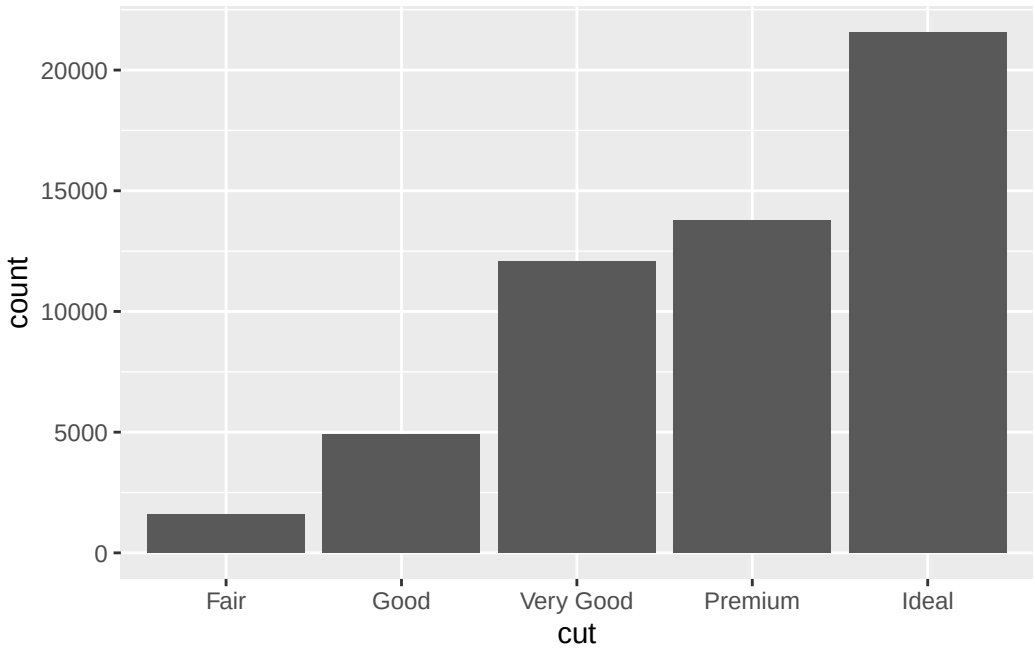


```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_wrap(~drv, ncol = 1, strip.position = "right")
```



9.5 Statistical Transformations

```
ggplot(diamonds, aes(x = cut)) +
  geom_bar()
```




```
# d <- ggplot(diamonds, aes(x, y)) + xlim(4, 10) + ylim(4, 10)
# d + geom_bin_2d(drop = FALSE, bins = 6, boundary = 0.1)
```

there values plotted in a ggplot plot can either be raw values directly from the data, or statistical transformations of the data:

- raw values
  - scatter plot (`geom_point`)
- binned counts: bin data then plot bin counts (number of points falling in each bin)
  - bar charts, histograms, frequency polygons
- smoother: fit model to data and plot predictions from model
- boxplots: compute five-number summary of distributuon and display as specially formatted box

algorithm used to calculate new values for graph is called `stat` (statistical transformation), see Figure 9.2 from book embedded below

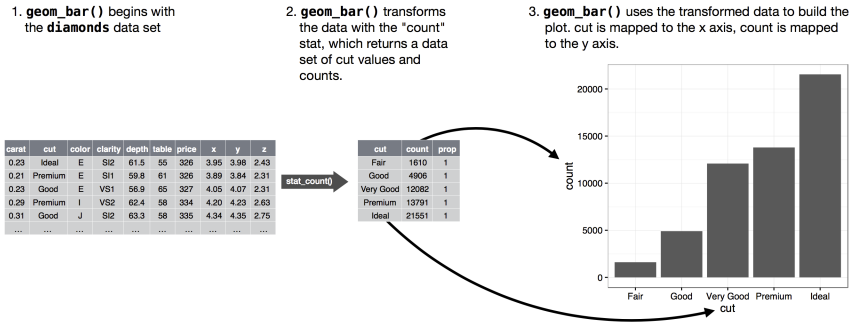


Figure 2: Figure 9.2: When creating a bar chart we first start with the raw data, then aggregate it to count the number of observations in each bar, and finally map those computed variables to plot aesthetics.

- every geom has default stat, every stat has default geom
- can inspect default stat with ?geom
- sometimes, overriding the stat is desirable:

```
result <- tryCatch({
  # The 'try' block: trying to convert text to numeric
  # This triggers an error: ! `stat_count()` must only have an x or y aesthetic.

  p <- diamonds |>
    count(cut) |> # we already computed the counts. let's use them
    ggplot(aes(x = cut, y = n)) +  # causes failure
    geom_bar()                     # causes failure
  print(p)
}, warning = function(w) {
  # The 'warning' block (Python doesn't have a direct equivalent for this)
  message(paste("Intercepted a warning:", w$message))

  # You can choose what to return if a warning happens:
  return("!? a warning occured ?!")
}, error = function(e) {

  # The 'except' block for hard failures
  message(paste("An error occurred:", e$message))
  print(e)
  return("!! an error occured !!")
}, finally = {
  # The 'finally' block
```

```
  message("Execution finished.")
})
```

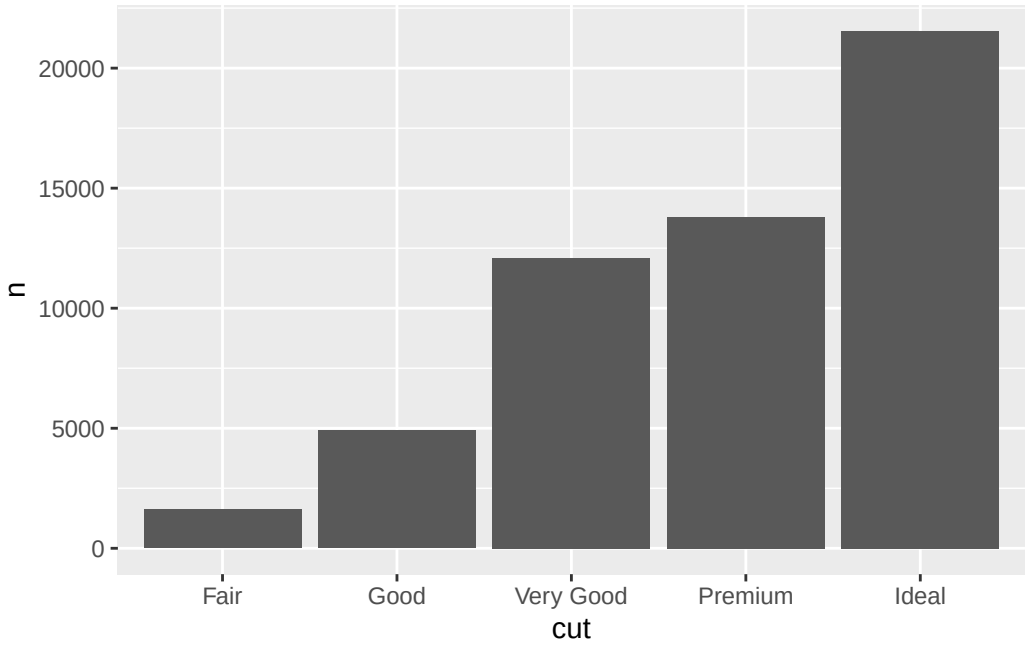
```
An error occurred: Problem while computing stat.

<error/rlang_error>
Error in `geom_bar()`:
! Problem while computing stat.
i Error occurred in the 1st layer.
Caused by error in `setup_params()`:
! `stat_count()` must only have an x or y aesthetic.
---
Backtrace:
     x
  1. +-base::tryCatch(...)
  2. | \-base (local) tryCatchList(expr, classes, parentenv, handlers)
  3. |   +-base (local) tryCatchOne(...)
  4. |   | \-base (local) doTryCatch(return(expr), name, parentenv, handler)
  5. |   \-base (local) tryCatchList(expr, names[-nh], parentenv, handlers[-
nh])
  6. |     \-base (local) tryCatchOne(expr, names, parentenv, handlers[[1L]])
  7. |       \-base (local) doTryCatch(return(expr), name, parentenv, handler)
  8. +-base::print(p)
  9. \-ggplot2 (local) `print.ggplot2::ggplot`(p)
 10.   +-ggplot2::ggplot_build(x)
 11.   \-ggplot2 (local) `ggplot_build.ggplot2::ggplot`(x)
 12.     \-ggplot2:::by_layer(...)
 13.       +-rlang::try_fetch(...)
 14.       | +-base::tryCatch(...)
 15.       | | \-base (local) tryCatchList(expr, classes, parentenv, handlers)
 16.       | |   \-base (local) tryCatchOne(expr, names, parentenv, handlers[[1L]])
 17.       | |     \-base (local) doTryCatch(return(expr), name, parentenv, handler)
 18.       | \-base::withCallingHandlers(...)
 19.       \-ggplot2 (local) f(l = layers[[i]], d = data[[i]])
 20.         \-l$compute_statistic(d, layout)
 21.           \-ggplot2 (local) compute_statistic(..., self = self)
 22.             \-self$stat$setup_params(data, self$stat_params)
 23.               \-ggplot2 (local) setup_params(..., self = self)

Execution finished.
```

```
# Print the final result
print(result)
```

```
[1] "!! an error occured !!"
```

1. `stat = "identity"` allows us to map the stat of a geom to raw value in data

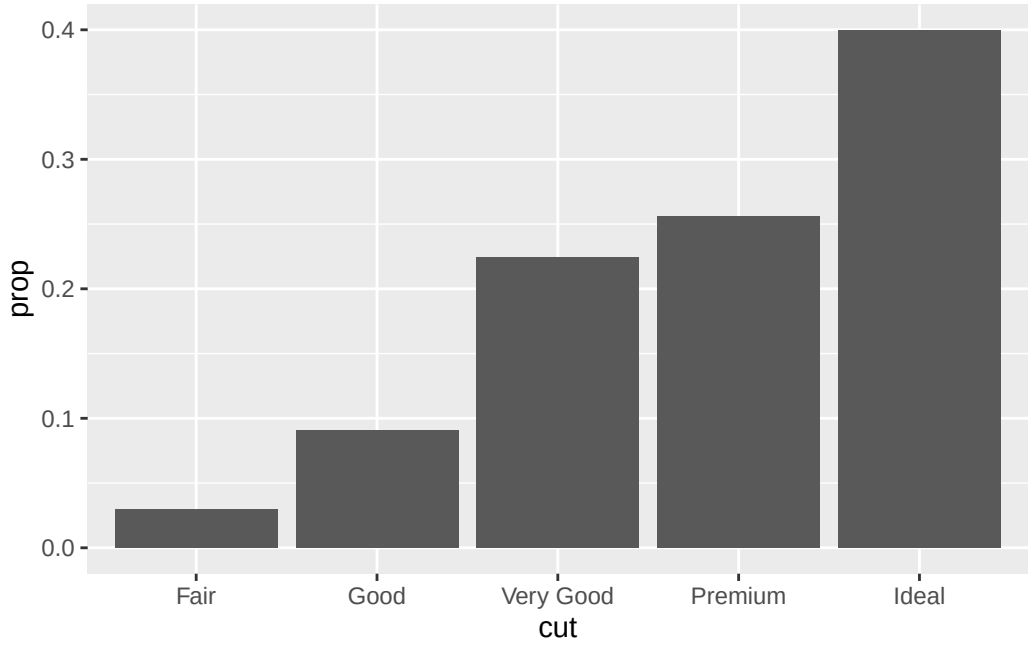
```
diamonds |>
  count(cut) |>
  ggplot(aes(x = cut, y = n)) +
  geom_bar(stat = "identity")
```



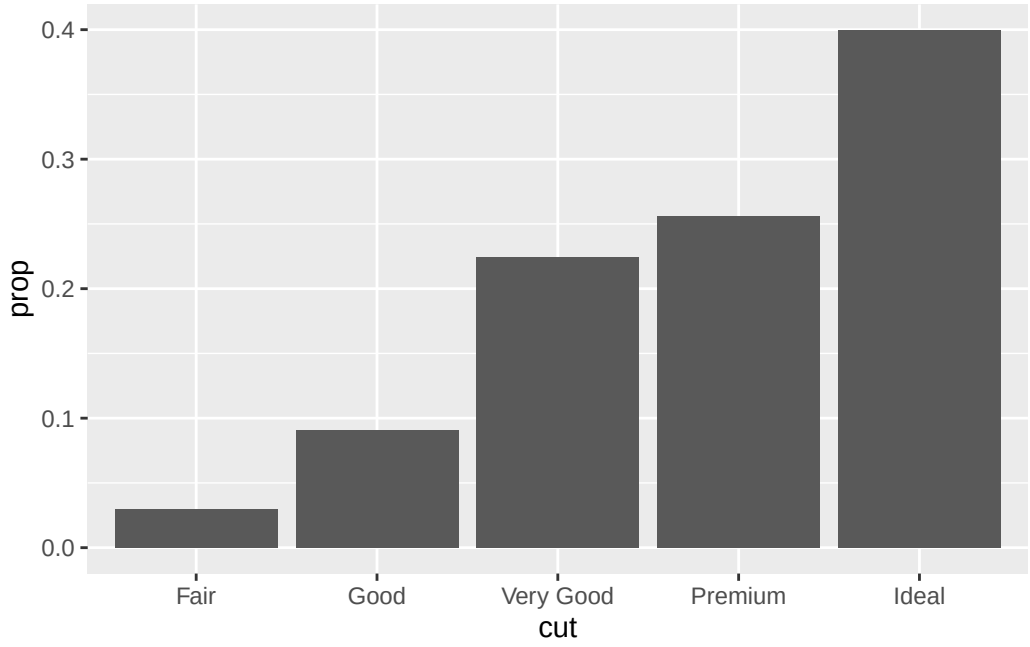
2. override default mapping from transformed variables to aesthetics. for example, display bar chart of proportions instead of counts

    1. note: to find possible vars computed by stat, look for section "computed variables" in the help for geom
    2. `after_stat`, `after_scale`, and `stage` can be used to select at which step in the ggplot2 plot creation pipeline the data will be mapped to `aes`

```
# this is equivalent to plot with after stat below!
diamonds |>
  count(cut) |>
  mutate(prop = n / sum(n)) |>
  ggplot(aes(x = cut, y = prop, group = 1)) +
  geom_bar(stat = "identity")
```

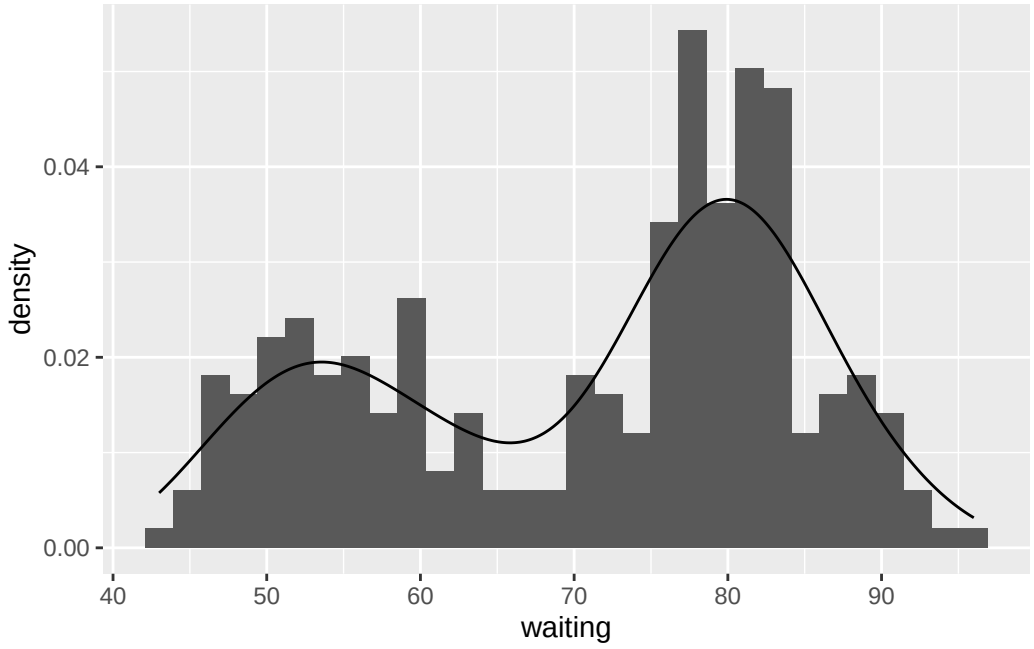



```
# this is equivalent to plot manually calculating stats above
ggplot(diamonds, aes(x = cut, y = after_stat(prop), group = 1)) +
  geom_bar()
```



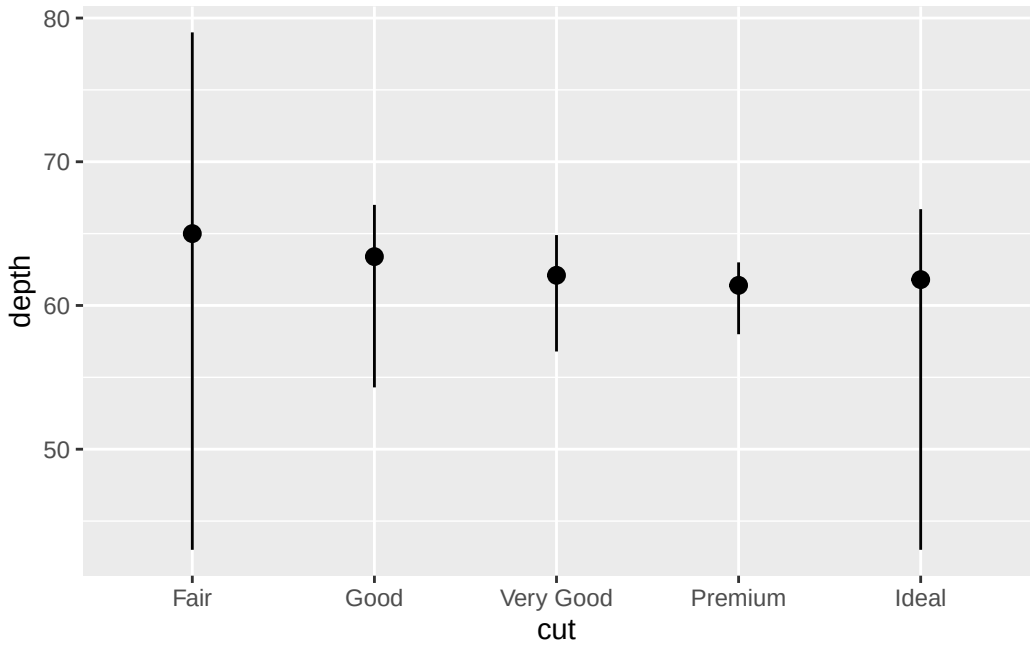
```
# Choosing a different computed variable to display, matching up the
# histogram with the density plot
ggplot(faithful, aes(x = waiting)) +
  geom_histogram(aes(y = after_stat(density))) +
  geom_density()
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



3. draw greater attention to statistical transformation. for example: might use `stat_summary` to draw attention to summary being computed (`stat_summary` summarizes y values for each unique x value)

```
# you can straight up add a stat to ggplot???
ggplot(diamonds) +
  stat_summary(
    aes(x = cut, y = depth),
    fun.min = min,
    fun.max = max,
    fun = median
  )
```

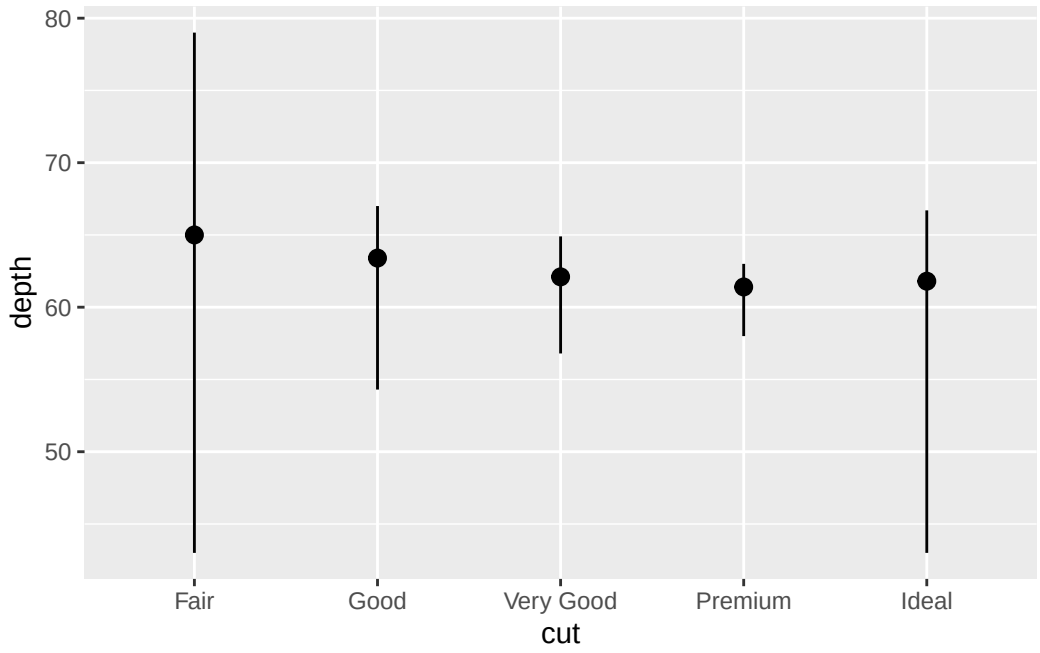


9.5.1 Exercises

1. What is the default geom associated with stat_summary()? How could you rewrite the previous plot to use that geom function instead of the stat function?

- answer: `geom = "pointrange"` is the default. see below for how to use `geom_pointrange` to reproduce plot from above.

```
?stat_summary
# argument geom = "pointrange"

# make previous plot
ggplot(diamonds, aes(x = cut, y = depth)) +
geom_pointrange(
  aes(x = cut, y = depth),
  # specify which stat we want
  stat = "summary",
  # arguments that go into stat_summary can be placed into geom
  fun.max = max,
  fun.min = min,
  fun = median
)
```




```
# # make previous plot
# ggplot(diamonds, aes(x = cut, y = depth)) +
#   geom_pointrange(stat = "summary")
```

2. What does `geom_col()` do? How is it different from `geom_bar()`?

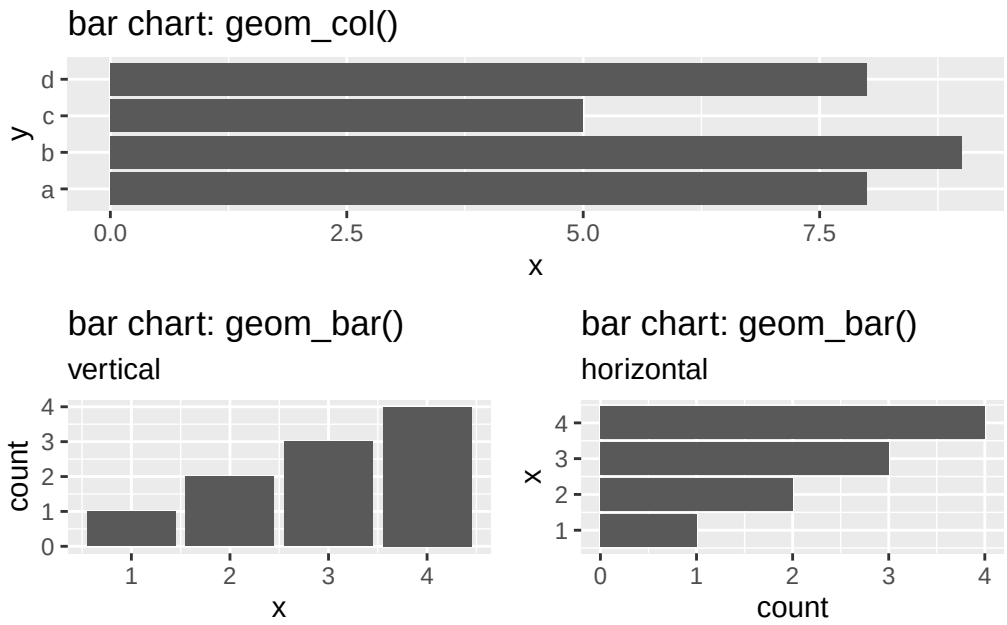
- answer: `geom_col()` is a bar chart where heights match actual values in the data whereas `geom_bar` is a bar chart where heights represents number of cases in each group (or if `weight` aesthetic is applied, then sum of weights). `geom_col` requires `x` and `y` args since the one axis determines bar position while the other axis determines the bar height (by default `y` for position, `x` for height). `geom_bar` is `x` XOR `y` (only assign ONE variable to either), and then it counts/bins each unique instance of that variable; `x` means a vertical bar chart by counting binned data, `y` means a horizontal bar chart by counting binned data.

```
library(patchwork)
# fake_data = tibble(x = rep(c(1:3),3), y = letters[1:9])
fake_data <- tibble(x = c(1, 2, 2, 3, 3, 3, 4, 4, 4, 4), y = c(rep(c("a", "b", "c"), 2),

# geom_col takes the sum of each x value for the bin and this becomes height
plot_fake_data_1 <- ggplot(fake_data, aes(x = x, y = y)) +
geom_col() +
ggtitle("bar chart: geom_col()")
```

```r
# this shows the heights of the bars, which isnt accesible by plot_fake_data_1$data
fake_data |>
summarize(sum(x), .by = y)
```

```
# A tibble: 4 x 2
  y     `sum(x)`
  <chr>    <dbl>
1 a            8
2 b            9
3 c            5
4 d            8
```

```r
# geom_bar counts the x
plot_fake_data_2 <- ggplot(fake_data, aes(x = x)) +
geom_bar() +
ggtitle("bar chart: geom_bar()", subtitle = "vertical")

# example of horizontal
plot_fake_data_3 <- ggplot(fake_data, aes(y = x)) +
geom_bar() +
ggtitle("bar chart: geom_bar()", subtitle = "horizontal")

# side by side
plot_fake_data_1 / (plot_fake_data_2 + plot_fake_data_3)
```



3. Most geoms and stats come in pairs that are almost always used in concert. Make a list of all the pairs. What do they have in common? (Hint: Read through the documentation.)

- note: data masking and tidy selection, how to use dplyr functions in your own functions and for loops (how to loop over columns of a dataframe!): https://dplyr.tidyverse.org/articles/programming.html

  – answer: https://ggplot2.tidyverse.org/reference/index.html, generally they have in common the operation they do (geom_X and stat_X, X is the common portion). note: this table is based on the entire geoms section rather than just finding geoms that have a paired stat, so there is more info here. there is also missing info for stats that don't have a geom

geom	stat	description
geom_abline	NA	add reference line for annotating plots
geom_hline	NA	add reference line for annotating plots
geom_vline	NA	add reference line for annotating plots

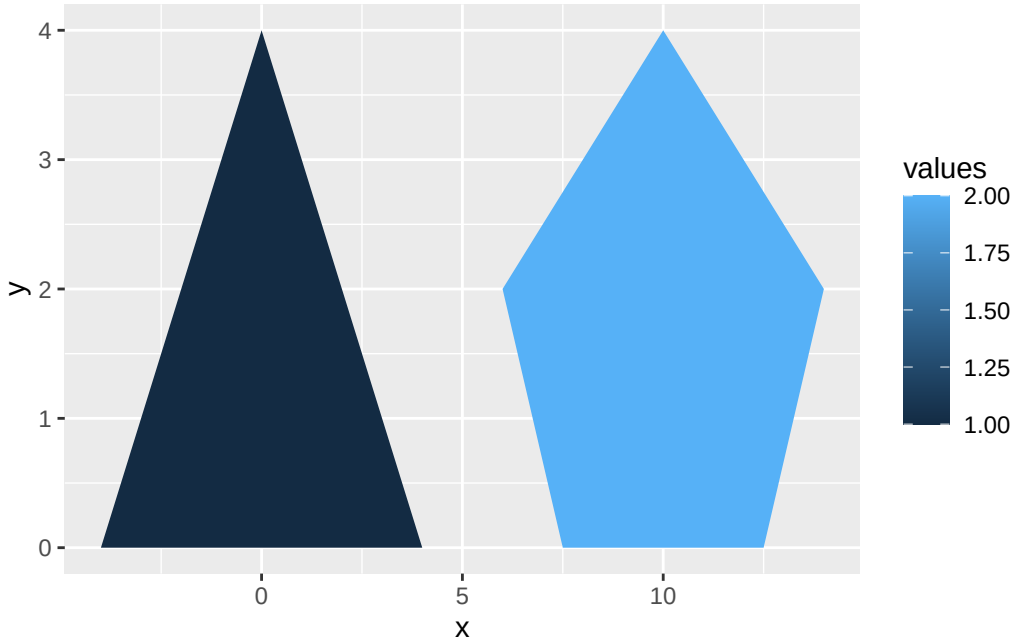
geom	stat	description
geom_bar	stat_count	bar chart, after aggregating counts of specified x var (can override by setting `stat = "identity"` if the x values are already aggregated and counts given in y)
geom_col	NA	bar chart, using values specified in y for heights
geom_bin_2d	stat_bin_2d	heatmap of 2D bin counts
geom_blank	NA	do nothing
geom_boxplot	stat_boxplot	box and whisker plots (Tukey style)
geom_contour	stat_contour	visualize 3D surface in 2D
geom_contour_filled	stat_contour_filled	visualize 3D surface in 2D
geom_count	stat_sum	variant of geom_point to count overlapping points for discrete data (size of point will correspond to # of points with same exact x,y val)
geom_density	stat_density	smoothed density estimates
geom_density_2d	stat_density_2d	contours of 2D density estimate, just contour lines
geom_density_2d_filled	stat_density_2d_filled	contours of 2D density estimate, filled contour bands
geom_dotplot	NA	dot plot
geom_function	stat_function	draw a function as continous curve
geom_hex	stat_bin_hex	hexagonal heatmap of 2d bin counts. hexagon bins help avoid "visual artifacts sometimes generated by the very regular alignment of geom_bin_2d"
geom_freqpoly	stat_bin	ditto geom_histogram description. frequency polygons more suitable when comparing the distribution of a single continuous variable across levels of a categorical variable
geom_histogram	stat_bin	visualize distribution of single continuous variable by dividing x axis into bins and counting number of observations in each bin

geom	stat	description
geom_jitter	NA	shortcut for `geom_point(position = "jitter")`, adds a small amount of random variation to location of each point. especially useful when overplotting due to discreteness in smaller datesets
geom_crossbar	NA	represent vertical interval defined by `x`, `ymin`, and `ymax`, drawing a single graphical object .
geom_errorbar	NA	represent vertical interval defined by `x`, `ymin`, and `ymax`, drawing a single graphical object. error bars (typically for replicates or confidence intervals)
~~geom_errorbarh~~	NA	~~represent vertical interval defined by `x`, `ymin`, and `ymax`, drawing a single graphical object~~. function is deprecated, use `geom_errorbar(orientation = "y")`
geom_linerange	NA	represent vertical interval defined by `x`, `ymin`, and `ymax`, drawing a single graphical object. just a line
geom_pointrange	NA	represent vertical interval defined by `x`, `ymin`, and `ymax`, drawing a single graphical object. linerange but with a point on line defined by `y`
geom_map	NA	polygons from a reference map
geom_path	NA	connect observations. connect following order in the data
geom_line	NA	connect observations. connect in order of the x axis
geom_step	NA	connect observations. connect in order of x axis but in a step-wise fashion to highlight y axis changes
geom_point	NA	scatter plot
geom_polygon	NA	polygons
geom_qq_line	stat_qq_line	slope and intercept connecting points at specified quartiles of theoretical and sample distributions

geom	stat	description
geom_qq	stat_qq	quantile-quantile plots (see if data from column in df follows a theoretical distribution, typically normal. if follows the theoretical dist then will fall along line from stat_qq_line)
geom_quantile	stat_quantile	quantile regression
geom_ribbon	stat_align	ribbon and area plots
geom_area	stat_align	ribbon and area plots
geom_rug	NA	rug plots in the margins (rug plot = compact visual to supplement a 2d display with the two 1d marginal distributions)
geom_segment	NA	line segments
geom_curve	NA	line curves
geom_smooth	stat_smooth	smoothed conditional means
geom_spoke	NA	line segments parameterised by location, direction and distance (useful for vector fields?)
geom_label	NA	add text to geom
geom_text	NA	add text to geom
geom_raster	NA	rectangles
geom_rect	NA	rectangles
geom_tile	NA	rectangles (sometimes used for heatmaps?)
geom_violin	stat_ydensity	violin plot
geom_sf, coord_sf, geom_sf_label, geom_sf_text	stat_sf	visualize sf (simple feature) objects

```
offset <- 10
ids <- c("Triangle", "Pentagon")
values <- data.frame(
  id = ids,
  values = c(1, 2)
)
positions <- data.frame(
  id = c(rep("Triangle", 3), rep("Pentagon", 5)),
  x = c(
# triangle
```

```
-4, 0, 4,

# pentagon
-2.5 + offset * 1, -4 + offset * 1, 0 + offset * 1, 4 + offset * 1, 2.5 + offset * 1
  ),
  y = c(
# triangle
0, 4, 0,

# pentagon
0, 2, 4, 2, 0
  )
)

df_datapoly <- merge(values, positions, by = c("id"))

p <- ggplot(df_datapoly, aes(x = x, y = y)) +
  geom_polygon(aes(fill = values, group = id))
p
```



4. What variables does `stat_smooth()` compute? What arguments control its behavior?

- answer:

- `stat_smooth()` helps eye see patterns in presence of overplotting. computes the following variables:
  - `after_stat(y)` or `after_stat(x)`, predicted value
  - `after_stat(ymin)` or `after_stat(xmin)` lower pointwise confidence interval around mean
  - `after_stat(ymax)` or `after_stat(xmax)` upper pointwise confidence interval around mean
  - `after_stat(se)` standard error
- the most important arguments controlling this behavior are `method`, `formula`, and `orientation`. these will affect statistical transformations performed and therefore what is displayed

5. In our proportion bar chart, we needed to set `group = 1`. Why? In other words, what is the problem with these two graphs?

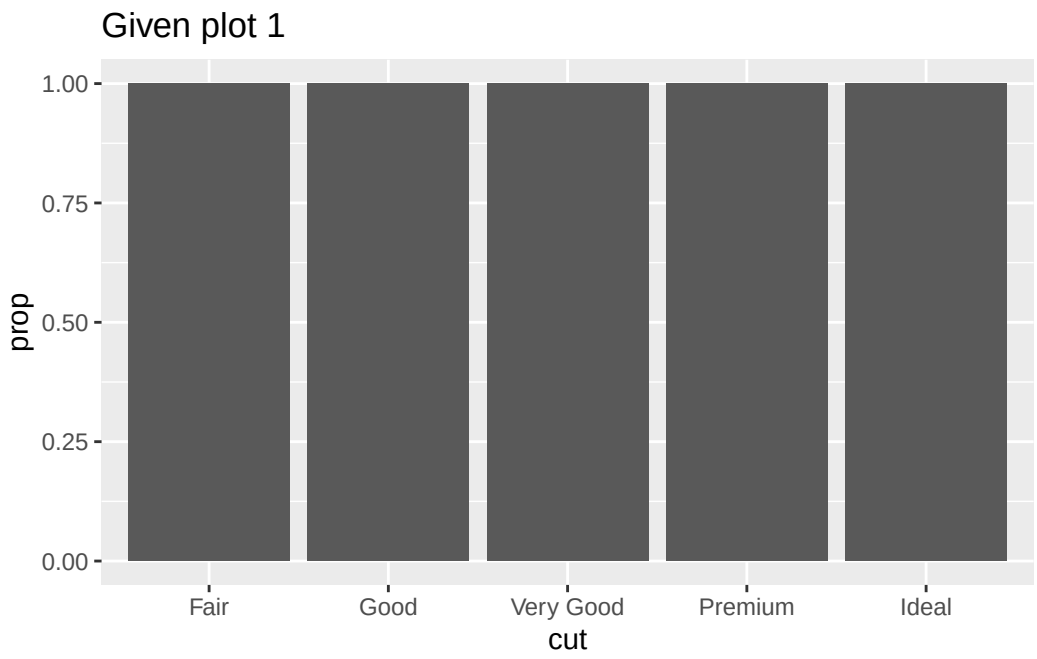
- # visual result same as original

  ggplot(diamonds, aes(x = cut, y = after_stat(prop), group = "1")) +

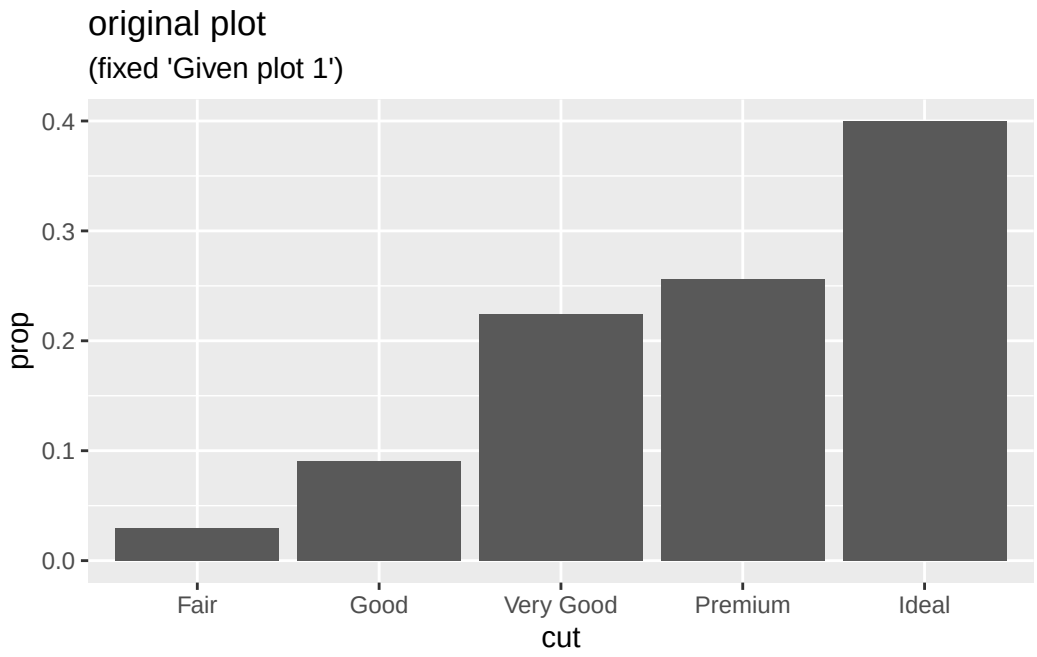
  geom_bar()

- answer: for 'Given plot 1' setting `group = 1` is saying that the value of each group should be static, the same thing can be achieved by setting group to any integer or to a string that is not contained in the dataset, group = 100, group = "a" give same as group = 1; this is because there is only 1 variable being displayed in the plot in 1 way, which is displayed in the bars.
  - https://ggplot2.tidyverse.org/reference/aes_group_order.html

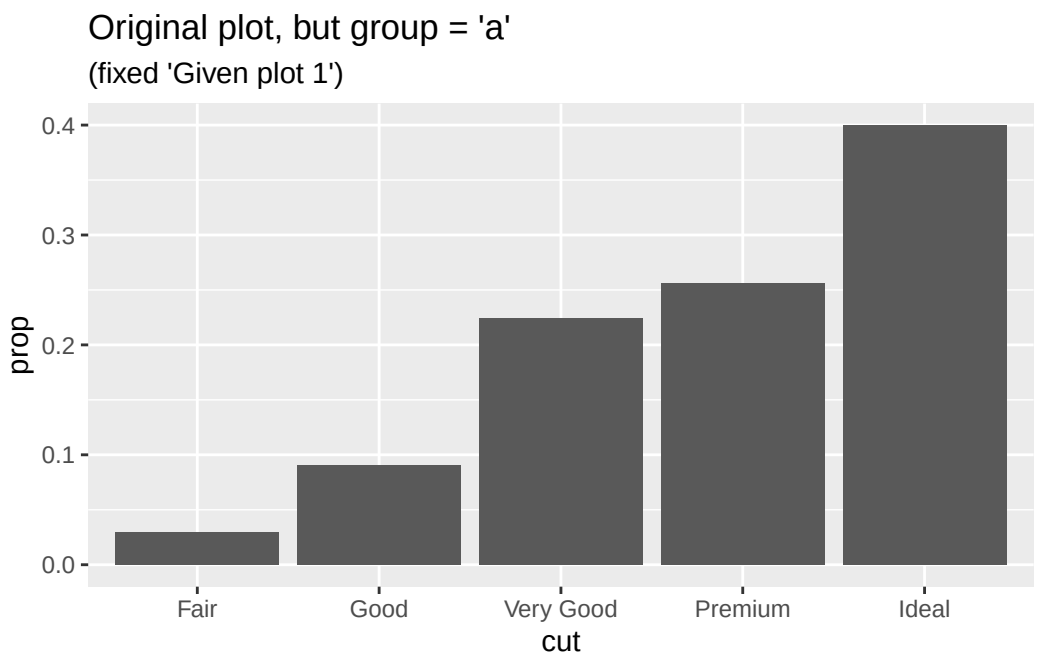
```
ggplot(diamonds, aes(x = cut, y = after_stat(prop))) +
geom_bar() +
ggtitle("Given plot 1")
```



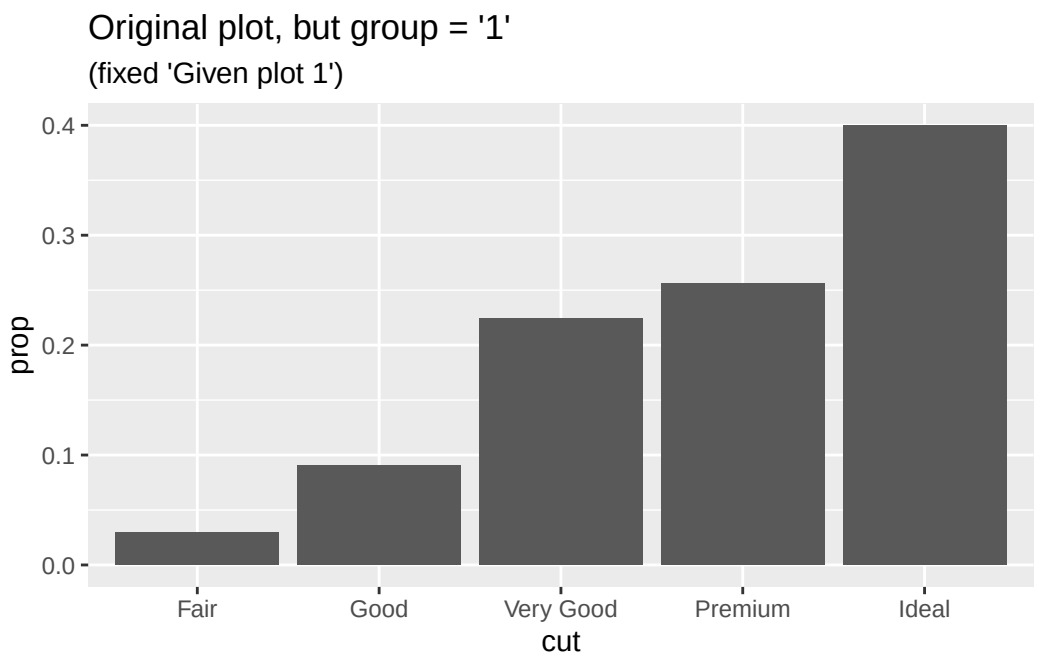

```
# original plot
ggplot(diamonds, aes(x = cut, y = after_stat(prop), group = 1)) +
geom_bar() +
ggtitle("original plot", subtitle = "(fixed 'Given plot 1')")
```



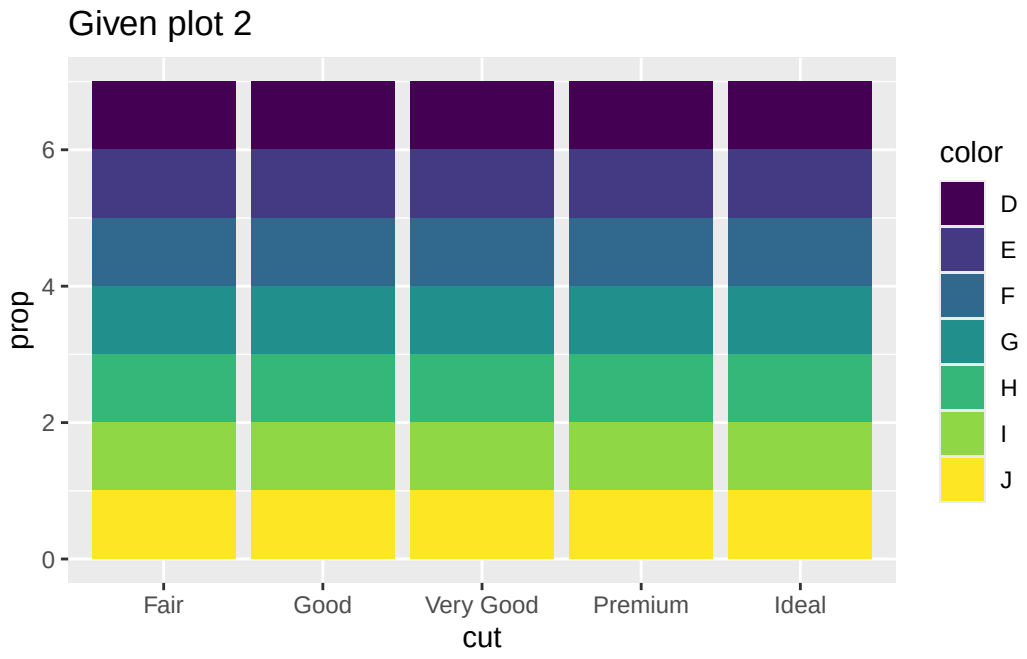
```
# visual result same as original
ggplot(diamonds, aes(x = cut, y = after_stat(prop), group = "a")) +
geom_bar() +
ggtitle("Original plot, but group = 'a'", subtitle = "(fixed 'Given plot 1')")
```



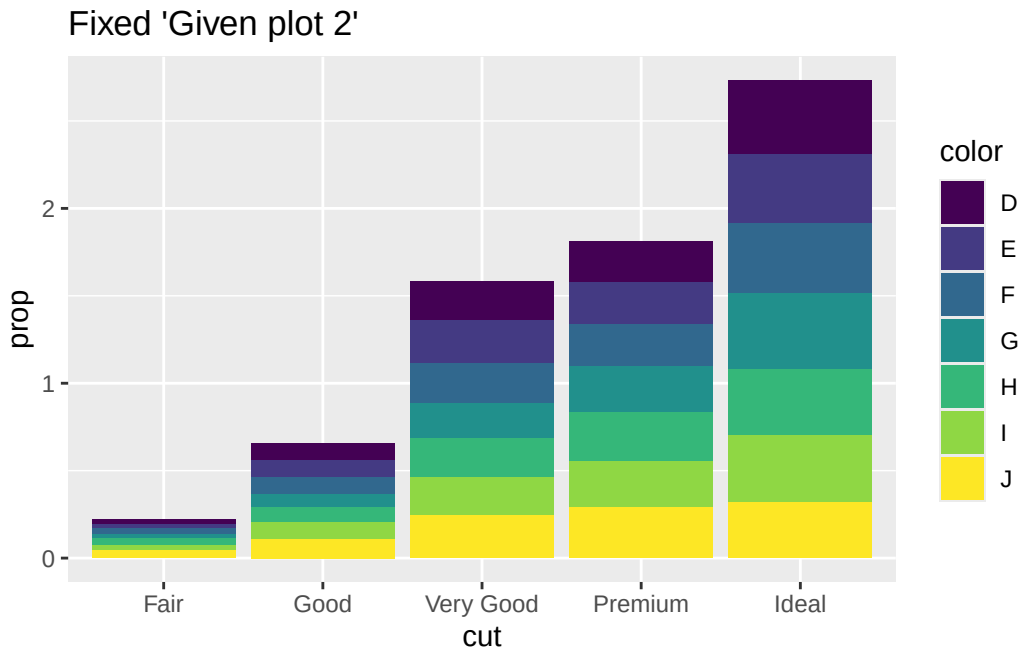

```
# visual result same as original
ggplot(diamonds, aes(x = cut, y = after_stat(prop), group = "1")) +
geom_bar() +
ggtitle("Original plot, but group = '1'", subtitle = "(fixed 'Given plot 1')")
```



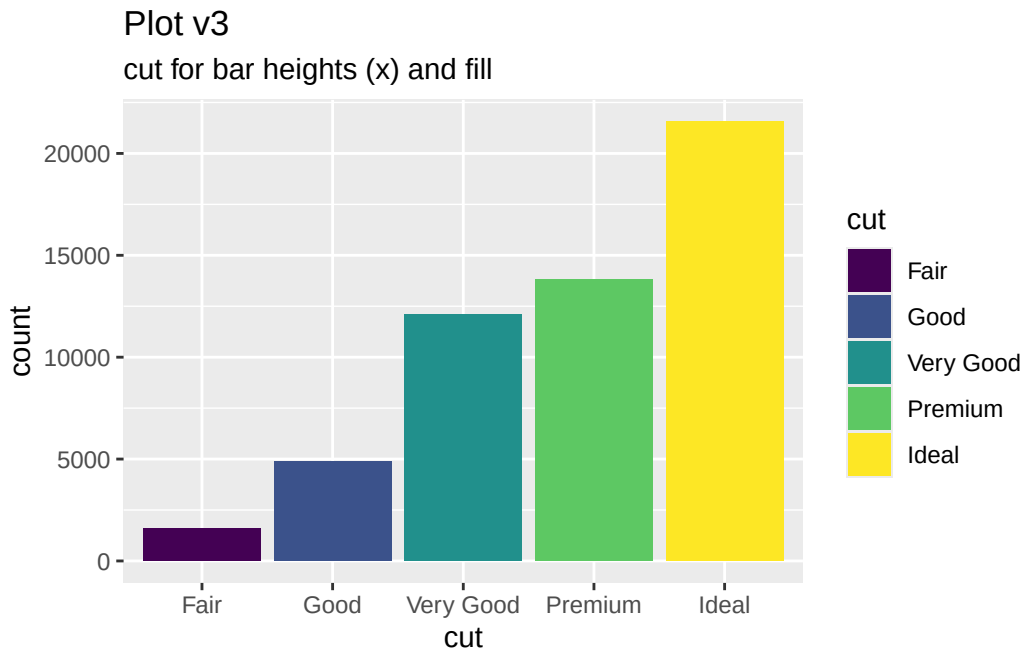

```
ggplot(diamonds, aes(x = cut, fill = color, y = after_stat(prop))) +
geom_bar() +
ggtitle("Given plot 2")
```



```
ggplot(diamonds, aes(x = cut, fill = color, y = after_stat(prop), group = color)) +
geom_bar() +
ggtitle("Fixed 'Given plot 2'")
```




```
ggplot(diamonds, aes(x = cut, fill = cut)) +
geom_bar() +
ggtitle("Plot v3", subtitle = "cut for bar heights (x) and fill")
```




```
df_diamonds_cut_marginalized_by_color <- diamonds |>
  count(cut, color) |>
  ungroup() |>
  group_by(cut) |>
  reframe(
    cut = cut,
    color = color,
    n_cut_color = n,
    n_cut = sum(n)
  ) |>
  ungroup() |>
  mutate(
    marginal_prop_cut_color = n_cut_color / n_cut
  )

df_diamonds_cut_marginalized_by_color
```

```
# A tibble: 35 x 5
   cut   color n_cut_color n_cut marginal_prop_cut_color
   <ord> <ord>       <int> <int>                   <dbl>
 1 Fair  D             163  1610                   0.101
 2 Fair  E             224  1610                   0.139
```

```
 3 Fair  F             312  1610                  0.194
 4 Fair  G             314  1610                  0.195
 5 Fair  H             303  1610                  0.188
 6 Fair  I             175  1610                  0.109
 7 Fair  J             119  1610                  0.0739
 8 Good  D             662  4906                  0.135
 9 Good  E             933  4906                  0.190
10 Good  F             909  4906                  0.185
# i 25 more rows
```

```
# ensure  each cut group marginal_prop_cut_color sums to 1
df_diamonds_cut_marginalized_by_color |>
  group_by(cut) |>
  summarize(check = sum(marginal_prop_cut_color))
```

```
# A tibble: 5 x 2
  cut       check
  <ord>     <dbl>
1 Fair          1
2 Good          1
3 Very Good     1
4 Premium       1
5 Ideal         1
```

hmmm this data frame doesnt look like the values inside of "Fixed 'Given plot 2' " above... let's use `ggplot_build()` to get those values.

```
tmp <- ggplot(diamonds, aes(x = cut, fill = color, y = after_stat(prop), group = color)) +
  geom_bar() +
  ggtitle("Fixed 'Given plot 2'")

ggplot2::ggplot_build(tmp)$data |>
  data.frame()
```

```
   count       prop x flipped_aes      fill group PANEL          y       ymin
1    163 0.02405904 1       FALSE #440154FF     1     1 0.21857130 0.19451226
2    662 0.09771218 2       FALSE #440154FF     1     1 0.65548521 0.55777303
3   1513 0.22332103 3       FALSE #440154FF     1     1 1.58184074 1.35851971
4   1603 0.23660517 4       FALSE #440154FF     1     1 1.81370149 1.57709633
5   2834 0.41830258 5       FALSE #440154FF     1     1 2.73040126 2.31209868
6    224 0.02286414 1       FALSE #443A83FF     2     1 0.19451226 0.17164811
```

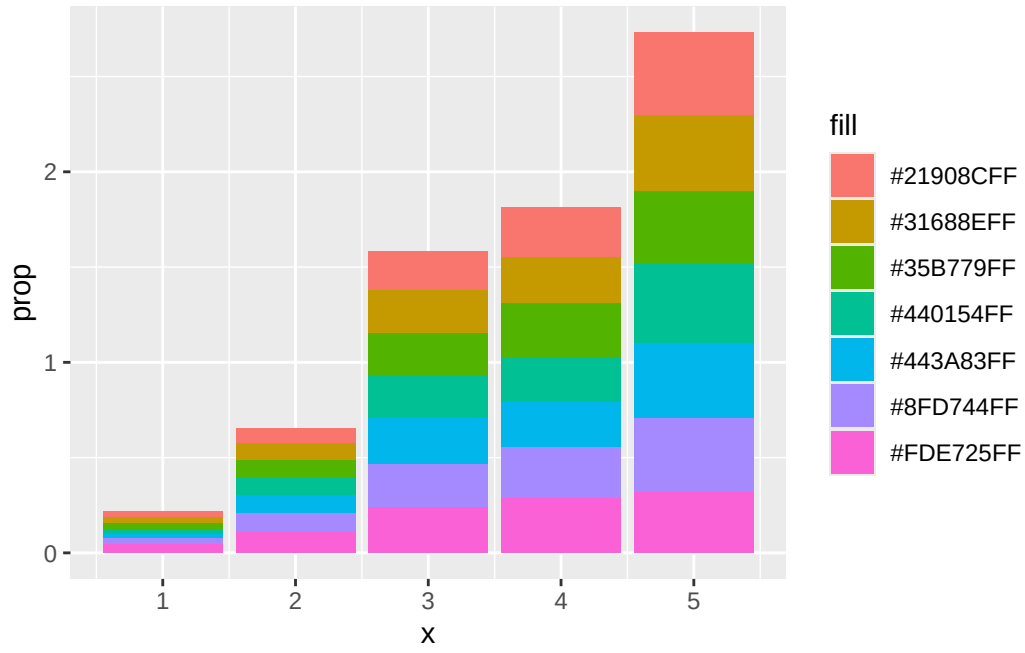
```
7    933 0.09523323 2       FALSE #443A83FF     2     1 0.55777303 0.46253980
8   2400 0.24497295 3       FALSE #443A83FF     2     1 1.35851971 1.11354675
9   2337 0.23854241 4       FALSE #443A83FF     2     1 1.57709633 1.33855392
10  3903 0.39838726 5       FALSE #443A83FF     2     1 2.31209868 1.91371142
11   312 0.03269755 1       FALSE #31688EFF     3     1 0.17164811 0.13895057
12   909 0.09526305 2       FALSE #31688EFF     3     1 0.46253980 0.36727675
13  2164 0.22678684 3       FALSE #31688EFF     3     1 1.11354675 0.88675992
14  2331 0.24428841 4       FALSE #31688EFF     3     1 1.33855392 1.09426551
15  3826 0.40096416 5       FALSE #31688EFF     3     1 1.91371142 1.51274726
16   314 0.02780730 1       FALSE #21908CFF     4     1 0.13895057 0.11114327
17   871 0.07713425 2       FALSE #21908CFF     4     1 0.36727675 0.29014249
18  2299 0.20359547 3       FALSE #21908CFF     4     1 0.88675992 0.68316445
19  2924 0.25894439 4       FALSE #21908CFF     4     1 1.09426551 0.83532112
20  4884 0.43251860 5       FALSE #21908CFF     4     1 1.51274726 1.08022866
21   303 0.03648844 1       FALSE #35B779FF     5     1 0.11114327 0.07465483
22   702 0.08453757 2       FALSE #35B779FF     5     1 0.29014249 0.20560492
23  1824 0.21965318 3       FALSE #35B779FF     5     1 0.68316445 0.46351127
24  2360 0.28420039 4       FALSE #35B779FF     5     1 0.83532112 0.55112074
25  3115 0.37512042 5       FALSE #35B779FF     5     1 1.08022866 0.70510824
26   175 0.03227591 1       FALSE #8FD744FF     6     1 0.07465483 0.04237892
27   522 0.09627444 2       FALSE #8FD744FF     6     1 0.20560492 0.10933048
28  1204 0.22205828 3       FALSE #8FD744FF     6     1 0.46351127 0.24145299
29  1428 0.26337145 4       FALSE #8FD744FF     6     1 0.55112074 0.28774929
30  2093 0.38601992 5       FALSE #8FD744FF     6     1 0.70510824 0.31908832
31   119 0.04237892 1       FALSE #FDE725FF     7     1 0.04237892 0.00000000
32   307 0.10933048 2       FALSE #FDE725FF     7     1 0.10933048 0.00000000
33   678 0.24145299 3       FALSE #FDE725FF     7     1 0.24145299 0.00000000
34   808 0.28774929 4       FALSE #FDE725FF     7     1 0.28774929 0.00000000
35   896 0.31908832 5       FALSE #FDE725FF     7     1 0.31908832 0.00000000
         ymax xmin xmax colour linewidth linetype alpha width
1  0.21857130 0.55 1.45     NA       0.5        1    NA   0.9
2  0.65548521 1.55 2.45     NA       0.5        1    NA   0.9
3  1.58184074 2.55 3.45     NA       0.5        1    NA   0.9
4  1.81370149 3.55 4.45     NA       0.5        1    NA   0.9
5  2.73040126 4.55 5.45     NA       0.5        1    NA   0.9
6  0.19451226 0.55 1.45     NA       0.5        1    NA   0.9
7  0.55777303 1.55 2.45     NA       0.5        1    NA   0.9
8  1.35851971 2.55 3.45     NA       0.5        1    NA   0.9
9  1.57709633 3.55 4.45     NA       0.5        1    NA   0.9
10 2.31209868 4.55 5.45     NA       0.5        1    NA   0.9
11 0.17164811 0.55 1.45     NA       0.5        1    NA   0.9
12 0.46253980 1.55 2.45     NA       0.5        1    NA   0.9
13 1.11354675 2.55 3.45     NA       0.5        1    NA   0.9
```

```
14 1.33855392 3.55 4.45     NA       0.5        1    NA   0.9
15 1.91371142 4.55 5.45     NA       0.5        1    NA   0.9
16 0.13895057 0.55 1.45     NA       0.5        1    NA   0.9
17 0.36727675 1.55 2.45     NA       0.5        1    NA   0.9
18 0.88675992 2.55 3.45     NA       0.5        1    NA   0.9
19 1.09426551 3.55 4.45     NA       0.5        1    NA   0.9
20 1.51274726 4.55 5.45     NA       0.5        1    NA   0.9
21 0.11114327 0.55 1.45     NA       0.5        1    NA   0.9
22 0.29014249 1.55 2.45     NA       0.5        1    NA   0.9
23 0.68316445 2.55 3.45     NA       0.5        1    NA   0.9
24 0.83532112 3.55 4.45     NA       0.5        1    NA   0.9
25 1.08022866 4.55 5.45     NA       0.5        1    NA   0.9
26 0.07465483 0.55 1.45     NA       0.5        1    NA   0.9
27 0.20560492 1.55 2.45     NA       0.5        1    NA   0.9
28 0.46351127 2.55 3.45     NA       0.5        1    NA   0.9
29 0.55112074 3.55 4.45     NA       0.5        1    NA   0.9
30 0.70510824 4.55 5.45     NA       0.5        1    NA   0.9
31 0.04237892 0.55 1.45     NA       0.5        1    NA   0.9
32 0.10933048 1.55 2.45     NA       0.5        1    NA   0.9
33 0.24145299 2.55 3.45     NA       0.5        1    NA   0.9
34 0.28774929 3.55 4.45     NA       0.5        1    NA   0.9
35 0.31908832 4.55 5.45     NA       0.5        1    NA   0.9
```

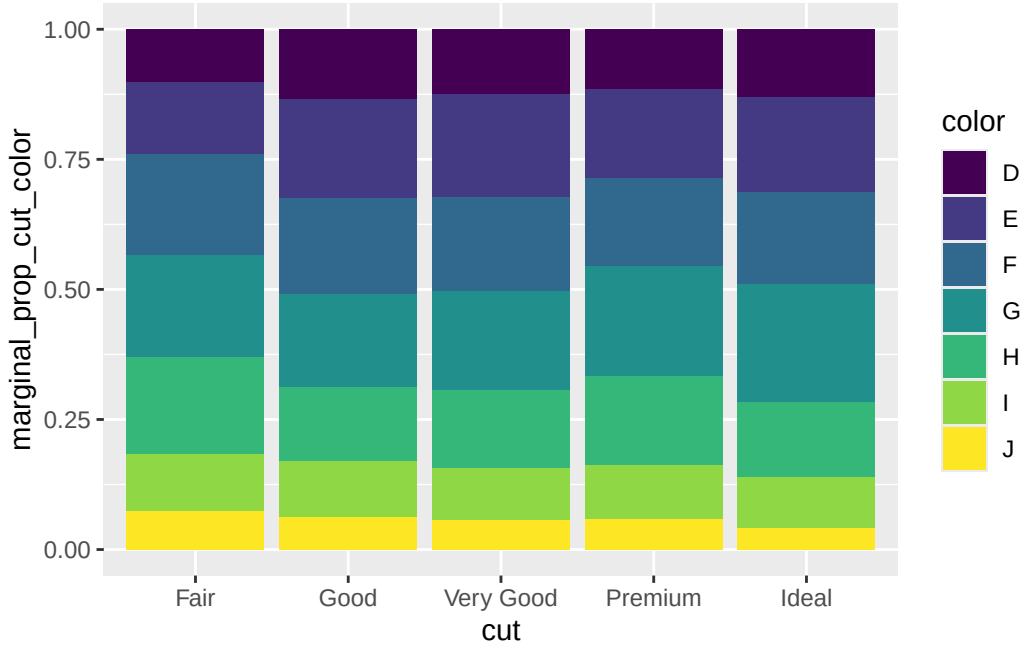
```r
  # View()

# FALSE
identical(
  ggplot_build(tmp)$data |> data.frame() |> dplyr::pull(prop) |> sort(),
  df_diamonds_cut_marginalized_by_color$marginal_prop_cut_color |> sort()
)
```

```
[1] FALSE
```

```r
ggplot_build(tmp)$data |>
  as.data.frame() |>
  ggplot(aes(x = x, y = prop, fill = fill)) +
  geom_col()
```



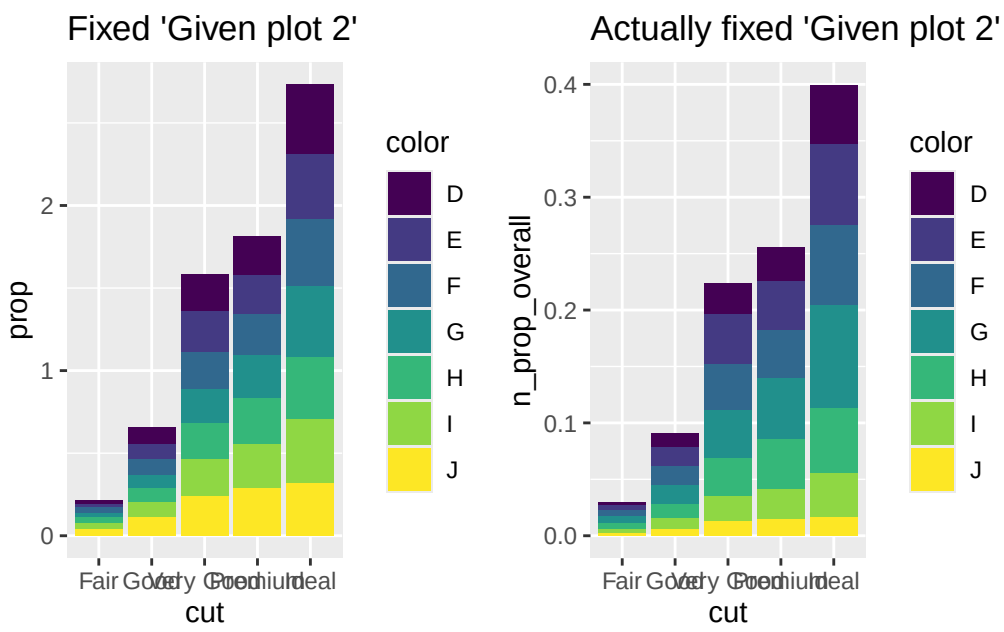

```
ggplot(df_diamonds_cut_marginalized_by_color, aes(x = cut, y = marginal_prop_cut_color, fill
  geom_col()
```



```r
df_diamonds_cut_marginalized_by_color_v2 <- diamonds |>
  count(cut, color) |>
  mutate(n_prop_overall = n / sum(n))

df_diamonds_cut_marginalized_by_color_v2
```

```
# A tibble: 35 x 4
   cut   color     n n_prop_overall
   <ord> <ord> <int>          <dbl>
 1 Fair  D       163        0.00302
 2 Fair  E       224        0.00415
 3 Fair  F       312        0.00578
 4 Fair  G       314        0.00582
 5 Fair  H       303        0.00562
 6 Fair  I       175        0.00324
 7 Fair  J       119        0.00221
 8 Good  D       662        0.0123
 9 Good  E       933        0.0173
10 Good  F       909        0.0169
# i 25 more rows
```

```r
tmp_v2 <- ggplot(df_diamonds_cut_marginalized_by_color_v2, aes(x = cut, y = n_prop_overall, 
  geom_col() +
  ggtitle("Actually fixed 'Given plot 2'")

library(patchwork)
tmp + tmp_v2
```

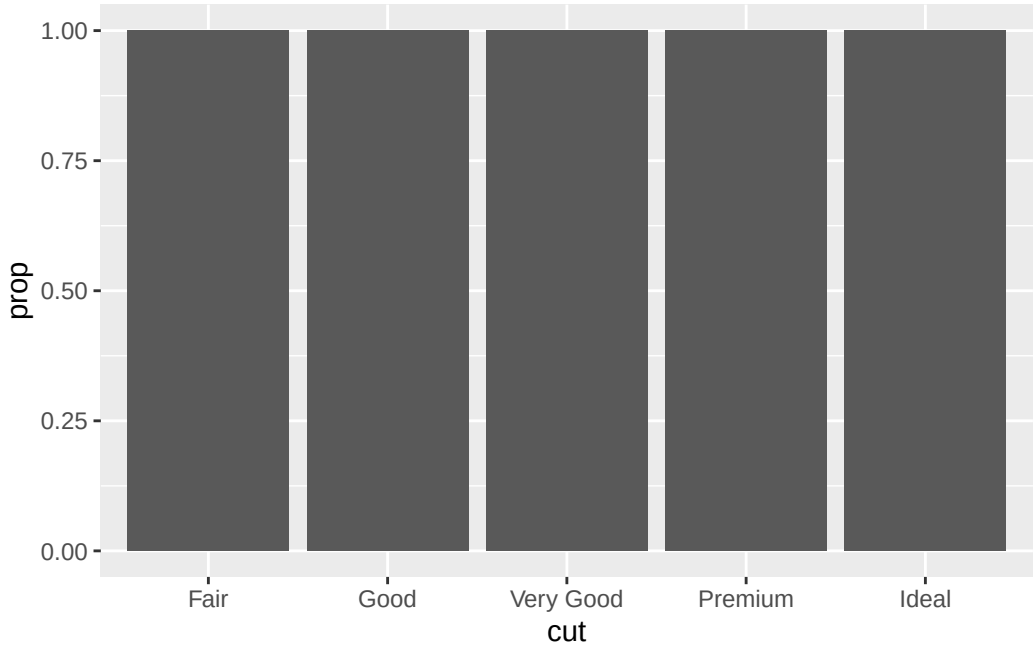



```
# FALSE
identical(
  ggplot_build(tmp)$data |> data.frame() |> dplyr::pull(prop) |> sort(),
  ggplot_build(tmp_v2)$data |> data.frame() |> dplyr::pull(y) |> sort()
)
```

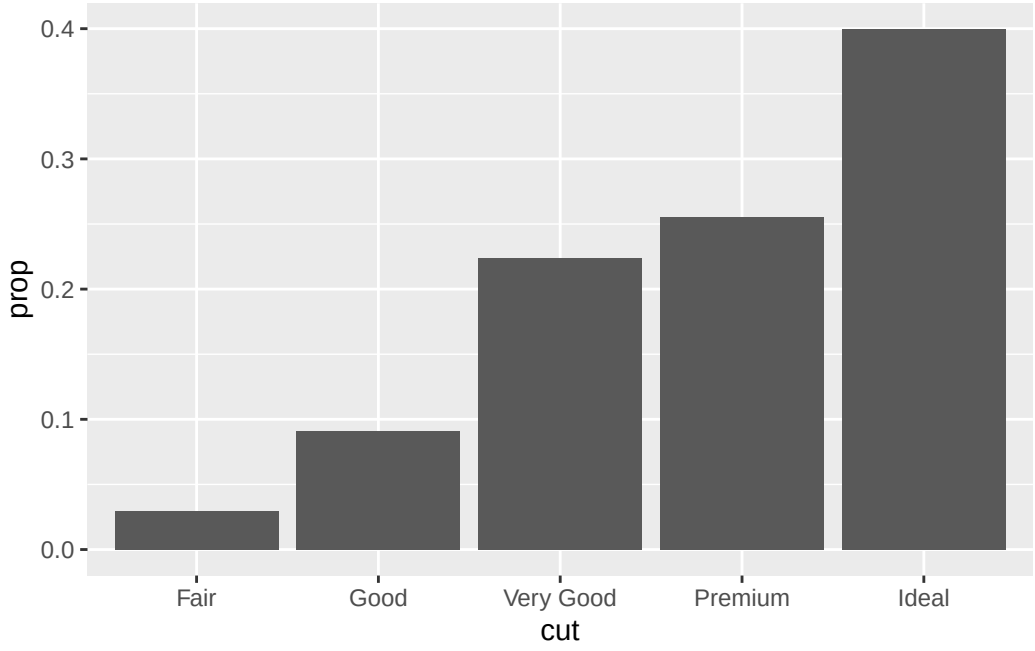
```
[1] FALSE
```

- key take away: i think setting `group = color` leads to a meaningless plot… comparing `tmp` (using `group = color`) to `tmp_v2` (n_prop_overall) shows that the y axis `prop` makes more sense in `v2`. im not really sure why the prop ranges from 0 to 4 in the first version… the dataframe doesnt have values of 4.
- from Mine Çetinkaya-Rundel's solution, "setting `group = 1` results in marginal proportions of `cut`s being plotted, and setting `group = color` results in proportions of `color`s within each `cut` being plotted." I believe the default is proportions wrt to entire data whereas "marginal" is the proportion wrt to each category of the x axis (the marginal distribution given the current x axis category)

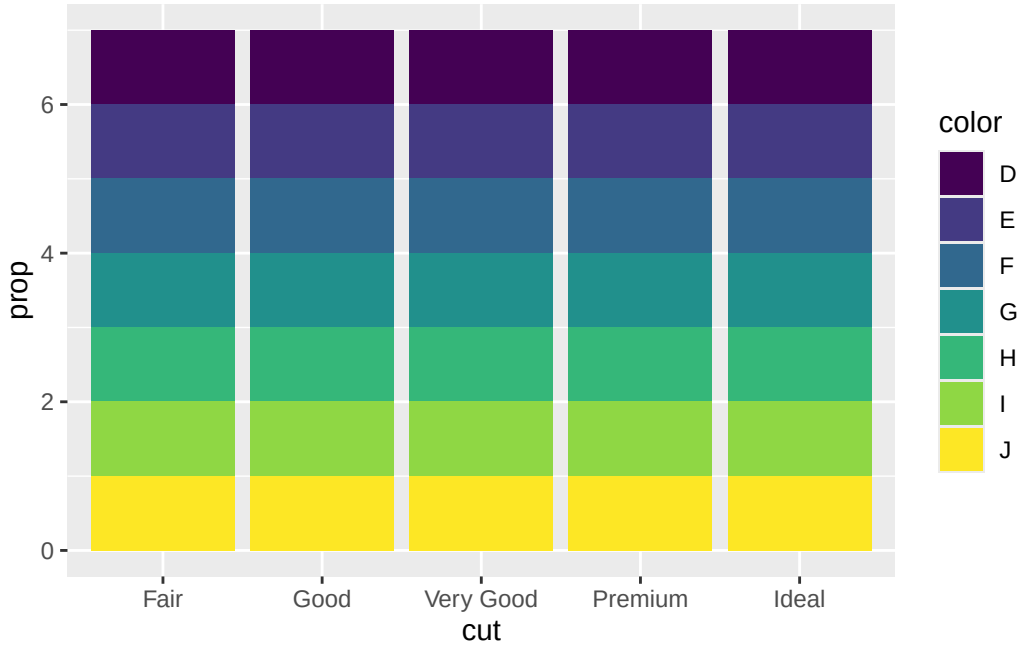
```
# one variable
ggplot(diamonds, aes(x = cut, y = after_stat(prop))) +
  geom_bar()
```



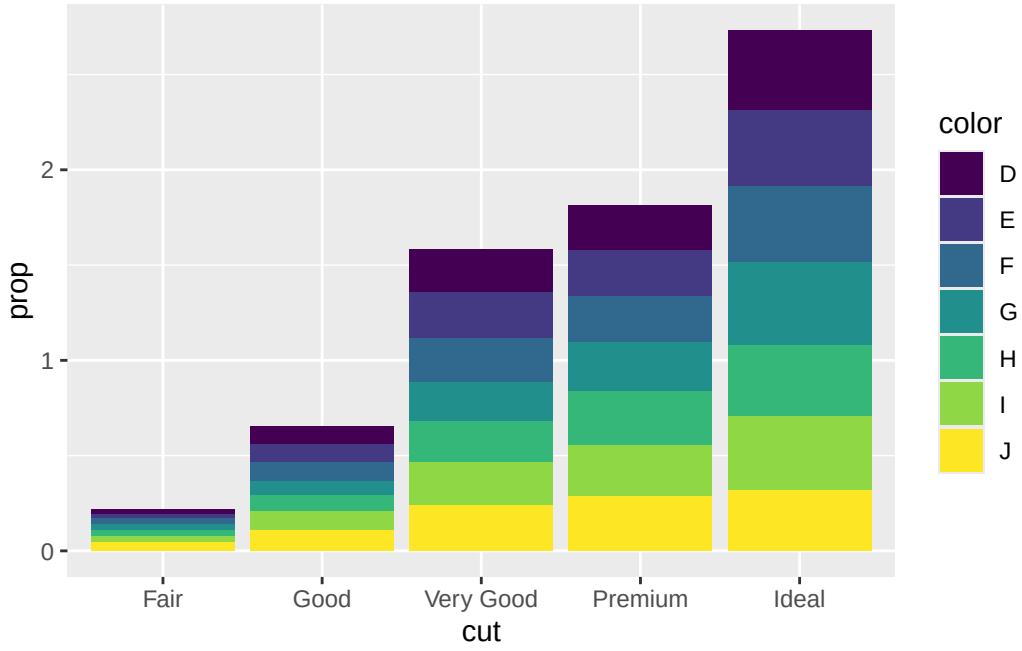

```
ggplot(diamonds, aes(x = cut, y = after_stat(prop), group = 1)) +
  geom_bar()
```



```r
# two variables
ggplot(diamonds, aes(x = cut, fill = color, y = after_stat(prop))) +
  geom_bar()
```



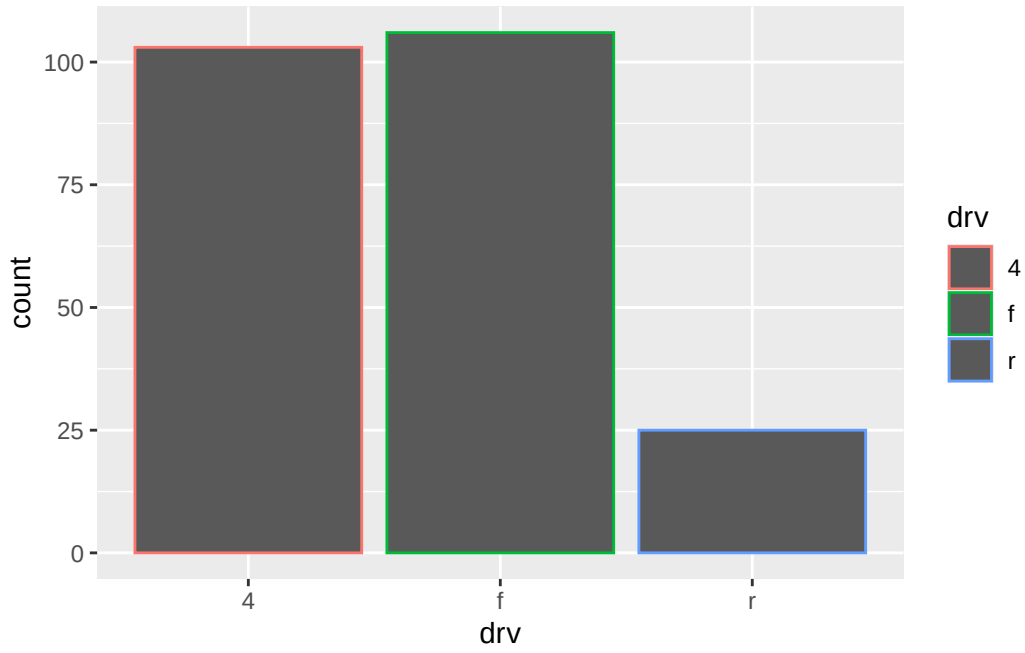

```r
ggplot(diamonds, aes(x = cut, fill = color, y = after_stat(prop), group = color)) +
  geom_bar()
```



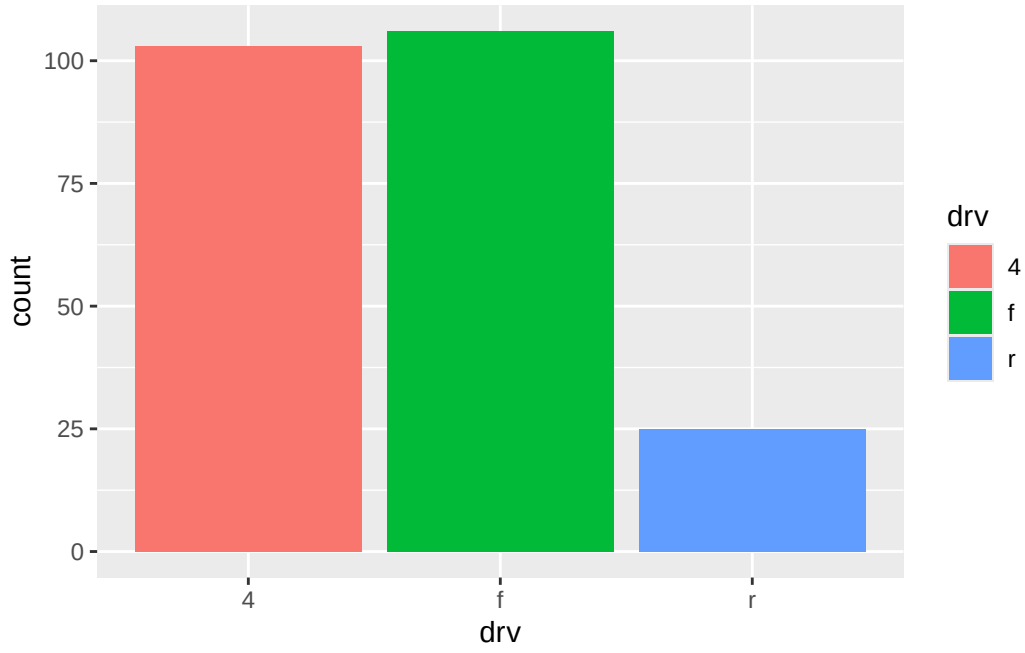
9.6 Position adjustments

- `geom_bar`:
  - fill to change color
    - `fill` + `position = identity` = stacked bar chart (ditto)

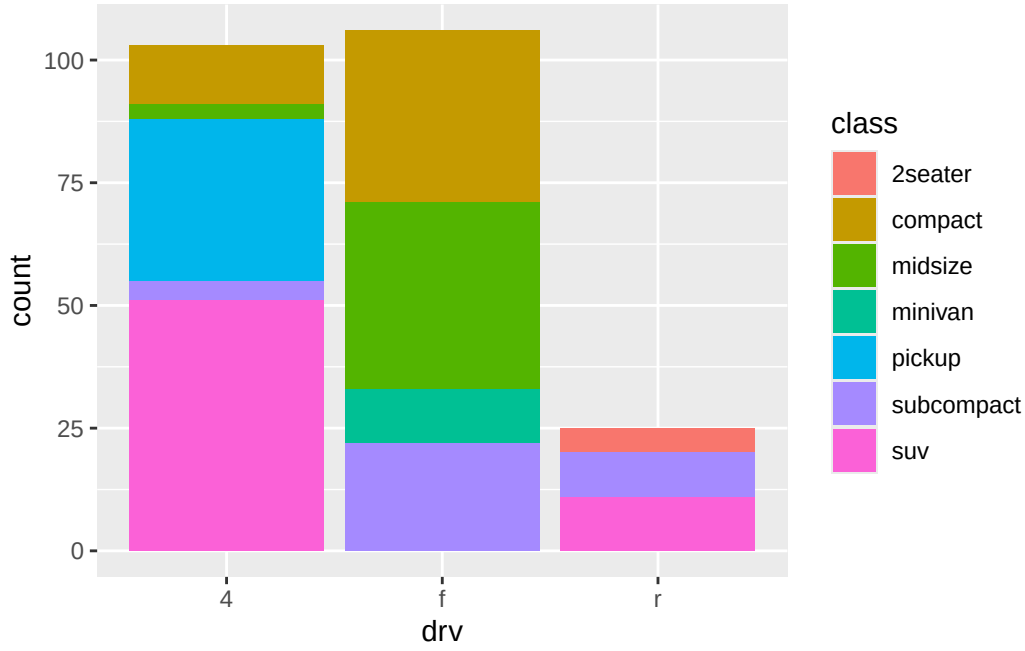
```
ggplot(mpg, aes(x = drv, color = drv)) +
  geom_bar()
```




```
ggplot(mpg, aes(x = drv, fill = drv)) +
  geom_bar()
```

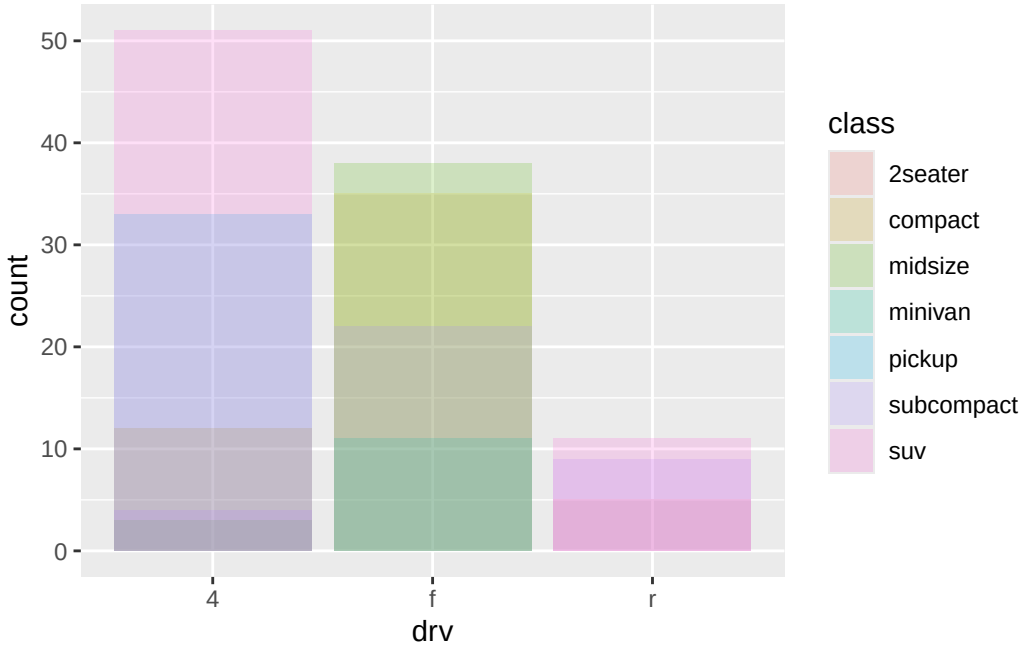


```
# stacked/ditto barplot
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar()
```

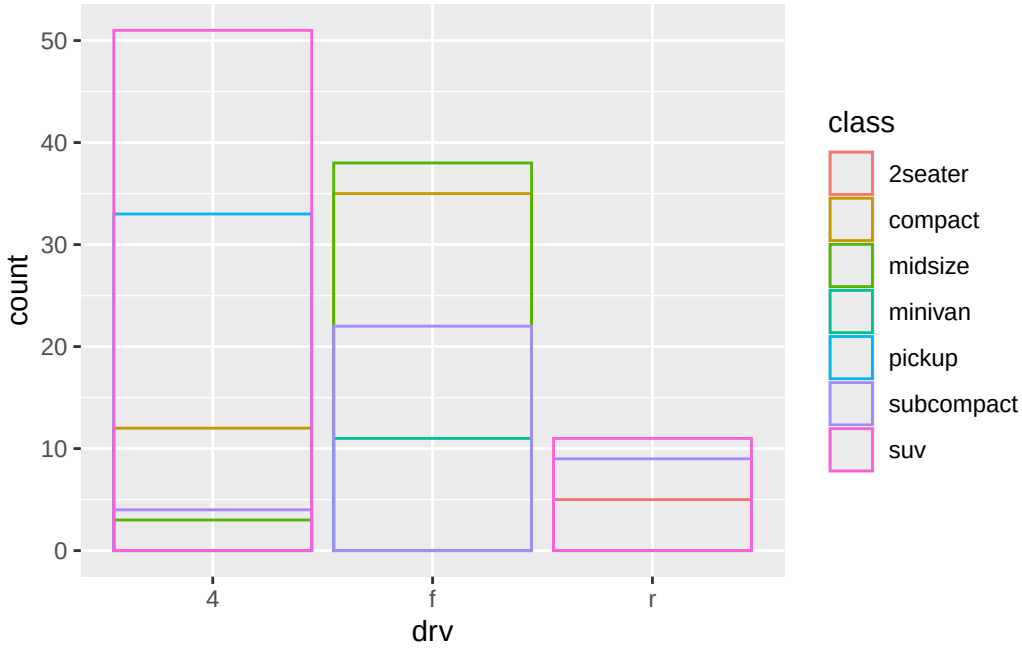


- arguments to `position =`:
  - `identity` places each object exactly where it falls in context of graph (if x axis variable is different than y axis, then get stacked bar plot)
  - `stack` absolute value barplot and stacked on top of each other
  - `fill` proportions barplot
  - `dodge` non stacked barplot (each x axis position gets multiple bars)
  - `jitter` adds a little randomness (not useful for barplots)
- `position_stack` in place of string `"stack"`, `position_fill` in place of string `"fill"`, etc. to access the description of each position

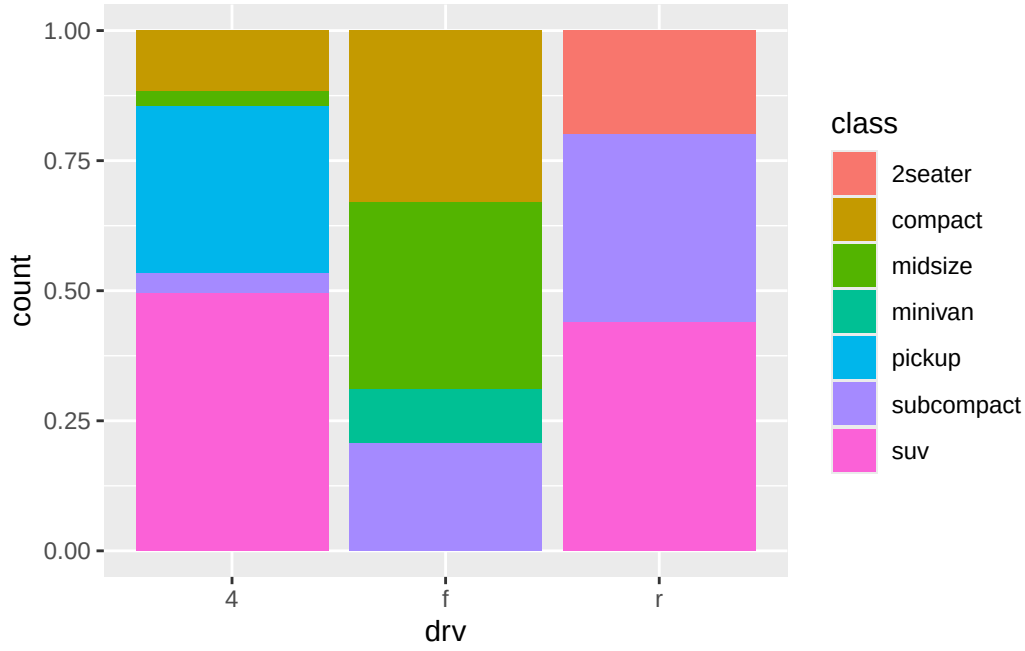
```
# increase transparency
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(alpha = 1 / 5, position = "identity")
```



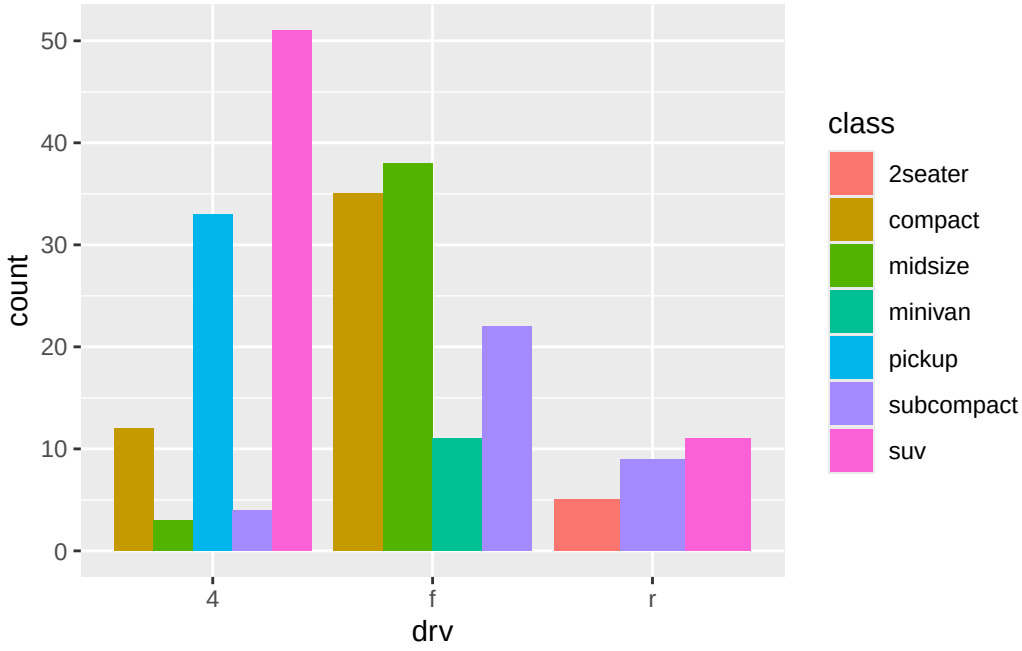

```
# completely transparent (no fill)
ggplot(mpg, aes(x = drv, color = class)) +
  geom_bar(fill = NA, position = "identity")
```



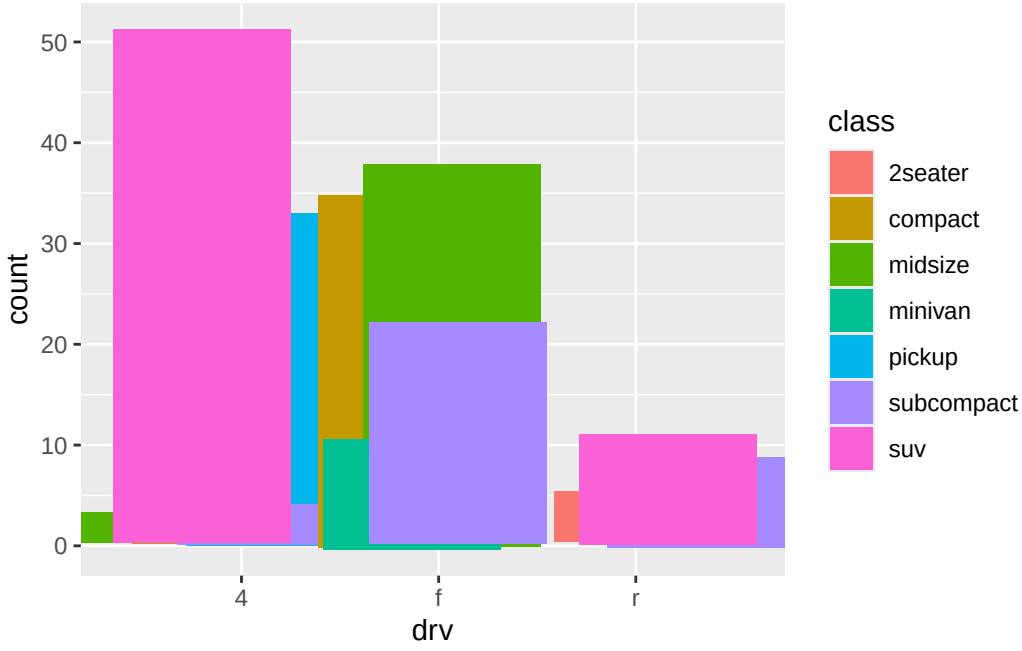
```
# proportions bar chart (easier to see proportions across groups)
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = "fill")
```




```
# see overlapping objects directly beside one another (easier to compare individual values wi
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = "dodge")
```




```
# not great for bar plots!
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = position_jitter(seed = 101))
```

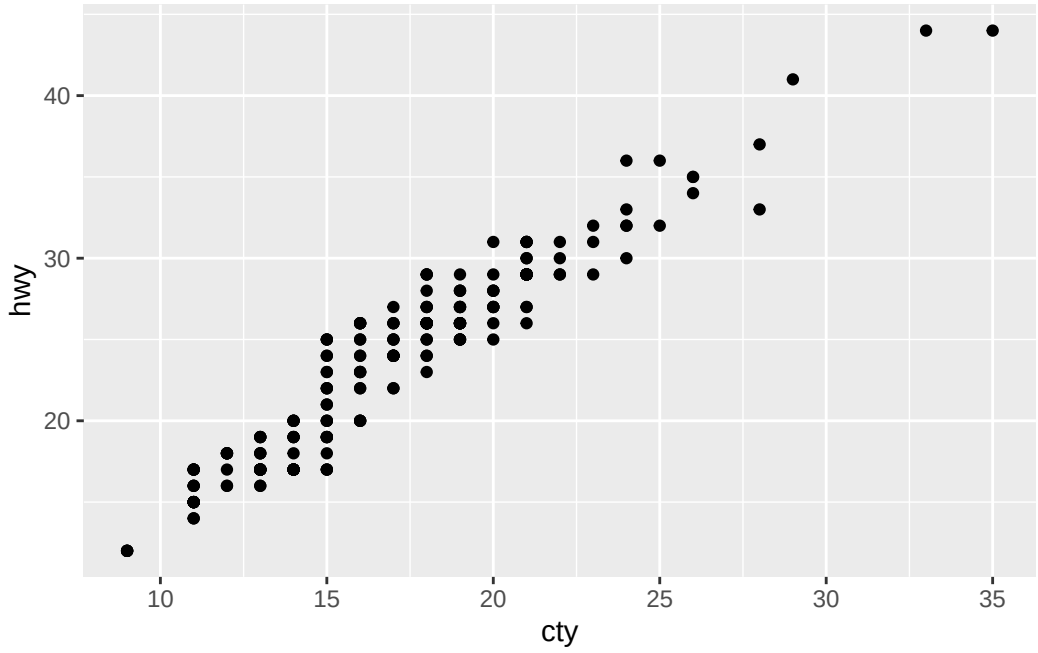


9.6.1 Exercises

1. What is the problem with the following plot? How could you improve it?

- answer: the points are lining up like they're on a grid, all along straight vertical lines since cty and hwy are all integers (rounded whole numbers for gas mileage). This leads to potential overplotting - many points may be overlapping a single (x,y) location. We can calculate this visually by counting or programmatically. The total number of points to be plotted is 234. We see that the number of unique (x,y) pairs is 78. That means that there 78 positions where black points are being plotted. So there's 234-78=156 points missing due to overplotting! We can improve this plot by adding a little bit of random noise to each point so that almost all, if not all, of the points are visible.

```
ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point()
```



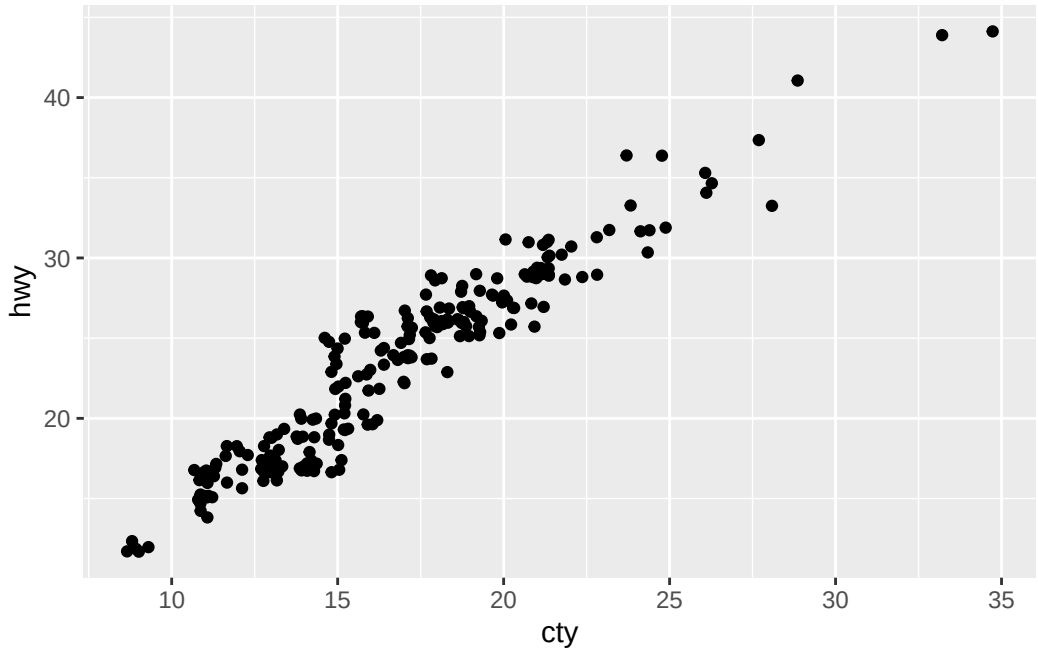

```
mpg |>
  mutate(x_y = paste0(cty, hwy)) |>
  distinct(x_y) |>
  nrow() # 78
```

```
[1] 78
```

```
mpg |> nrow() # 234
```

```
[1] 234
```

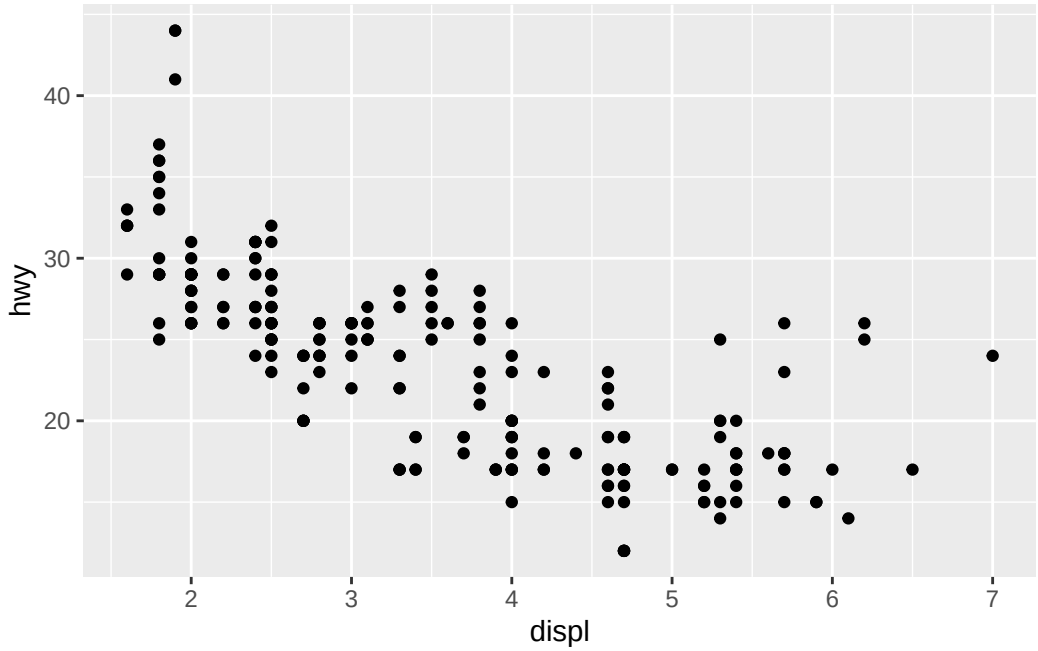
```
ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point(position = position_jitter(seed = 1))
```



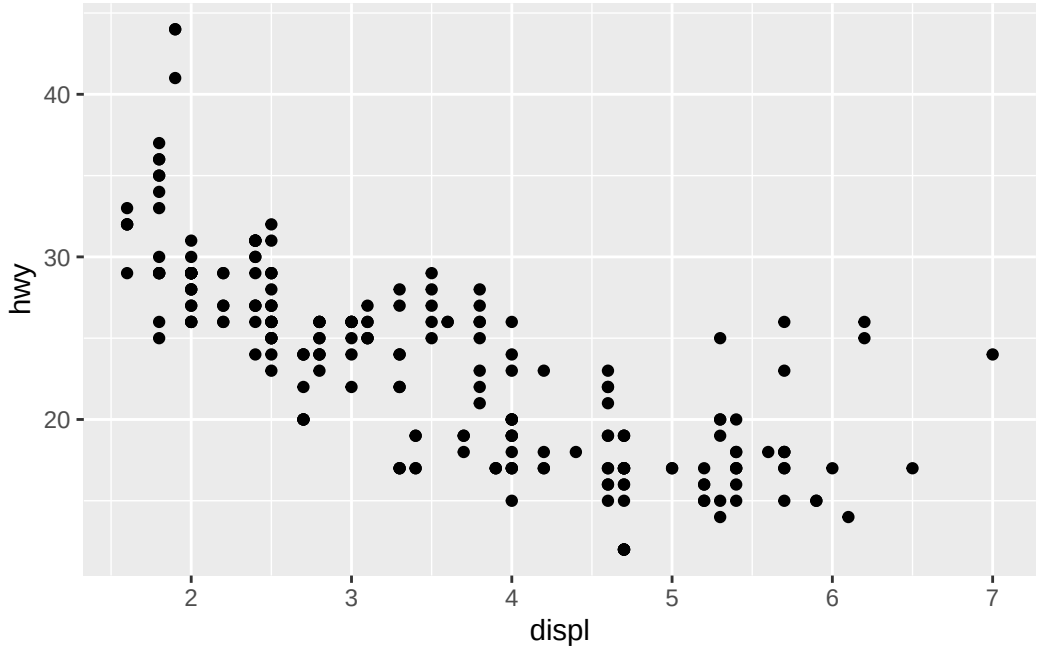
2. What, if anything, is the difference between the two plots? Why?

- answer: there is no difference between the two plots. they have the same exact inputs for their functions, the second plot has just explicitly made clear that `position = "identity"` is the default `position` of ggplot

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point()
```




```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(position = "identity")
```



```
# both of the below show the default argument value of poition = "identity"
?geom_point
geom_point
```

```
function (mapping = NULL, data = NULL, stat = "identity", position = "identity",
    ..., na.rm = FALSE, show.legend = NA, inherit.aes = TRUE)
{
    layer(mapping = mapping, data = data, geom = "point", stat = stat,
        position = position, show.legend = show.legend, inherit.aes = inherit.aes,
        params = list2(na.rm = na.rm, ...))
}
<bytecode: 0x115e042a8>
<environment: 0x106825af8>
```

3. What parameters to `geom_jitter()` control the amount of jittering?

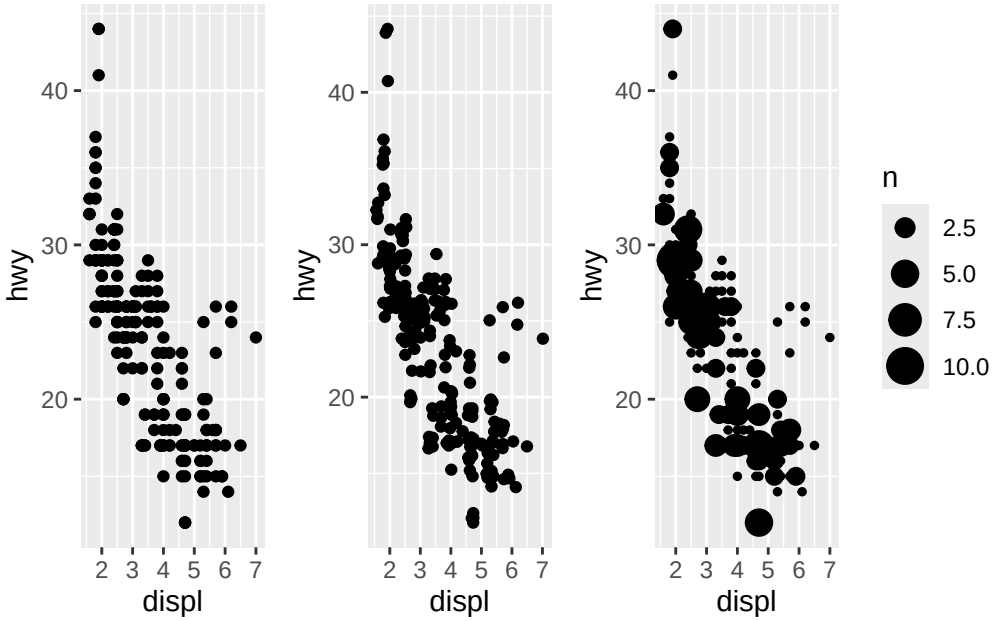
- answer: `width` and `height` control the amount of horizontal and vertical jitter in the 2D plot respectively. `seed` to make reproducible randomness for the PRNG.

```
?position_jitter # is the way to check
```

4. Compare and contrast `geom_jitter()` with `geom_count()`.

- answer: geom_jitter and geom_count can both be used to solve the issue of overplotting. geom_jitter solves the issue by adding random noise so that each point is moved slightly away from the (x,y) position it would overlap with other points. geom_count solves the problem by counting the number of points overlapping at each (x,y) position and making the dot size proportional to this count.

```
plot.9.6.1.4 <- ggplot(mpg, aes(x = displ, y = hwy)) +
geom_point() + # patchwork
ggplot(mpg, aes(x = displ, y = hwy)) +
geom_jitter() + # patchwork
ggplot(mpg, aes(x = displ, y = hwy)) +
geom_count()

plot.9.6.1.4
```

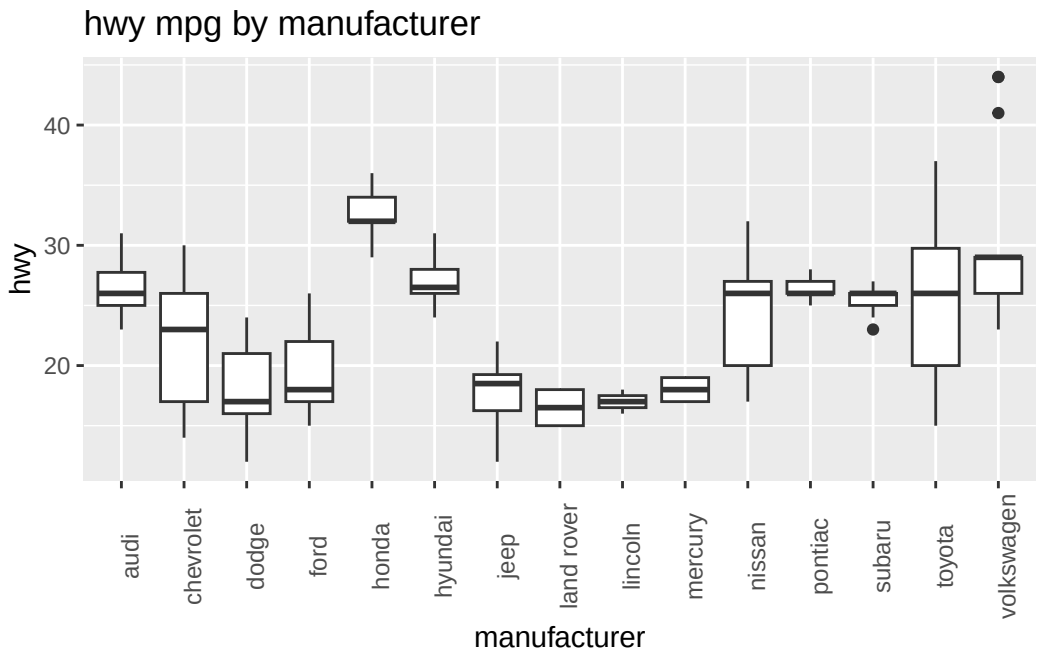



```
ggsave(
"./plots/9.6.1.4.plot.png",
plot.9.6.1.4,
height = 5,
width = 15
)
```

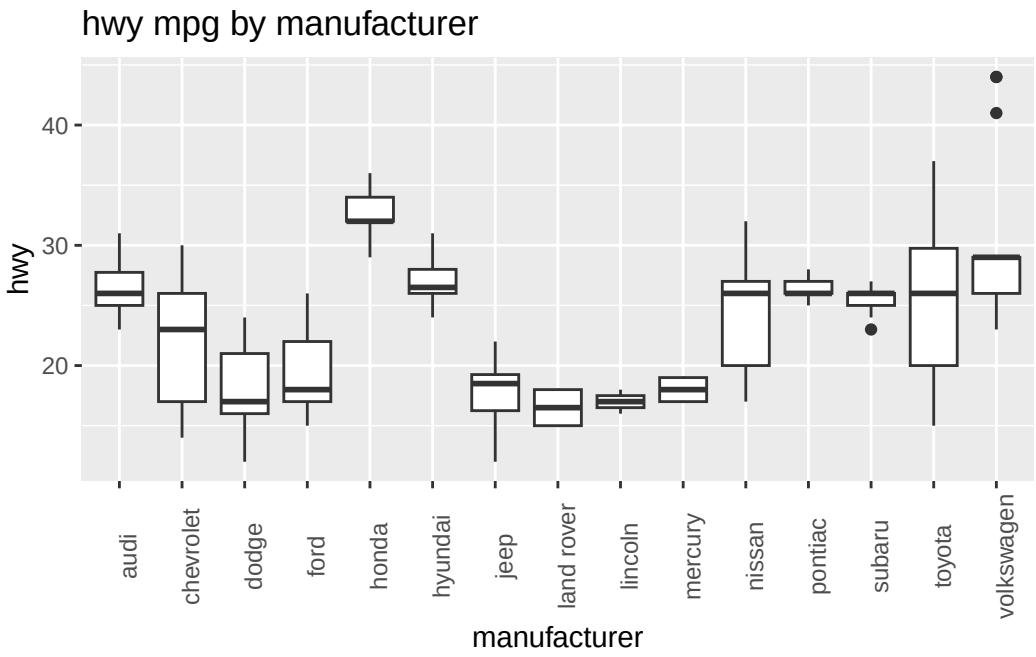
5. What's the default position adjustment for `geom_boxplot()`? Create a visualization of the mpg dataset that demonstrates it.

- answer: `position_dodge2()` which preserves vertical posiiton of a geom while adjusting the horizontal position.

```
?geom_boxplot

ggplot(mpg, aes(x = manufacturer, y = hwy)) +
  geom_boxplot() +
  theme(axis.text.x = element_text(angle = 90)) +
  ggtitle("hwy mpg by manufacturer")
```



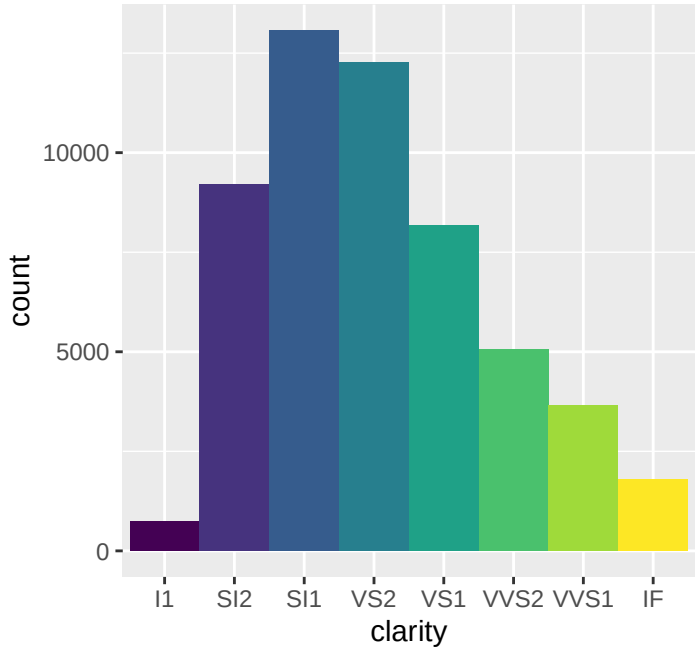

```
ggplot(mpg, aes(x = manufacturer, y = hwy)) +
  geom_boxplot(position = "dodge2") +
  theme(axis.text.x = element_text(angle = 90)) +
  ggtitle("hwy mpg by manufacturer")
```



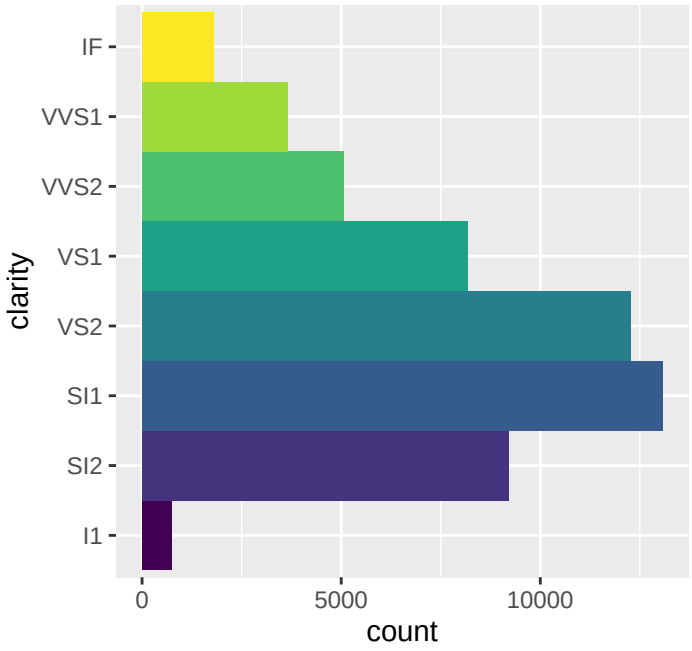
9.7 Coordinate systems

- coordinate systems change what `x` (horizontal axis) and `y` (vertical axis) mean
- 3 most useful coordinate systems in ggplot:
  - `coord_cartesian()`: classic 2d plot. for `aes()`: x axis and y axis (default)
  - `coord_sf()`: for making geographic map projections. for `aes()`: x=longitude, y=latitude (this func replaces `coord_map()`/`coord_quickmap()`)
  - `coord_polar()`: for making polar coordinate maps. for `aes()`: x=angle, y=radius
- fun fact: Coxcomb chart is essentially a bar chart in polar coordinates

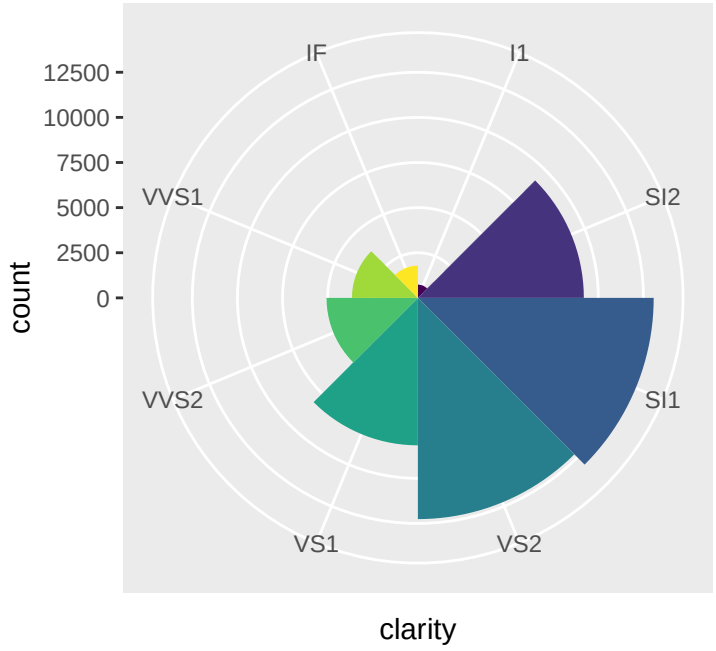
```
bar <- ggplot(data = diamonds) +
  geom_bar(
    mapping = aes(x = clarity, fill = clarity),
    show.legend = FALSE,
    width = 1
  ) +
  theme(aspect.ratio = 1)

bar
```




```
bar + coord_flip()
```



```
bar + coord_polar()
```

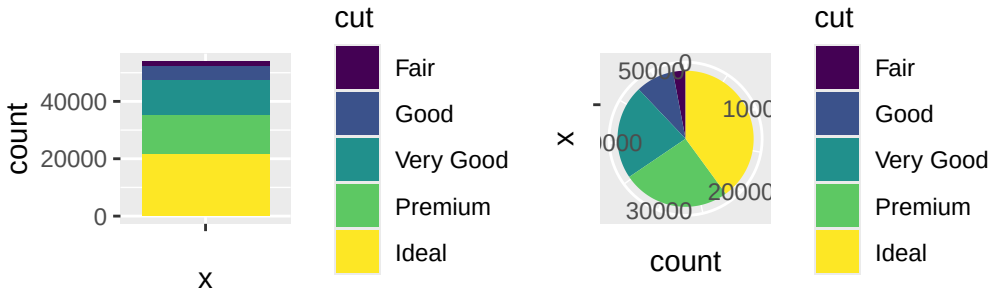


9.7.1 Exercises

1. Turn a stacked bar chart into a pie chart using `coord_polar`.

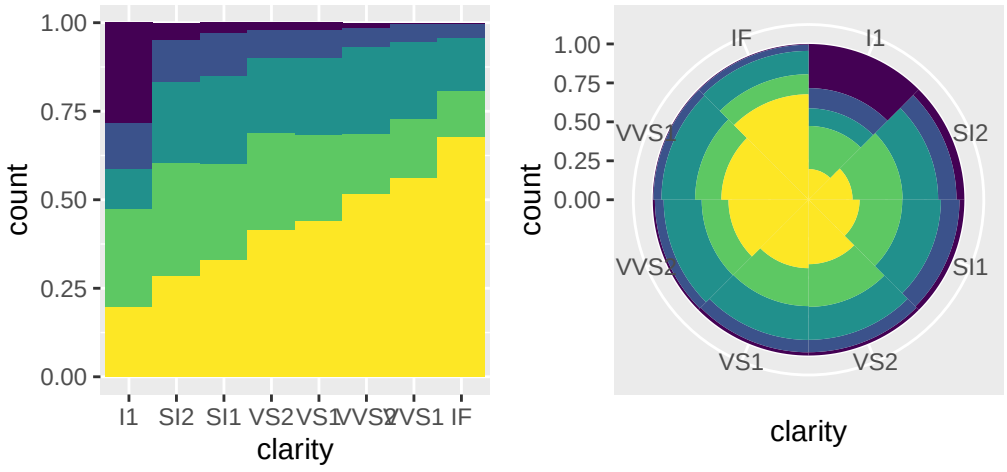
- answer:

```
# create a single bar sacked bar, so when it's converted there is only 1 color per each slic
ggplot(diamonds, aes(x = "", fill = cut)) +
  geom_bar() +
  ggplot(diamonds, aes(x = "", fill = cut)) +
  geom_bar() +
  coord_polar(theta = "y")
```



- wrong answer:

```
bar <- ggplot(data = diamonds) +
  geom_bar(
    mapping = aes(x = clarity, fill = cut),
    show.legend = FALSE,
    width = 1,
    position = "fill" # creates a proportion bar chart
  ) +
  theme(aspect.ratio = 1)

bar +
  bar + coord_polar()
```



2. What's the difference between `coord_quickmap()` and `coord_map()`?

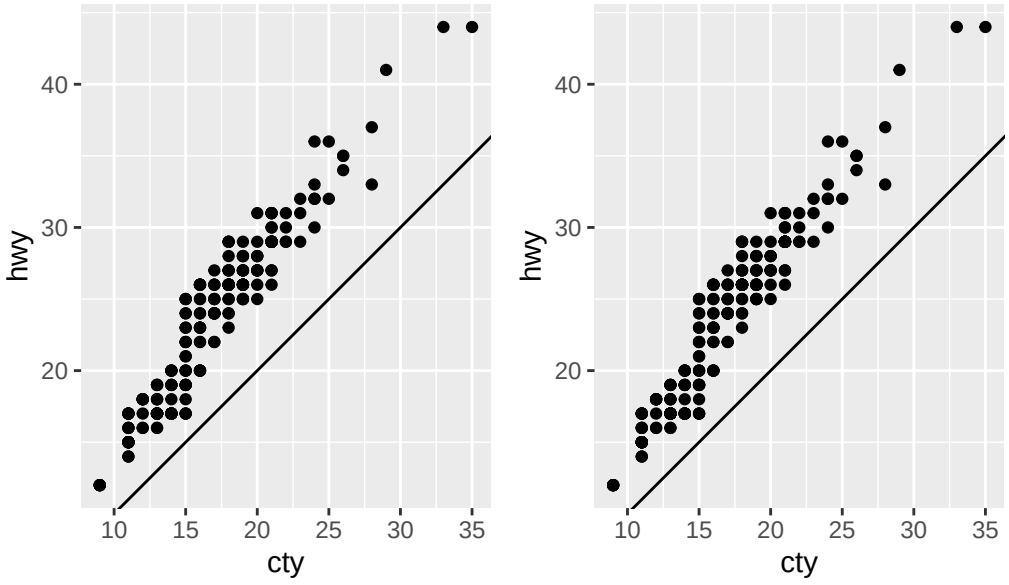
- answer: **note** `coord_sf()` supersedes both `coord_map()` and `coord_quickmap()`, the latter two should NOT be used in new code. `coord_map()` projects a portion of the earth onto a flat 2D plane using any projection defined by the package `mapproj`. Map projections generally do not preserve straight lines so this process takes a lot of computation. `coord_quickmap()` is a quick approximation that does preserve straight lines. `coord_quickmap()` is more accurate for smaller areas and closer to the equator.

3. What does the following plot tell you about the relationship between city and highway mpg? Why is `coord_fixed()` important? What does `geom_abline()` do?

- answer: city and highway mpg are positively and approximately linearly correlated. `geom_abline` adds a reference line (aka rule) to a plot for annotation and comparison. `coord_fixed` forces a specified ratio between the physical representation of data units on the axes. For example, 1 cm represents the same units on each. I'm not sure why `coord_fixed()` is important, as the result is the same even if I remove it. Since the coordinate system is fixed with a ratio of 1, we know the line represents y = x. As for `geom_abline`, since the points are above the line we know that `hwy` is always higher than `cty` for a given observation `(x, y) = (cty, hwy)`. In other words, for any car in our dataset, the highway mileage is always higher than the city mileage.

```
ggplot(data = mpg, mapping = aes(x = cty, y = hwy)) +
geom_point() +
```

```
geom_abline() +
coord_fixed() +
ggplot(data = mpg, mapping = aes(x = cty, y = hwy)) +
geom_point() +
geom_abline()
```



9.8 The layered grammar of graphics

given the seven parameters for the ggplot grammatical structure below, we can create *any* plot imaginable. add `theme()` and some form of `scale_*` and we can further customize plots a great deal.

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
     mapping = aes(<MAPPINGS>),
     stat = <STAT>,
     position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION>
```

9.9 Summary

Two very useful resources for getting an overview of the complete ggplot2 functionality are the ggplot2 cheatsheet (which you can find at https://posit.co/resources/cheatsheets) and the ggplot2 package website (https://ggplot2.tidyverse.org).